

AUDIT COMPLET · OCTOBRE 2026

FIT PULSE ***PLAN DE*** ***GLOW UP***

Analyse du code ligne par ligne sur huit chantiers, avec pour chaque point une analyse courte et des prompts prêts à l'emploi pour Claude Code et ChatGPT.

Préparé pour Kévin Guellec · 5 octobre 2026

SOMMAIRE

Synthèse

1. Direction artistique : en finir avec le style IA générique

2. Ergonomie et plaisir d'usage des commerciaux

3. Relances et CRM de rétention

4. Alertes et notifications

5. Fiabilité des chiffres

6. Pilotage managérial

7. Revenus : maximiser le chiffre d'affaires du club

8. Technique, sécurité, performance et RGPD

Feuille de route

Mode d'emploi : chaque point donne 2 à 3 lignes d'analyse, puis un prompt Claude à coller dans Claude Code avec le fichier HTML de Fit Pulse. Un prompt ChatGPT accompagne les points qui demandent un visuel sur mesure. Les prompts se lancent dans l'ordre de la feuille de route, en fin de document.

SYNTHÈSE

Le squelette de Fit Pulse est solide : imports Resamania reconnus par colonnes, objectifs, paliers d'équipe, rétention, impayés, résiliations. Pour devenir la meilleure appli du marché, il lui manque quatre choses : des chiffres irréprochables, des relances faites dans l'appli, des alertes qui partent vraiment, et une DA qui ne fait plus "IA générique".

Huit agents ont audité le code ligne par ligne, un par chantier. Chaque point du document suit le même format : 2 à 3 lignes d'analyse, puis un prompt Claude prêt à coller dans Claude Code avec le fichier de Fit Pulse, et un prompt ChatGPT quand un visuel sur mesure est utile.

CE QU'IL FAUT SAVOIR AVANT TOUT

- **DES CHIFFRES FAUX AUJOURD'HUI.** "1.234,56" est lu 1,23 ; un fichier XLSX inverse jour et mois ; annuler un import Resamania peut faire tomber 26 contrats à 7 ; une saisie manuelle et l'import du même fait comptent deux fois. Tous ces bugs ont été rejoués et confirmés.
- **DES FAILLES DE SÉCURITÉ CRITIQUES.** En mode Firebase, un membre peut lire et réécrire toute la base, se passer créateur ou tout effacer. En mode local, une ligne dans la console donne les droits créateur. Aucun effacement d'adhérent n'existe (RGPD).
- **AUCUNE ALERTE NE PART.** Tout est local et visible seulement appli ouverte. La préférence "Bilan hebdomadaire" n'est lue nulle part.
- **LES RELANCES SONT ÉCLATÉES.** Trois briques (Rétention, Impayés, Résiliations) qui ne se parlent pas, aucun lien d'appel, aucun téléphone importé depuis Resamania, et un J+15 réussi qui supprime le J+30.
- **LA DA TRAHIT L'OUTIL GÉNÉRÉ.** 67 tirets cadratins, 84 emojis, 82 pluriels "(s)", 3 couches de CSS empilées, deux jaunes différents, 33 tailles de police.
- **LE MANAGER PILOTE À L'AVEUGLE.** Pas de vue équipe par KPI, un statut de rythme qui affiche "en retard" un commercial en avance, et un barème où un shake à 30 € vaut un contrat.
- **L'ARGENT DORT DANS LES IMPORTS.** L'offre, le prix et le canal de chaque adhérent sont déjà importés mais jamais affichés. La courbe "CA" n'inclut pas les abonnements.

LES 10 CHANTIERS PRIORITAIRES

| Ordre | Chantier | Pourquoi maintenant | Section |

| --- | --- | --- | --- |

| 1 | Verrouiller la base (règles Firebase par club et par rôle, XSS, mode local) | Un membre peut tout effacer | 8 |

| 2 | Corriger nombres, dates et doubles comptes | Un chiffre faux détruit la confiance des équipes | 5 |

| 3 | Charte d'écriture et socle CSS unique | Base de toute la nouvelle DA, rapide à faire | 1 |

| 4 | Un seul modèle de relance, file d'appels, appel en un tap | C'est là que se gagne le revenu au quotidien | 3 |

| 5 | Accueil mobile "moi, maintenant" et saisie en 2 taps | Le plaisir d'usage des commerciaux se joue ici | 2 |

| 6 | Vue équipe manager et rythme juste | Le retour managérial demandé | 6 |

- | 7 | Socle valeur (revenu récurrent, valeur adhérent) et page Opportunités | Classer chaque action par euros attendus | 7 |
- | 8 | PWA, push et moteur de règles d'alertes | Les relances et sauvetages partent à l'heure | 4 |
- | 9 | Icônes et visuels sur mesure (niveaux, trophées, états vides) | Fin des emojis, identité Fitness Park | 1 |
- | 10 | Page Contrôle qualité et suite de tests | Chaque chiffre traçable jusqu'à sa ligne source | 5 |

1. DIRECTION ARTISTIQUE : EN FINIR AVEC LE STYLE IA GÉNÉRIQUE

Fichier audité : `fitpulse.html` , 4 474 lignes. CSS lignes 1 à 699, JS lignes 703 à 4 474.

État des lieux chiffré :

Signal "IA générique"	Occurrences	Où
Tiret cadratin — 67 au total : environ 51 dans le code de l'interface, 16 dans des commentaires toasts, libellés, cellules vides, options de select		
Tiret demi-cadratin – 0 rien à faire		
Point médian · 102 séparateur de méta partout, parfois 3 par ligne		
Emojis et pictos Unicode 84 glyphes sur 72 lignes, 39 emojis différents KPI, trophées, toasts, relances, imports		
Pluriels (s) 82 "saisie(s) enregistrée(s)", "jour(s) de suite"		
Couches CSS 3 (v1 ligne 7, v2 ligne 331, v3 ligne 470) :root déclaré 2 fois, mode clair 4 fois, h1 4 fois		
Tailles de police 33 valeurs en CSS + 16 en style="" inline de 8,5 px à 88 px		
Rayons d'angle 14 valeurs de 2 px à 99 px		
Attributs style="..." dans le JS 319 rendus HTML des pages		

1. CHARTE D'ÉCRITURE DE L'INTERFACE

Le ton est majoritairement au vouvoiement (17 occurrences de "vous / votre" dans les messages, "Prenez un dossier", "Saisissez votre e-mail"). Seul le bilan mensuel tutoie : `PAGES.wrap` , lignes 2651 à 2659 ("Ton score global", "Tes trophées", "Tu as tout donné"). On garde donc le vouvoiement partout, wrap compris.

Les tirets cadratins visibles servent de deux-points ou de séparateur : toast de saisie ligne 1003 (`${fullName(u)}` — `${fmtV(...)} — ${c.name}`), `OUTCOMES.ok` ligne 1445 ("Joint — OK"), bannière classement ligne 2199 ("Vous êtes #2 ce mois-ci — 640 pts"), options "— choisir —" lignes 2809, 3669, 3949, 4039, chemins Resamania lignes 3398 et 3603 ("... > Exporter — ou Exports de gestion"), notes d'historique lignes 3910 et 3954 (— « note »), et 15 cellules vides affichées — (`fmtP` ligne 768, `HEALTH.none` ligne 1763, `tierName` ligne 2523, récap lignes 4364 à 4416). Les emojis de ton sont dans les toasts : "Import terminé ✅" (2926, 3762), "saisie(s) enregistrée(s) 🔥" (2160), "Défi lancé ⚡" (2343), "Régulé : ajouté à vos impayés récupérés 📺" (3021), "Mission accomplie 🏆" (2109), "Objectif atteint ... 🎉" (1306), "Tous les paliers sont atteints 🎉" (4126), "Objectifs du mois tenus 🏆" (4248), et le titre de page "⚡ Défis flash" (2323). Les parenthèses à rallonge sont dans les messages d'aide : "Impayés en cours (photo du jour)" ligne 3658, "annulé / avoir (pas d'argent encaissé)" ligne 3656, "Lancez d'abord les exports de gestion (ils se préparent en fond, lien par e-mail)".

PROMPT CLAUDE (INTÉGRATION)

Contexte : fichier unique `fitpulse.html` (application Fit Pulse, cockpit commercial Fitness Park). Tu modifies uniquement les textes visibles par l'utilisateur, jamais les commentaires de code, les identifiants, les clés de données ni la logique.

Objectif : appliquer la charte d'écriture suivante à toute l'interface.

Règles de ton :

1. Vouvoiement partout. Réécris PAGES.wrap (lignes 2640 à 2700 environ) : "Ton score global" devient "Votre score global", "Tes trophées" devient "Vos trophées", "Tu as tout donné" devient "Vous avez tout donné", etc. Même chose dans le canvas de partage (fonction qui dessine #wrap-canvas).
2. Phrases courtes : 12 mots maximum dans un toast, 20 mots maximum dans une aide.
3. Verbes d'action à l'infinitif sur les boutons ("Enregistrer", "Relancer", "Noter un appel"), jamais "Valider" seul quand un verbe précis existe.
4. Toasts au passé composé, sans ponctuation finale décorative : "Saisie enregistrée", "Import terminé : 3 fichiers", "Défi lancé".
5. Chiffre d'abord quand il y en a un : "640 pts, 2e du club" plutôt que "Vous êtes 2e avec 640 pts".

Purges :

- A. Tiret cadratin U+2014 et demi-cadratin U+2013 : aucun dans le texte affiché.
- Séparateur de phrase : remplace par deux-points ou point. Ex. ligne 1003 : `\${fullName(u)} : +\${fmtV(...)} \${k.label}` puis `, \${c.name}` si club.
 - "Joint – OK" (OUTCOMES.ok) devient "Joint, OK".
 - Bannière classement ligne 2199 : "Vous êtes #2 ce mois-ci : 640 pts".
 - Options de select "– choisir –" et "– aucune –" : "Choisir..." et "Aucune".
 - Notes d'historique `« note »` : passer la note sur une seconde ligne en .muted, sans tiret.
 - Valeur absente : n'affiche aucun tiret. Utilise un petit trait CSS : `</span>` avec .nil { display:inline-block; width:12px; height:2px; background:var(--muted); vertical-align:middle; border-radius:1px }. Remplace dans fmtP, HEALTH.none.label (mettre 'Sans donnée'), tierName, delta() et tile() du récap, tables impayés et résiliations, options.
- B. Emojis : aucun emoji dans le texte affiché. Supprime ceux des toasts (lignes 1306, 2109, 2160, 2343, 2926, 3021, 3762, 4126, 4248) et du titre "< Défis flash" ligne 2323. Les emojis porteurs de sens (KPI, trophées, relances, imports, réactions) seront remplacés par des icônes SVG dans un autre lot : pour l'instant, ne casse pas les champs emoji/icon des objets de données.
- C. Point médian : : garde-le uniquement comme séparateur de métadonnées, un seul par ligne. Au-delà, passe à la ligne ou utilise une virgule. Ne touche pas aux formes inclusives "Champion·ne" et "Meilleur·e commercial·e" : remplace-les par "Champion de la semaine" / "Championne de la semaine" selon u.gender si dispo, sinon par "N°1 de la semaine" et "N°1 du mois".
- D. Pluriels "(s)" : remplace par une fonction plur(n, 'saisie', 'saisies') qui accorde. 82 occurrences à traiter, dont "saisie(s) enregistrée(s)", "jour(s) de suite", "fichier(s)", "résiliation(s)", "impayé(s)", "TROPHÉE(S)".
- E. Parenthèses explicatives : aucune parenthèse de plus de 4 mots dans un libellé. Transforme en sous-ligne .muted.small ou en attribut title. Exemples : "Impayés en cours (photo du jour)" devient "Impayés en cours" + sous-ligne "Situation du jour" ; "annulé / avoir (pas d'argent encaissé)" devient "Annulé ou avoir" + title "Aucun encaissement".

Critères d'acceptation vérifiables :

- `grep -c "--" sur le fichier ne renvoie que des occurrences situées dans des commentaires (lignes commençant par // ou /*). Idem pour "--".
- Une regex Unicode des emojis (plages U+1F300 à U+1FAFF et U+2600 à U+27BF) ne trouve plus rien dans les chaînes de toast(), toastUndo(), celebrate() ni dans les titres h1.
- `grep -c "(s)"` renvoie 0 hors commentaires.
- `grep -n "Ton \|Tes \|Tu "` ne renvoie aucune chaîne affichée.
- Aucune régression : l'app se charge, les toasts s'affichent, le select Responsable fonctionne.

2. UN SEUL SOCLE CSS : FUSIONNER V1, V2 ET V3

Trois couches se surchargent. :root est posé ligne 9 puis ligne 474, le mode clair est écrit 4 fois (lignes 30, 41, 483, 484), h1 reçoit 4 tailles (30 px ligne 58, letter-spacing ligne 333, 40 px ligne 490, 32 px ligne 647). Conséquences visibles : le jaune existe en deux teintes, --fp: #FFD200 (v1) et #FFD600 (v3), et 12 couleurs #FFD200 restent codées

en dur dans le JS (confettis `celebrate()` ligne 4133, `barChart` ligne 2081, `RECOV_CHANNELS.equipe` ligne 3245, mascotte ligne 1553, canvas du wrap lignes 2686 à 2689). Les badges `.badge.ok/.warn/.bad` (lignes 127 à 129) teintent avec les verts et rouges de la v1 (`rgba(31,157,85)` , `rgba(217,58,43)`) alors que `--ok` vaut `#22C55E` en v3. `--font-title` v1 vaut 'Anton', police jamais chargée (ligne 5 ne charge que Barlow Condensed et Montserrat), mais toujours appelée par le canvas du wrap et la mascotte.

PROMPT CLAUDE (INTÉGRATION)

Contexte : `fitpulse.html` contient trois couches CSS successives dans le même `<style>` : v1 (lignes 7 à 330), v2 identité (lignes 331 à 467), v3 (lignes 469 à 698). Elles se surchargent. Tu dois produire UNE feuille propre, sans changer le rendu voulu par la v3.

Étapes :

1. Crée en tête du `<style>` un bloc unique de tokens :

```
:root {
  --fp-yellow: #FFD600; --fp-yellow-press: #E6C200; --fp-black: #0A0A0A; --fp-white: #FFFFFF;
  --fp: var(--fp-yellow); --fp-ink: var(--fp-black); --fp-soft: rgb(255 214 0 / .14);
  --bg: #0A0A0A; --surface: #141414; --surface-2: #1D1D1D; --line: #262626; --text: #F5F5F3; --muted:
#9B9B9B;
  --ok: #22C55E; --warn: #F59E0B; --bad: #EF4444; --info: #3B82F6;
  --ok-soft: rgb(34 197 94 / .14); --warn-soft: rgb(245 158 11 / .16); --bad-soft: rgb(239 68 68 /
.15); --info-soft: rgb(59 130 246 / .14);
  --accent-text: var(--fp);
  --font-title: 'Barlow Condensed', Impact, sans-serif; --font: 'Montserrat', system-ui, sans-serif;
  --r-xs: 4px; --r-sm: 8px; --r-md: 12px; --r-lg: 16px; --r-xl: 20px; --r-pill: 999px;
  --s-1: 4px; --s-2: 8px; --s-3: 12px; --s-4: 16px; --s-5: 24px; --s-6: 32px; --s-7: 48px;
  --shadow-1: 0 1px 2px rgb(0 0 0 / .4); --shadow-2: 0 8px 24px rgb(0 0 0 / .35);
  color-scheme: dark;
}
Puis un seul bloc clair, écrit deux fois comme l'exige le contrat de thème :
@media (prefers-color-scheme: light) { :root:not([data-theme="dark"]) { ...valeurs claires... } }
:root[data-theme="light"] { ...mêmes valeurs claires... }
Valeurs claires : --bg #F3F3F0, --surface #FFFFFF, --surface-2 #F1F1EE, --line #E4E4DE, --text
#0E0E0E, --muted #66666B, --ok #16A34A, --warn #B45309, --bad #DC2626, --accent-text #7A5F00.
```

2. Supprime les déclarations `:root`, `@media light` et `[data-theme="light"]` des lignes 9 à 51 et 474 à 484.

3. Pour chaque sélecteur défini plusieurs fois (`h1`, `h2`, `h3`, `.card`, `.btn.primary`, `.brand-mark`, `.brand-name`, `.nav a`, `.nav a.on`, `.side`, `.palier-track`, `.palier-msg`, `.page-head h1::before`, `.stat b`, `.ck b`, `.podium .block`), fusionne en une seule règle qui reproduit le rendu final actuel (la dernière déclaration gagne), puis supprime les doublons.

4. Remplace toutes les couleurs `rgba()` codées à partir des anciennes valeurs (31,157,85 / 217,58,43 / 224,138,0 / 47,111,219 / 255,210,0) par les tokens `-soft` correspondants. Les `.badge.ok/.warn/.bad/.info` utilisent `--ok-soft` etc.

5. Dans le JS, remplace chaque `'#FFD200'` par `'var(--fp)'` quand la valeur sert dans du HTML/SVG, et par `getComputedStyle(document.documentElement).getPropertyValue('--fp').trim()` quand elle sert dans un canvas (wrap, lignes 2686 à 2690). Remplace `'Anton, Impact'` par `'800 italic <taille> "Barlow Condensed", Impact'` dans le canvas et `font-family="Anton,Impact"` par `font-family="Barlow Condensed"` dans `mascot()`.

6. Ordonne la feuille en sections commentées : 1 Tokens, 2 Base, 3 Typo, 4 Layout, 5 Composants, 6 Pages, 7 Responsive, 8 Print.

Critères d'acceptation :

- ``grep -c ":root {"`` = 1 ; ``grep -c 'data-theme="light"'`` = 1 dans le CSS (hors JS).

- ``grep -ci "#FFD200"`` = 0 ; ``grep -c "Anton"`` = 0.

- ``grep -c "^h1 {\|^h1,"`` : une seule règle définit la taille de `h1`.

- Le fichier CSS fait au moins 25 % de lignes en moins qu'avant.

- Captures avant/après de l'accueil, du classement et des impayés, en sombre et en clair : aucune différence visible autre que le jaune unifié.

3. ÉCHELLE TYPOGRAPHIQUE ET FIN DES STYLES INLINE

La CSS utilise 33 tailles différentes, dont des demi-pixels (10,5 / 11,5 / 12,5 / 13,5 px) et un texte de graphique à 8,5 px (`.chart text` ligne 190, illisible sur téléphone). Le JS ajoute 319 attributs `style=""`, avec 16 tailles en dur : `font-size:18px` 11 fois et `font-size:22px` 11 fois, surtout dans `PAGES.wrap` (2653 à 2659) et les en-têtes de profil (`h1` `style="font-size:26px"` ligne 2599). Les libellés en capitales (`.stat span`, `.ck2-h span`, `.bk span`, `.rc-tile span`, `.eyebrow`) ont chacun leur letter-spacing (.06em, .1em, .14em).

PROMPT CLAUDE (INTÉGRATION)

Contexte : `fitpulse.html`, CSS déjà fusionnée en une couche (voir lot "socle CSS"). Tu poses une échelle typographique et tu retires les tailles inline.

Échelle à créer dans `:root` :

```
--t-display: clamp(44px, 6vw, 64px); /* bannière d'accueil, célébration */
--t-h1: 40px; --t-h2: 28px; --t-h3: 20px; /* Barlow Condensed 800 italique, capitales */
--t-num-xl: 56px; --t-num-l: 40px; --t-num-m: 28px; /* chiffres clés, Barlow Condensed 800 italique, font-variant-numeric: tabular-nums */
--t-body: 14px; --t-small: 12px; --t-label: 11px; /* Montserrat */
--lh-tight: .95; --lh-body: 1.5;
--ls-label: .1em;
```

Sur mobile (max-width: 860px) : `--t-h1 32px`, `--t-h2 24px`, `--t-num-xl 44px`, `--t-num-l 32px`.

Règles :

1. Toute taille de police du CSS doit pointer vers une de ces variables. Supprime les valeurs à demi-pixel.
2. Crée trois classes utilitaires : `.num-xl`, `.num-l`, `.num-m` (Barlow 800 italique, tabular-nums, line-height .95) et `.label` (Montserrat 700, `var(--t-label)`, uppercase, letter-spacing `var(--ls-label)`, color `var(--muted)`).
3. Applique `.label` à `.stat span`, `.ck span`, `.ck2-h span`, `.bk span`, `.rc-tile span`, `.eyebrow` et supprime leurs déclarations propres.
4. Texte des graphiques : 11px minimum (`.chart text`, `.c-tick`). Aucune valeur sous 11px dans tout le fichier.
5. Dans le JS, remplace chaque `style="font-size:XXpx"` par la classe correspondante. Priorité : `PAGES.wrap` (lignes 2640 à 2700), `PAGES.profile` (lignes 2596 à 2630), `PAGES.dashboard` (1940 à 2100), `PAGES.leaderboard` (2190 à 2210).
6. Les styles inline de mise en page récurrents (`style="padding:10px 0;border-bottom:1px solid var(--line)"` apparaît sur les listes) deviennent une classe `.list-row`.

Critères d'acceptation :

- ``grep -o "font-size: *[0-9.]*px" `` dans le CSS ne renvoie que les lignes de définition des variables.
- ``grep -c 'style="font-size'"`` dans le JS = 0.
- Le nombre total de ``style=""`` dans le JS passe de 319 à moins de 120.
- Aucune taille calculée inférieure à 11px (vérifier dans l'inspecteur sur l'accueil et le classement en 375px de large).

4. COULEURS, CONTRASTE ET VRAI MODE CLAIR

Le jaune #FFD600 sur blanc a un contraste de 1,3:1. En mode clair, il est pourtant utilisé en trait de 3,5 px pour la courbe de course au palier (.chart .c-line ligne 570), en remplissage des barres de récap (.rc-bars i.cur ligne 669) et en bord de toast. La v3 ne redéfinit pas --accent-text en clair : seul le v1 le passe à #8A6D00, valeur qui disparaît si on nettoie mal. La barre latérale reste noire en clair (.side { background: #000 } ligne 499), la tabbar aussi (#0B0B0C ligne 445), et la carte Paliers est forcée en noir (.paliers ligne 385, .paliers.col4 ligne 579). C'est défendable comme signature Fitness Park, à condition d'en faire une règle explicite plutôt qu'un accident.

PROMPT CLAUDE (INTÉGRATION)

Contexte : fitpulse.html, tokens déjà unifiés. Tu fixes les règles d'usage du jaune et le mode clair.

Règles de marque à appliquer :

1. Le jaune est une surface, pas une couleur de texte sur fond clair. En mode clair, tout texte "accent" utilise --accent-text (#7A5F00, contraste >= 4.5:1 sur blanc). En sombre, --accent-text = --fp.
2. Texte posé sur jaune : toujours --fp-ink (#0A0A0A). Jamais de blanc sur jaune.
3. Zones "signature" toujours noires dans les deux thèmes : .side, .tabbar, .banner, .paliers, .auth-visual, .celebrate. Crée une classe .zone-black qui redéfinit localement les tokens (--surface: #141414; --text: #F5F5F3; --muted: #9B9B9B; --line: #262626; --accent-text: var(--fp)) et applique-la à ces zones, à la place des couleurs codées en dur (#000, #0B0B0C, #0F0F11, #26262a, #9a9aa0, #c9c9cc, #8d8d92, #5a5a60).
4. Graphiques en mode clair : la série principale garde le jaune en remplissage, mais reçoit un contour 1px var(--fp-ink) à 25 % d'opacité ; la courbe .c-line passe à stroke var(--text) avec une aire .c-area jaune à .35 d'opacité. Le point courant .c-dot reste jaune avec contour --fp-ink.
5. Barres de récap (.rc-bars i.cur) et .rc-cell i : même traitement que la série principale.
6. Ajoute un sélecteur de thème visible (soleil / lune, déjà dans ICONS) qui pose data-theme sur <html> et le mémorise en localStorage dans un try/catch.

Critères d'acceptation :

- Contraste mesuré (outil de l'inspecteur) >= 4.5:1 pour tout texte, >= 3:1 pour tout trait de graphique, dans les deux thèmes.
- `grep -c "#000\b|#0B0B0C|#0F0F11|#26262a"` dans le CSS hors bloc de tokens = 0.
- En thème clair, la barre latérale et la bannière restent noires, toutes les cartes de contenu sont blanches.

5. UNE SEULE ICONOGRAPHIE : REMPLACER LES 39 EMOJIS PAR DES ICÔNES TRAIT

Le fichier possède déjà un bon jeu d'icônes trait maison : ICONS ligne 803, 39 pictos 24x24, ico() ligne 843, stroke 2, bouts arrondis. Mais les objets métier embarquent des emojis qui s'affichent à côté : DEFAULT_KPIS lignes 849 à 857 (emoji étoile, gobelet, main qui écrit, casquette, billet, immeuble, ticket, bouée, aimant), LOYALTY_TYPES lignes 1438 à 1442 (téléphone, flèches, gâteau, billet, banque), todoList() lignes 4194 à 4196 (porte, billet), résumé d'import lignes 3656 à 3661 (billet, silhouettes, alerte, banque, porte, loupe), réactions du fil ligne 2236 (flamme, biceps, applaudissements) et du chat ligne 2262. Le marqueur "KPI obligatoire" est une couronne emoji répétée 7 fois (lignes 2021, 2201, 2507, 2510, 3097, 3098). Un emoji se rend différemment sur iPhone, Android et Windows : la DA n'est jamais maîtrisée.

PROMPT CLAUDE (INTÉGRATION)

Contexte : fitpulse.html possède un jeu d'icônes trait ICONS (ligne 803) et ico(n, cls) (ligne 843). Tu remplaces TOUS les emojis porteurs de sens par des icônes de ce jeu, dessinées dans le même style.

Style imposé pour les nouvelles icônes : viewBox 0 0 24 24, stroke currentColor, stroke-width 2, stroke-linecap round, stroke-linejoin round, fill none, formes géométriques simples, pas de détail sous 2px, pas de remplissage sauf un point de 1px de rayon.

1. Ajoute à ICONS : star, cup (shaker nutrition), pen (contrat signé), cap (accessoires), coins (impayés récupérés), briefcase (B2B), ticket (invités), lifebuoy (sauvetage), magnet (prospects), crown (obligatoire), cake (anniversaire), repeat (renouvellement), bank (sans mandat), flame, medal, lock, flag (défi), sparkle (célébration), alert (triangle), calendar-check (régularité).

2. Dans DEFAULT_KPIS, renomme le champ emoji en icon et mets le nom d'icône : avis 'star', nutrition 'cup', contrats 'pen', accessoires 'cap', impayés 'coins', b2b 'briefcase', invités 'ticket', sauvetage 'lifebuoy', prospects 'magnet'. Ajoute une migration au chargement : si un KPI stocké possède emoji et pas icon, déduis icon de son id, sinon 'target'.

3. Crée kpiIcon(k, cls) qui renvoie ico(k.icon || 'target', cls). Remplace tous les usages de k.emoji (lignes 1003, 1369, 2021, 2655, 4123 et autres) par kpiIcon(k).

4. LOYALTY_TYPES : icon devient 'phone', 'repeat', 'cake', 'coins', 'bank'. todoList : 'door', 'coins', puis LOYALTY_TYPES[...].icon, rendus par ico() dans .todo-i.

5. Résumé d'import (lignes 3656 à 3661) : coins, users, alert, bank, door, search.

6. Marqueur obligatoire : remplace la couronne par `<span class="req" title="Compte dans le classement global">${ico('crown','ico ico-xs')}</span>` ; légende textuelle "Obligatoire" dans les en-têtes de tableau.

7. Réactions du fil et du chat : garde les clés de données existantes ('🔥','👉','👎','👍') pour ne pas casser l'historique, mais affiche flame, medal, sparkle, check via une table REACT_ICON.

8. Tailles : .ico 16px par défaut, .ico-xs 12px, .ico-lg 24px, .ico-xl 32px. Pastille d'icône KPI : .kpi-ico { width:32px; height:32px; border-radius:var(--r-sm); background:var(--fp-soft); color:var(--accent-text); display:grid; place-items:center }.

9. Dans l'écran Réglages > KPI (ligne 3106, ACTIONS.kpiNew), remplace le champ emoji libre par une grille de choix d'icônes.

Critères d'acceptation :

- Regex emoji (U+1F300 à U+1FAFF, U+2600 à U+27BF, U+2B50) : aucun résultat hors clés de réaction historiques et commentaires.
- Toutes les icônes ont la même épaisseur visuelle à 16px (contrôle visuel sur la page Imports et sur l'accueil).
- Un KPI créé avant migration s'affiche avec une icône, jamais vide.

6. BADGES DE NIVEAU ROOKIE, PERFORMER, WARRIOR, ÉLITE, LÉGENDE

LEVELS (ligne 870) définit 5 niveaux par points cumulés, mais le profil ne les montre que comme une frise de pastilles grises (.levels lignes 270 à 275) et un badge fp texte (ligne 2598). Un second vocabulaire concurrent existe : tierName() ligne 2523 renvoie ELITE, PRO, RISING, STARTER selon le score du mois, réutilisé dans le canvas du wrap (ligne 2690). "Élite" existe donc deux fois avec deux sens. Il faut un seul système de niveaux, illustré, et renommer les paliers mensuels (par exemple "Mois en or, argent, bronze").

PROMPT CHATGPT (VISUEL)

Crée une série de 5 badges de niveau pour une application commerciale de salle de sport, marque Fitness Park (identité jaune et noir, sportive, énergique, premium).

Niveaux, du plus bas au plus haut : ROOKIE, PERFORMER, WARRIOR, ÉLITE, LÉGENDE.

Style : emblème vectoriel plat, géométrique, forme d'écusson incliné à 8 degrés vers la droite (effet italique), bords nets, traits épais réguliers. Inspiration : badges de ligue sportive et insignes militaires modernes, pas de style cartoon.

Palette stricte : jaune #FFD600, noir #0A0A0A, blanc #FFFFFF, gris #9B9B9B. Aucune autre couleur, pas de dégradé arc-en-ciel, pas de métal réaliste.

Progression visible : ROOKIE écusson simple gris et blanc ; PERFORMER une bande jaune ; WARRIOR deux bandes et chevron ; ÉLITE écusson noir à liseré jaune avec étoile ; LÉGENDE écusson jaune plein avec couronne stylisée et éclats.

Le nom du niveau en capitales condensées italiques type Barlow Condensed ExtraBold Italic, intégré dans un bandeau sous l'écusson.

Format : 5 fichiers PNG séparés, 512 x 512 px, fond transparent, badge centré avec 32 px de marge, même taille optique et même épaisseur de trait sur les 5.

Lisible à 32 px : silhouettes distinctes, pas de petits détails.

À ne pas faire : pas d'emoji, pas de texte autre que le nom du niveau, pas d'ombre portée réaliste, pas de 3D, pas de reflets, pas de visage ni personnage, pas de logo Fitness Park.

PROMPT CLAUDE (INTÉGRATION)

Contexte : fitpulse.html. LEVELS (ligne 870) liste 5 niveaux. J'ajoute 5 visuels de badge dans assets/levels/ : rookie.png, performer.png, warrior.png, elite.png, legende.png (512x512, fond transparent).

À faire :

- Ajoute à LEVELS un champ `img: 'assets/levels/<id>.png'`. Crée `levelBadge(level, size)` qui renvoie `">`, avec repli SVG inline (écusson incliné jaune et noir dessiné en `<path>`, nom du niveau en Barlow 800 italique) si l'image ne charge pas (onerror).
- Profil (PAGES.profile, ligne 2596) : remplace `<span class="badge fp">${lv.label}</span>` par `levelBadge(lv, 72)` à gauche du nom. Remplace la frise `.levels` par 5 badges à 40px : niveaux atteints en couleur, niveau suivant à 60 % d'opacité avec barre de progression jaune dessous ("1 240 pts avant Warrior"), niveaux lointains en niveaux de gris (filter: grayscale(1) opacity(.35)).
- Classement (PAGES.leaderboard, lignes 2198 à 2207) : badge 20px à côté du nom de chaque membre.
- Supprime le vocabulaire concurrent : `tierName()` (ligne 2523) devient `monthTier()` et renvoie 'OR', 'ARGENT', 'BRONZE', 'EN COURS' (score >= 1, >= .75, >= .5, sinon). Mets à jour le canvas du wrap (ligne 2690) et le tableau du bilan (ligne 2537).
- Passage de niveau : quand `allTime(ME.id)` franchit un min de LEVELS après une saisie, appelle `celebrate()` avec le badge (voir lot célébration).

Critères d'acceptation :

- Le mot "ELITE" sans accent n'apparaît plus ; "Élite" n'apparaît que dans LEVELS.
- Les 5 badges s'affichent à 40px sans flou sur écran Retina (img en 512px).
- Image manquante : le repli SVG s'affiche, aucune icône cassée.

7. TROPHÉES ET MÉDAILLES DE PODIUM

`allTrophies()` (lignes 1356 à 1395) attribue 6 sortes de trophées représentées par des emojis : coupe pour le meilleur du mois (1365), emoji du KPI ou médaille militaire pour le KPI du mois (1369), couronne pour le champion de saison (1378), médailles or, argent, bronze pour le podium hebdo (1385), éclair pour un défi flash (1392). Le profil ajoute 3 accomplissements emoji avec cadenas (lignes 2614 à 2616 : calendrier, "100", médaille). Le wrap aligne les

emojis bout à bout (`tr.map(t => t.icon).join(' ')`, ligne 2658). Le podium du classement (`.podium .block`, lignes 194 à 200) n'est qu'un rectangle avec un chiffre.

PROMPT CHATGPT (VISUEL)

Crée une collection cohérente de 12 trophées et médailles pour l'application commerciale d'une chaîne de salles de sport (identité jaune et noir, sportive, premium).

Liste : 1 MÉDAILLE OR (1er de la semaine), 2 MÉDAILLE ARGENT (2e), 3 MÉDAILLE BRONZE (3e), 4 COUPE DU MOIS (meilleur commercial du mois), 5 TROPHÉE KPI DU MOIS (plaque avec cible), 6 COURONNE DE SAISON (champion de saison), 7 DÉFI FLASH (éclair dans un hexagone), 8 RÉGULARITÉ (calendrier coché, flamme), 9 PREMIER 100 % (jauge pleine), 10 GRAND CHELEM (quatre étoiles en carré), 11 PALIER D'ÉQUIPE (drapeau sur sommet), 12 VERSION VERROUILLÉE générique (silhouette de trophée grise avec cadenas).

Style : illustration vectorielle plate, formes géométriques, angle d'inclinaison de 8 degrés vers la droite pour l'énergie, traits noirs épais réguliers de 6 px à 512 px, une seule ombre plate décalée en noir.

Palette stricte : jaune #FFD600, noir #0A0A0A, blanc #FFFFFF. Pour argent et bronze uniquement : gris #C9C9CC et cuivre #C8834A en aplat. Aucune autre couleur.

Format : 12 PNG séparés, 512 x 512 px, fond transparent, objet centré, 40 px de marge, même taille optique et même point de vue de face sur toute la série.

Lisible à 40 px.

À ne pas faire : pas de rendu 3D, pas de métal réaliste ni reflets, pas de rubans multicolores, pas de texte, pas de chiffres sauf "1", "2", "3" sur les médailles, pas d'emoji, pas de personnage, pas de logo de marque.

PROMPT CLAUDE (INTÉGRATION)

Contexte : `fitpulse.html`. J'ajoute `assets/trophies/` avec : `gold.png`, `silver.png`, `bronze.png`, `month.png`, `kpi.png`, `season.png`, `flash.png`, `streak.png`, `first100.png`, `slam.png`, `palier.png`, `locked.png` (512x512, fond transparent).

À faire :

1. Dans `allTrophies()` (lignes 1356 à 1395), remplace le champ `icon` emoji par un champ `art` : `'month'`, `'kpi'`, `'season'`, `'gold'` / `'silver'` / `'bronze'`, `'flash'`. Garde un repli : si une donnée ancienne contient `icon` emoji, mappe-la vers `art`.

2. Crée `trophyArt(art, size, locked=false)` : `` ; `locked = locked.png`. Repli : `ico('trophy')` ou `ico('medal')` en 32px si l'image échoue.

3. Profil, accomplissements (lignes 2614 à 2616) : `streak`, `first100`, `slam`, avec `locked.png` tant que non obtenu. Carte `.trophy` : image 56px, titre en `.label`, date en `.small`.

4. Profil, liste des trophées (ligne 2617) : grille de cartes avec image 48px.

5. Classement, podium (ligne 2198) : au-dessus de chaque avatar, médaille 44px (gold au centre, silver à gauche, bronze à droite). Le bloc `.podium .block` passe en noir pour 2 et 3 avec chiffre blanc, en jaune pour 1 avec chiffre noir, coins supérieurs biseautés via `clip-path: polygon(8% 0, 100% 0, 92% 100%, 0 100%)`.

6. Liste du classement (ligne 2200) : les 4 derniers trophées en images 18px au lieu des emojis.

7. Wrap (ligne 2658) : grille de 4 colonnes d'images 56px, libellé dessous, au lieu des emojis concaténés.

Critères d'acceptation :

- Plus aucun emoji dans `allTrophies()`, `PAGES.profile`, `PAGES.leaderboard`, `PAGES.wrap`.
- Le podium reste lisible à 375px de large, médailles comprises.
- Les trophées verrouillés sont visuellement distincts sans dépendre de la couleur seule (cadenas visible).

8. BANNIÈRE D'ACCUEIL, CONNEXION ET MASCOTTE

La bannière d'accueil (`PAGES.home` ligne 4235, CSS lignes 533 à 547) charge une photo d'ambiance `assets/hero-banner.jpg` volontairement floutée (`filter: blur(3px)` ligne 534) et assombrie par un dégradé à 88 % : la photo ne se voit presque plus, il reste un fond noir avec une bande jaune. Même schéma pour la connexion (`.auth-visual`, `blur(2.5px)` ligne 614). La mascotte du tableau de bord `mascot()` (ligne 1544) est un personnage cartoon avec 4 teintes de peau, gouttes de sueur bleues `#7cc4ff` et "FP" en Anton : c'est le seul élément illustré de l'app et il ne correspond pas à la DA sportive premium.

PROMPT CHATGPT (VISUEL)

Crée deux visuels d'en-tête pour l'application commerciale interne d'une salle de sport Fitness Park (identité jaune et noir).
Visuel A, bannière d'accueil : format 2400 x 800 px (ratio 3:1). Visuel B, écran de connexion : 1600 x 2000 px (portrait 4:5).
Composition : à gauche 55 % de noir profond `#0A0A0A` laissé libre pour du texte blanc, à droite une scène photographique d'un plateau de musculation premium en basse lumière, éclairage rasant chaud, silhouette d'un athlète en action floue de mouvement, sans visage identifiable. Deux bandes diagonales jaunes `#FFD600` inclinées à 18 degrés traversent l'image à droite, l'une pleine, l'autre à 35 % d'opacité. Grain photo léger, contraste fort, tons désaturés sauf le jaune.
Pas de texte, pas de logo, pas de marque d'équipement visible, pas d'emoji, pas d'effet néon, pas de personnes souriant face caméra, pas de stock photo générique.
Livrer en JPG qualité 85, plus une variante "zone texte" où le tiers gauche est un noir uni parfait pour garantir la lisibilité.

PROMPT CLAUDE (INTÉGRATION)

Contexte : `fitpulse.html`. Les visuels sont référencés par `window.PARKPULSE_ASSETS` (ligne 739) : banner et login. Je fournis `assets/hero-banner.jpg` (2400x800) et `assets/hero-login.jpg` (1600x2000) déjà composés avec une zone noire à gauche.

À faire :

- `.banner::before` (ligne 534) et `.auth-visual::before` (ligne 614) : supprime `filter: blur()` et `transform: scale()`. Garde `background-size: cover`, `background-position: right center`.
- `.banner::after` (ligne 535) : remplace le dégradé par `linear-gradient(90deg, #0A0A0A 0%, rgb(10 10 10 / .85) 35%, transparent 65%)`. Sur mobile (`<= 860px`), dégradé vertical du bas vers le haut pour garder le texte lisible.
- Ajoute `<link rel="preload" as="image">` pour la bannière et un `srcset 1200w / 2400w` via `image-set()`.
- Mascotte : retire `mascot()` du tableau de bord (ligne 1998) et le choix d'avatar cartoon du profil (ligne 2627). À la place, affiche une jauge d'humeur abstraite : `anneau ring()` existant, couleur `HEALTH`, et une phrase courte (`moodOf`). Garde la fonction `mascot()` inutilisée supprimée du code.
- Les avatars restent des initiales sur pastille jaune (`.avatar` ligne 139) : passe la police en `Barlow 800 italique`, et autorise une photo (`u.photo`) en `object-fit: cover`.

Critères d'acceptation :

- La photo de bannière est nette et visible sur au moins 40 % de la largeur en desktop.
- Le titre "Bonjour <prénom>" garde un contraste $\geq 7:1$ sur toute sa surface (mesure sur 3 points).
- ``grep -c "mascot()" = 0`. Aucune couleur de peau ni `#7cc4ff` dans le fichier.

9. ÉTATS VIDES ILLUSTRÉS

L'app compte au moins 16 états vides, tous traités en texte gris centré (`.empty` ligne 142) : "Pas encore d'activité" (feed, 2241), "Aucun message" (chat, 2272), "Aucun défi flash terminé" (2326), "Aucune tâche configurée" (2166), "Aucun membre actif" (2469), "Aucune tâche" (3012), "Aucune action ce mois-ci" (3037), "Aucun client perdu" (3041), "Aucune demande à traiter" (3895), "Rien dans cette vue" (4043), "Rien d'assigné pour l'instant" (4198), "Pas d'objectif pour ..." (1972), "Aucun palier ce mois-ci" (4243), "Pas encore de classement" (4250). Un état vide est le meilleur endroit pour donner envie d'agir : il faut une illustration, une phrase et un bouton.

PROMPT CHATGPT (VISUEL)

Crée une série de 6 illustrations "état vide" pour une application commerciale de salle de sport (identité Fitness Park jaune et noir).

Sujets : 1 FIL D'ACTIVITÉ VIDE (piste d'athlétisme vide avec ligne de départ), 2 AUCUN MESSAGE (bulle de discussion posée sur un banc de musculation), 3 AUCUNE RELANCE (combiné de téléphone posé, coche), 4 AUCUN DÉFI (chronomètre à zéro), 5 TOUT EST TRAITÉ (haltère reposé sur son rack, coche), 6 PAS D'OBJECTIF (cible vierge avec un crayon).

Style : illustration au trait, monoligne, trait noir #0A0A0A de 8 px à 800 px, coins arrondis, un seul aplat jaune #FFD600 par illustration sur l'élément principal, perspective légèrement inclinée à 8 degrés. Fond transparent.

Même grille et même épaisseur de trait sur les 6, même niveau de détail, sujet centré.

Format : 6 PNG séparés, 800 x 600 px, fond transparent. Fournir aussi une variante où le trait est blanc #F5F5F3 pour le mode sombre.

À ne pas faire : pas de personnage, pas de visage, pas de texte, pas d'emoji, pas de dégradé, pas d'ombre, pas d'autres couleurs, pas de style 3D ou isométrique chargé.

PROMPT CLAUDE (INTÉGRATION)

Contexte : `fitpulse.html`. Les états vides utilisent `<div class="empty">` ou `<div class="card empty">` avec un `.title` et un `<p>`. J'ajoute `assets/empty/` : `feed`, `chat`, `todo`, `challenge`, `done`, `target`, en deux variantes `<nom>-dark.png` et `<nom>-light.png` (800x600, transparent).

À faire :

1. Crée `emptyState({ art, title, text, cta })` qui renvoie :

```
<div class="empty"><picture><source srcset="assets/empty/<art>-dark.png" media="(prefers-color-scheme: dark)"></picture><div class="title"><title></div><p><text></p><p><cta></p></div>
```

Gère aussi `data-theme` forcé : si `<html data-theme="dark">`, force la variante dark (petite fonction `themeAsset()`).

2. Remplace les 14 états vides listés (lignes 1972, 2166, 2241, 2272, 2326, 2469, 3012, 3037, 3041, 3895, 4043, 4198, 4243, 4250) par `emptyState()`, avec un bouton d'action chaque fois qu'il existe une action logique. Exemples :

- Feed : art `feed`, "Le fil démarre à la première saisie", bouton "Saisir" (`data-act="tbSaisir"`).
- Relances : art `done`, "Aucune relance à votre nom", bouton "Voir les impayés" (`#/impayés`).
- Défis : art `challenge`, "Aucun défi terminé", bouton "Lancer un défi" pour les managers.
- Paliers : art `target`, "Aucun palier ce mois-ci", bouton "Régler les paliers" pour les managers.

3. CSS `.empty` : `padding var(--s-6) var(--s-4)`, image 160px de large sur mobile, `.title` en `var(--t-h3)`, texte max 42ch, bouton `.btn.primary sm`.

Critères d'acceptation :

- Chaque état vide a une image, un titre de 6 mots maximum, une phrase de 15 mots maximum, et un bouton quand une action existe.
- Les images ne provoquent aucun décalage de mise en page (`width` et `height` posés).
- Bascule de thème : la variante d'illustration change sans rechargement.

10. CÉLÉBRATION DE PALIER ET RETOURS DE SAISIE

`celebrate()` (ligne 4130) affiche un titre en capitales et 28 rectangles de confettis jaunes et blancs codés `#FFD200` (ligne 4133), pendant 2,6 s. Elle sert à trois moments très différents : "PALIER 2 ATTEINT" (ligne 4140), "CLIENT SAUVÉ" (ligne 3938) et "IMPAYÉ RÉCUPÉRÉ" (ligne 4071). Les jalons d'équipe P1, P2, P3 (`.palier-tiers span`, ligne 465) sont des pilules texte avec une coche Unicode (ligne 4125). Pour un commercial, le passage de palier est le moment fort du mois : il mérite un visuel de récompense dédié et une hiérarchie (petite victoire, victoire d'équipe, niveau personnel).

PROMPT CHATGPT (VISUEL)

Crée 3 insignes de palier d'équipe pour une application commerciale de salle de sport (identité Fitness Park jaune et noir) : PALIER 1, PALIER 2, PALIER 3.
Forme : chevron ou écusson incliné de 8 degrés vers la droite, avec le chiffre 1, 2 ou 3 en très grand, police condensée italique très grasse type Barlow Condensed Black Italic, et le mot PALIER en petites capitales au-dessus.
Progression : P1 contour jaune `#FFD600` sur fond noir `#0A0A0A` ; P2 fond jaune plein, chiffre noir ; P3 fond jaune plein, chiffre noir, double liseré et éclats rayonnants en jaune autour.
Palette stricte : `#FFD600`, `#0A0A0A`, `#FFFFFF`.
Format : 3 PNG séparés, 600 x 600 px, fond transparent, même taille optique. Fournir aussi chaque insigne en SVG si possible.
Style : vectoriel plat, net, énergique, esprit affiche sportive.
À ne pas faire : pas de confettis multicolores, pas de 3D, pas de métal, pas d'emoji, pas de personnage, pas d'autre texte que PALIER et le chiffre.

PROMPT CLAUDE (INTÉGRATION)

Contexte : `fitpulse.html`. `celebrate(title, sub)` ligne 4130 sert pour paliers, client sauvé et impayé récupéré. J'ajoute `assets/paliers/pl.png`, `p2.png`, `p3.png` (600x600, transparent).

À faire :

- Refonds `celebrate` en `celebrate({ kind, title, sub, art })` avec 3 niveaux :
 - `kind 'win'` (client sauvé, impayé récupéré) : pas de plein écran. Toast enrichi 3 s en bas : icône `ico('check')` dans un carré jaune, titre en Barlow 800 italique, sous-titre. Pas de confettis.
 - `kind 'team'` (palier franchi) : plein écran `.zone-black`, insigne `assets/paliers/p<N>.png` en 220px qui entre avec un `scale` de `.6` à `1` et une rotation de `-8` à `0` degré en 450 ms, titre "PALIER 2 ATTEINT" en `var(--t-display)`, récompense en sous-titre, deux bandes diagonales jaunes qui balaient l'écran de gauche à droite en 600 ms à la place des confettis. Fermeture au clic ou après 3,5 s. Bouton "Partager au fil" qui publie dans le feed.
 - `kind 'level'` (nouveau niveau personnel, voir lot badges) : même plein écran avec `levelBadge(lv, 220)`.
- Mets à jour les appels : ligne 3938 `kind 'win'`, ligne 4071 `kind 'win'`, ligne 4140 `kind 'team'` avec `art p<N>`.
- `palierBlock()` (ligne 4116) : les pilules `.palier-tiers` deviennent des mini-insignes 28px (image `pl/p2/p3`) ; non atteints en `grayscale` et `opacité .35`. Supprime la coche Unicode de la ligne 4125 et le texte "🏆" de la ligne 4126 ; message final "Tous les paliers sont atteints" avec l'insigne `p3` à côté.
- `prefers-reduced-motion` : pas d'animation, affichage direct 2,5 s.
- Couleurs : uniquement `var(--fp)`, `var(--fp-ink)`, blanc. Plus aucun `'#FFD200'` ni `'#fff'` codé dans `celebrate`.

Critères d'acceptation :

- Une saisie qui ne franchit pas de palier ne déclenche jamais de plein écran.
- Un palier franchi affiche le bon insigne (P1, P2 ou P3) et la récompense saisie dans Membres > Paliers.

- Avec reduced-motion activé, aucune animation ne joue.

11. BADGES, CHIPS ET STATUTS : UN SEUL COMPOSANT

Sept composants font le même travail avec sept styles : `.badge` (ligne 125, rayon 99 px, 11 px), `.chip` (ligne 264, rayon 8 px, 12,5 px), `.chip-radio span` (ligne 361, 13 px), `.hchip` avec point lumineux (ligne 518, 11,5 px), `.hdot` (ligne 524), `.palier-tiers span` (ligne 465, couleurs codées `#26262a` et `#b9b9be`) et `.jtag` (ligne 541, jaune incliné). Les teintes de `.badge.ok/.warn/.bad` sortent de la v1 alors que `.hchip` utilise les tokens v3 : un "OK" vert n'a pas la même couleur selon l'écran. Le rouge `.hchip.h-alert` pulse en continu (ligne 522), ce qui fatigue sur un tableau de 30 lignes. On compte 14 valeurs de `border-radius` différentes.

PROMPT CLAUDE (INTÉGRATION)

Contexte : fitpulse.html, tokens unifiés. Tu remplaces les composants de pastille par un seul système.

1. Crée `.tag` (base) : `display inline-flex, align-items center, gap var(--s-1), height 22px, padding 0 var(--s-2), border-radius var(--r-sm), font: 700 var(--t-label)/1 var(--font), letter-spacing .02em, background var(--surface-2), color var(--muted), white-space nowrap`.

Variante de sens : `.tag.is-ok (var(--ok-soft) / var(--ok)), .is-warn, .is-bad, .is-info, .is-brand (var(--fp) / var(--fp-ink)), .is-dark (var(--fp-ink) / var(--fp))`.

Option `.tag.has-dot` : pastille 6px `currentColor` avant le texte.

Option `.tag.skew` : `transform skewX(-8deg)` pour les chiffres forts (J-12, P1), contenu redressé avec `skewX(8deg)`.

2. Remplace : `.badge` par `.tag`, `.hchip` par `.tag.has-dot + variante de sens`, `.palier-tiers span` par `.tag` (et `.is-brand` si atteint), `.jtag` par `.tag.is-brand.skew`. Garde `.chip` uniquement pour les éléments déplaçables du planning, et `.chip-radio` pour les filtres (rayon `var(--r-pill)`, hauteur 32px).

3. Rayons : n'autorise que les tokens `--r-xs, --r-sm, --r-md, --r-lg, --r-xl, --r-pill`. Cartes `var(--r-lg)`, boutons `var(--r-sm)`, champs `var(--r-sm)`, modales `var(--r-xl)`, avatars 50 %.

4. Animation pulse-a de `.h-alert` : jouée 2 fois à l'apparition (`animation-iteration-count: 2`), plus en boucle.

5. Vocabulaire des statuts identique partout : "Dans le rythme", "À surveiller", "En retard" (remplace "Alerte"), "Sans donnée".

Critères d'acceptation :

- ``grep -c "\.badge\b|\.hchip\b|\.jtag\b"`` dans le CSS = 0.

- ``grep -o "border-radius: *[0-9.]*px"`` hors tokens et hors `.podium` = 0.

- Un statut "En retard" a exactement la même couleur sur l'accueil, le classement, le tableau de bord et le récap.

12. GRAPHIQUES : PALETTE DE DONNÉES ET LISIBILITÉ

Les graphiques sont faits maison (`barChart` ligne 1563, `palierRace` ligne 1770, `stackRows` ligne 1815, `resFunnel` ligne 1828, `monthBars` ligne 4384) et plutôt bien construits, mais leurs couleurs sont codées à la main : N-1 en `#8a8a90` contre N en `#FFD200` (ligne 2081), canaux d'impayés en jaune, bleu `#2F6FDB`, vert `#1F9D55`, violet `#8A63D2`, gris (`RECOV_CHANNELS` lignes 3245 à 3249), avis Google en bleu Google `#4285F4` (ligne 4430). Le vert "prélèvement automatique" se confond avec le vert "dans le rythme". Les infobulles SVG utilisent un tiret cadratin (`<title>${s.name} – ...`, ligne 1571).

PROMPT CLAUDE (INTÉGRATION)

Contexte : fitpulse.html. Les graphiques SVG maison prennent des couleurs en dur. Tu poses une palette de données de marque.

1. Tokens de données dans :root (sombre) et le bloc clair :

```
--d-1: var(--fp) /* série principale, période en cours, "équipe" */
--d-2: #F5F5F3 (sombre) / #0E0E0E (clair) /* série de comparaison forte */
--d-3: #8C8C8C /* période précédente, N-1 */
--d-4: #5C5C5C (sombre) / #BDBDBD (clair)
--d-5: #3A3A3A (sombre) / #DADADA (clair)
```

Les couleurs --ok, --warn, --bad sont réservées aux statuts, jamais aux séries.

2. barChart (ligne 2081) : N = var(--d-1), N-1 = var(--d-3).

3. RECOV_CHANNELS (lignes 3244 à 3249) : equipe var(--d-1) (le seul canal "humain", mis en avant), client var(--d-2), auto var(--d-3), automatismes var(--d-4), tiers var(--d-5). Légende ordonnée dans cet ordre.

4. Avis Google (ligne 4430) : var(--d-1), plus le bleu Google.

5. Toutes les barres : rx 3, espacement 2px, valeur affichée au-dessus de la dernière barre seulement, en Barlow 800 italique 15px.

6. Infobulles <title> (ligne 1571 et suivantes) : format "2026 : 42 contrats", sans tiret.

7. Axes : texte 11px minimum, couleur var(--muted), grille var(--line) en pointillé 2 4, pas de ligne d'axe Y.

8. Vérifie que palierRace en mode clair suit la règle du lot "couleurs" (courbe var(--text), aire jaune).

Critères d'acceptation :

- `grep -n "color: '#|color: '#" ` dans le JS = 0.

- Aucune série de graphique n'utilise var(--ok), var(--warn) ou var(--bad).

- Le graphique d'impayés par canal reste distinguable en niveaux de gris (impression du récap, @media print).

ORDRE DE MISE EN OEUVRE CONSEILLÉ

1. Socle CSS (point 2), puis échelle typo (3) et couleurs (4) : base technique, aucun visuel requis.
2. Charte d'écriture (1) et iconographie (5) : gain immédiat, toujours sans visuel.
3. Composants tag (11) et graphiques (12).
4. Commande des visuels ChatGPT dans cet ordre : trophées (7), badges de niveau (6), insignes de palier (10), états vides (9), bannières (8). Livrer chaque série en une seule session de génération pour garder la cohérence de trait.

2. ERGONOMIE ET PLAISIR D'USAGE DES COMMERCIAUX

Périmètre : parcours d'un commercial debout à l'accueil du club, téléphone en main. Connexion, accueil, saisie, position vs objectif et vs collègues, prochaine action, gamification, accessibilité. Classement du plus impactant au moins impactant.

Constat global : les briques sont bonnes. On a une saisie en un geste avec annulation (`quickAdd`, l. 4144), des points rapportés à l'objectif (`statsFor`, l. 1269), des défis normalisés (`challengeRanking`, l. 1417), un wrap façon stories et des paliers collectifs. Le problème est ailleurs. Sur mobile, l'essentiel pour le commercial (moi, maintenant, quoi faire) est enterré sous des contenus d'équipe et de manager. Le feedback personnel est pauvre. La gamification ne récompense que le résultat, jamais l'effort de relance.

1. ACCUEIL MOBILE : METTRE "MOI, MAINTENANT" AU-DESSUS DE LA LIGNE DE FLOTATION

`PAGES.home.render` (l. 4211 à 4255) empile sur mobile, en une colonne (`.gl2 { grid-template-columns: 1fr }`, CSS v3 vers l. 629) : bannière, puis graphique SVG "Course au palier" de 720 x 250 (`palierRace`, l. 1770), puis paliers d'équipe, et seulement ensuite "Saisir", "Ma progression" et "Mes relances". Le commercial doit faire défiler 3 à 5 écrans pour voir sa mission du jour (`mini-mission`, l. 4248) et ses relances (`todoList(5)`, l. 4251). Le graphique d'équipe a des libellés de 11 px illisibles sur 375 px. Il faut inverser : mon chiffre, ma mission du jour, mes 3 prochaines actions, puis l'équipe. Pipedrive mobile et HubSpot ouvrent sur "Aujourd'hui", pas sur un graphique.

PROMPT CLAUDE

Contexte : fichier unique `fitpulse.html` (app Fit Pulse). Rôle concerné : membre (commercial), écran de largeur ≤ 860 px.

Objectif : refondre l'ordre de l'accueil mobile dans `PAGES.home.render` (vers la ligne 4211) pour que le commercial voie en premier sa situation et sa prochaine action.

À faire :

1. Créer une carte "Ma journée" placée juste sous une bannière compactée (hauteur max 140 px sur mobile). Elle contient sur une seule ligne : mon % du mois (même indicateur que le classement, voir règle 4), mon rang "#3 sur 8", et "J-12". En dessous : la mission du jour existante (calcul "mission" déjà présent, KPI contrats, avis, nutrition, accessoires) sous forme de 2 à 4 puces "Contrats : 1 aujourd'hui", avec un état coché quand c'est fait.

2. Juste après, une carte "À faire maintenant" qui reprend `todoList(3)` (fonction existante vers la ligne 4191) avec le bouton d'action à droite et un lien "Voir tout" vers `#/relances`.

3. Puis le pavé `quickPad()` (inchangé dans sa logique).

4. Puis Top 5 et paliers d'équipe. Le graphique `palierRace` passe derrière un bouton "Voir la course au palier" (repliable, fermé par défaut sur mobile, ouvert par défaut au-dessus de 860 px).

5. Les blocs manager (`managerCockpit`, résiliations, impayés par canal) restent affichés uniquement si `isManager()`, après "À faire maintenant".

Règles :

- Aucune logique de calcul modifiée, seulement l'ordre et la présentation.
- Aucun emoji décoratif ajouté. Aucun tiret cadratin ni demi-cadratin dans les textes d'interface.

- Garder la charte : jaune #FFD600, noir, titres Barlow Condensed italique (var(--font-title)).

Critères d'acceptation :

- Sur un écran 375 x 812, sans défiler, on voit : mon %, mon rang, J-x, la mission du jour et au moins la première relance.
- Le graphique palierRace n'est plus rendu tant que l'utilisateur n'a pas ouvert la section sur mobile.
- Au-dessus de 860 px, la disposition desktop reste équivalente à l'actuelle.
- Aucun texte d'interface ne contient d'emoji ni de tiret cadratin.

2. SAISIE ÉCLAIR : UN SEUL CHEMIN, 2 TAPS, SANS DOUBLE COMPTAGE

Le bon chemin existe : bouton jaune central de la tabBar (`tabBar` , l. 4307) puis `quickPad` dans une modale (`ACTIONS.tbSaisir` , l. 4312), soit 2 taps pour "+1 contrat". Mais il cohabite avec un second système : le tiroir `openSaisies` (l. 2121), un formulaire de 9 champs numériques, ouvert par le bouton vertical "Saisir" de la page objectifs (l. 1959). Trois défauts concrets. Pour 3 avis, il faut 3 taps et on reçoit 3 toasts. "Prospects" vaut 0 point mais occupe une place du pavé. Le bouton "Impayés" en euros (`QUICK_EUR` , l. 4159) double le flux "Récupéré" (`dunPaidSave` , l. 4066) et "Réglé" de la rétention (`loyAct` , l. 3020), qui créent déjà une saisie : risque de double comptage.

PROMPT CLAUDE

Contexte : `fitpulse.html`, saisie des ventes par un commercial sur mobile.

Objectif : un seul chemin de saisie rapide, en 2 taps maximum pour une quantité de 1, et sans double comptage des impayés.

À faire :

1. `quickPad` (vers la ligne 4160) : un tap court sur un bouton quantité ajoute 1 (inchangé). Un appui long (500 ms) ou un petit bouton "+N" ouvre un sélecteur 2, 3, 4, 5 qui enregistre la quantité en une seule saisie et un seul toast.
 2. Retirer "impayés" de `QUICK_EUR` (ligne 4159). Les impayés récupérés se saisissent uniquement via Impayés > Récupéré ou Rétention > Réglé. Dans le pavé, à la place, afficher un lien discret "Impayé récupéré : passer par mes relances" vers `#/relances`.
 3. Déplacer "prospects" en fin de pavé, avec un style secondaire (fond `var(--surface-2)`), puisqu'il ne rapporte pas de points.
 4. Supprimer le bouton flottant vertical ".saisies-tab" de la page objectifs (ligne 1959) : la saisie passe par le bouton "Saisir" de la barre du haut et de la tabBar, qui ouvrent `quickPad`. Garder `openSaisies` uniquement comme "Saisie détaillée" (autre date, autre membre pour un manager), accessible depuis le lien existant sous le pavé.
 5. Ordre du pavé : les KPI dont il me reste le plus de retard sur le rythme en premier (calcul via `statsFor` et `elapsed` déjà disponibles), avec une option "ordre fixe" dans le profil.
- Règles :

- Ne pas changer le format des entrées (`S.entries`) ni `quickAdd`, sauf pour accepter une valeur > 1.
 - Aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin dans les textes d'interface.
- Critères d'acceptation :
- Depuis l'accueil mobile : "+1 contrat" en 2 taps (bouton Saisir puis Contrats).
 - "3 avis Google" en 3 taps au plus, une seule entrée créée de valeur 3, un seul toast avec Annuler.

- Impossible de saisir des impayés récupérés depuis le pavé rapide.
- Le bouton vertical "Saisir" n'existe plus sur la page Mes objectifs.

3. FEEDBACK INSTANTANÉ : DIRE AU COMMERCIAL CE QUE SA VENTE VIENT DE CHANGER

Après une saisie, le seul retour est un toast "+1 Contrats signés enregistré" (`quickAdd` , l. 4149). Rien sur mon avancement, mes points, mon rang. Les célébrations plein écran (`celebrate` , l. 4130) existent seulement pour un palier d'équipe, un client sauvé ou un impayé récupéré, jamais pour un jalon personnel (25, 50, 75, 100 % d'un KPI) ni pour un dépassement au classement. C'est pourtant le moment où l'app peut créer de l'envie. Spinify et Strava répondent à chaque action par un écart chiffré ("plus que 1 pour...") et une micro-récompense.

PROMPT CLAUDE

Contexte : `fitpulse.html`. Fonction `quickAdd` (vers la ligne 4144), `toastUndo` (4152), `celebrate` (4130), `statsFor` (1269), `ranking` (1315), TIERS (1186).

Objectif : après chaque saisie personnelle, un retour qui dit ce que la vente a changé, sans être envahissant.

À faire :

1. Dans `quickAdd`, prendre un instantané avant / après : `statsFor(CLUB.id, userId, rangeOf('month', curMonth()), { kpiIds: [kpiId] })` et le rang via `ranking(CLUB.id, r)`.
 2. Toast enrichi (toujours avec Annuler 5 s) : "+1 contrat. 7 sur 12. Plus que 2 pour l'étape 75 %". Réutiliser la logique de `paceMessage` (ligne 1303) pour la deuxième phrase.
 3. Si la saisie fait franchir une étape TIERS (25, 50, 75, 100 %) sur ce KPI : petite célébration non bloquante (bandeau jaune 1,5 s en haut, pas plein écran) "Étape 50 % Contrats". À 100 % : `celebrate()` plein écran "OBJECTIF ATTEINT".
 4. Si le rang s'améliore : ajouter au toast "Tu passes 3e".
 5. Retour haptique : `navigator.vibrate(15)` si disponible, et 3 impulsions courtes pour un franchissement d'étape. Désactivable dans Profil > Notifications (nouvelle préférence "Vibrations").
 6. Respecter `prefers-reduced-motion` : pas d'animation de confettis, texte seul.
 7. Le conteneur `#toasts` reçoit `role="status"` et `aria-live="polite"`.
- Règles :
- Aucun emoji décoratif dans les toasts et célébrations (retirer aussi l'emoji flamme de "saisie(s) enregistrée(s)" ligne 2160 et l'emoji fête de `paceMessage` ligne 1306).
 - Aucun tiret cadratin ni demi-cadratin dans les textes d'interface.
 - Une annulation via "Annuler" doit annuler aussi la célébration si elle n'est pas encore affichée.
- Critères d'acceptation :
- Saisir +1 contrat affiche un toast contenant le réalisé, l'objectif et l'écart à la prochaine étape.
 - Passer de 49 % à 50 % déclenche le bandeau "Étape 50 %", passer à 100 % déclenche `celebrate()`.
 - Un lecteur d'écran annonce le toast.
 - Avec `prefers-reduced-motion`, aucune animation n'est jouée.

4. UN SEUL CHIFFRE ET UN SEUL VOCABULAIRE POUR "OÙ J'EN SUIS"

Le commercial voit plusieurs pourcentages qui ne coïncident pas. L'anneau de l'accueil affiche "% des points" (`myPct = st.earned / st.max`, l. 4227). Le Top 5 et le classement affichent le "score pondéré" (`st.score`, plafonné à 150 %, l. 1285). La page objectifs montre les deux, avec la légende "moyenne des % pondérée par les points (plafond 150 % par KPI)" (l. 2010). Les échelles de statut se multiplient aussi : `statusOf` (En avance, À l'heure, En léger retard, Très en retard, l. 1291), `HEALTH` (Dans le rythme, À surveiller, Alerte, l. 1759), `tierName` (STARTER, RISING, PRO, ELITE, l. 2523), `LEVELS` (Rookie à Légende, l. 870). Les "paliers" désignent la prime d'équipe, mais les crans 25/50/75 % s'appellent "étapes" ou "tiers" selon l'écran. Un commercial debout doit lire une seule chose.

PROMPT CLAUDE

Contexte : `fitpulse.html`. Fonctions `statsFor` (1269), `statusOf` (1291), `HEALTH` et `healthOf` (1759 à 1767), `tierName` (2523), `LEVELS` (870), `PAGES.home` (4211), `dashObjectives` (1970), `PAGES.leaderboard` (2175), `drawWrapCard` (2682).

Objectif : un indicateur principal unique et un lexique cohérent dans toute l'interface membre.

À faire :

- Indicateur principal = "Score du mois" = `st.score` (celui qui sert au classement). L'afficher partout où l'on montre "ma progression" : anneau de l'accueil, hero de Mes objectifs, Récap du jour, wrap. Les points (`st.earned`) deviennent une information secondaire ("1 850 pts").
- Statut de rythme : une seule échelle à 3 niveaux, celle de `HEALTH` ("Dans le rythme", "À surveiller", "En retard"). Remplacer "Alerte" par "En retard" pour le membre. `statusOf` ne sert plus qu'à "Objectif atteint" et "Sans objectif".
- Supprimer `tierName` (STARTER, RISING, PRO, ELITE) de l'interface membre et du canvas du wrap. Le seul système de niveaux visible est `LEVELS`.
- Lexique : "Palier" = objectif collectif de l'équipe (prime). "Étape" = cran personnel 25, 50, 75, 100 % d'un KPI. Corriger tous les libellés concernés (classe `.tiers` dans le hero, `kpiCard`, `paceMessage`).
- Remplacer la légende technique du "Score pondéré" par une bulle d'aide accessible (bouton "i" avec `aria-label`) : "Moyenne de tes KPI obligatoires, chacun compté selon son importance. Un KPI compte au maximum pour 150 %."
- Aligner les titres : la page dont l'entrée de menu est "Mes objectifs" doit avoir le titre "Mes objectifs" (aujourd'hui "Tableau de bord", ligne 1941). "Bilan du jour" et "Récap du jour" : garder "Bilan du jour" partout.

Règles :

- Aucune modification des formules de calcul.
- Aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin dans les textes d'interface.

Critères d'acceptation :

- L'accueil, Mes objectifs, le Top 5 et le classement affichent la même valeur de score pour un même utilisateur et un même mois.
- Les chaînes "STARTER", "RISING", "PRO", "Très en retard", "Alerte" n'apparaissent plus dans l'interface d'un membre.
- Le mot "palier" n'est utilisé que pour les objectifs collectifs.

5. NAVIGATION : DE 9 ENTRÉES À 4 DESTINATIONS CLAIRES

Le membre voit dans le menu (`NAV`, l. 1606) : Accueil, Mes objectifs, Classement, Résiliations, Impayés, Action Rétention, Défis flash, Chat, Feed, plus Mon profil. Quatre entrées parlent du même geste, appeler un client : "Mes

relances" (onglet mobile, l. 4273), "Action Rétention", "Résiliations", "Impayés". La pastille de l'onglet Relances ne compte que résiliations et impayés dus (l. 4309), pas les appels de rétention. Feed et Chat sont deux fils sociaux séparés. Sur mobile, "Plus" ouvre le tiroir desktop complet. Salesforce mobile et HubSpot limitent la barre basse à 4 ou 5 destinations, avec un hub "Tâches" unique.

PROMPT CLAUDE

Contexte : fitpulse.html. NAV (ligne 1606), shell (1633), tabBar (4307), PAGES.relances (4273), PAGES.loyalty (2979), PAGES.resiliations (3873), PAGES.impayés (3994), PAGES.feed (2217), PAGES.chat (2247), PAGES.challenges (2309).

Objectif : une navigation membre en 4 destinations + Saisir, sans perdre de fonctionnalité.

À faire :

1. Tabbar mobile : Accueil, Relances, [Saisir], Classement, Équipe. "Plus" disparaît pour le membre. Le profil s'ouvre en touchant l'avatar en haut à droite de la topbar (à ajouter).

2. Relances devient un hub à segments : "À mon nom" (todoList actuel), "Résiliations", "Impayés", "Rétention". Les pages existantes sont rendues dans ces segments (réutiliser leurs fonctions de rendu). Les URL #/resiliations, #/impayés, #/loyalty continuent de fonctionner et ouvrent le bon segment.

3. La pastille de l'onglet Relances = résiliations à mon nom + impayés dus à mon nom + tâches de rétention du jour (types suivi, renouvellement, anniversaire dont l'échéance est aujourd'hui ou dépassée).

4. Équipe devient un hub : "Fil" (Feed actuel), "Chat", "Défis". Une pastille cumule les non-lus du fil et du chat et signale un défi en cours.

5. Menu latéral desktop pour le membre : Accueil, Mes objectifs, Relances, Classement, Équipe, puis Mon profil. Les entrées manager restent inchangées pour les managers.

6. Renommer "Feed" en "Fil d'équipe" et "Action Rétention" en "Rétention".

Règles :

- Aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin dans les textes d'interface.

- Cibles tactiles de la tabBar : 48 x 48 px minimum, libellés 12 px minimum.

Critères d'acceptation :

- Un membre atteint n'importe quelle fonctionnalité qui lui était accessible en 2 taps maximum depuis l'accueil mobile.

- Le menu d'un membre compte au plus 6 entrées.

- Les anciennes URL de hash restent valides.

6. RELANCES SUR TÉLÉPHONE : APPELER EN UN TAP, CONCLURE EN UN TAP

Le commercial doit appeler depuis l'accueil du club, mais aucun numéro n'est cliquable : zéro lien `tel:` dans tout le fichier (numéros affichés en texte dans `loyTasks`, l. 3010, `dunTable`, l. 4035, `todoList`, l. 4196). La page Impayés est un tableur de 8 colonnes avec des selects et des champs date dans chaque ligne (`dunTable`, l. 4033 à 4042), inutilisable à 375 px. Sur une carte de résiliation, "Sauvée" (`resSaveIt`, l. 3934) écrit immédiatement un sauvetage et lance une célébration, sans confirmation ni annulation, alors que "Résiliée" demande confirmation. En rétention, 6 boutons d'issue s'alignent sur chaque carte (`OUTCOMES`, l. 1444), dont "Joint" suivi d'un tiret cadratin puis "OK".

PROMPT CLAUDE

Contexte : fitpulse.html. `loyTasks` (2998), `loyAct` (3016), `OUTCOMES` (1444), `dunTable` (4002), `resCard` (3903), `resSaveIt` (3934), `todoList` (4191).

Objectif : traiter une relance depuis un téléphone en 2 taps (appeler, puis noter l'issue).

À faire :

1. Partout où un numéro de client est affiché, le rendre cliquable : `<a href="tel:+33...">` avec normalisation du numéro (retirer espaces et points, convertir 0X en +33X). Ajouter un bouton "Appeler" de 44 px minimum sur chaque carte de relance et dans `todoList`.
 2. Après un tap sur "Appeler", au retour sur l'app (événement `visibilitychange`), proposer automatiquement une feuille du bas "Comment s'est passé l'appel ?" avec 3 gros boutons : "Joint", "Pas de réponse", "Message laissé", plus "Autre issue" qui déplie RDV pris, Réglé, Refus.
 3. Renommer l'issue `OUTCOMES.ok` en "Joint, OK" (sans tiret cadratin).
 4. Impayés en dessous de 860 px : remplacer le tableau par des cartes (nom, montant en gros, depuis, prochaine relance, bouton Appeler, bouton Récupéré, menu "..." pour statut, responsable, note et date). Le tableau reste au-dessus de 860 px.
 5. `resSaveIt` : ajouter un toast "Client sauvé" avec Annuler pendant 5 s (même mécanique que `toastUndo`), qui retire l'entrée `sv_<id>` et remet le statut précédent. La célébration reste.
 6. Dans `todoList`, l'action principale d'un impayé n'est plus "Récupéré" (qui ouvre une saisie de montant) mais "Appeler". "Récupéré" passe en action secondaire.
- Règles :

- Aucun emoji décoratif (remplacer les icônes emoji de `LOYALTY_TYPES` et de `todoList` par les icônes SVG de `ICONS`, ligne 843).

- Aucun tiret cadratin ni demi-cadratin dans les textes d'interface.

Critères d'acceptation :

- Sur mobile, depuis Relances, un appel puis la saisie de son issue se font en 2 taps.
- Aucun défilement horizontal sur la page Impayés à 375 px de large.
- Un "Sauvée" touché par erreur s'annule en 1 tap dans les 5 secondes, sans trace dans le KPI sauvetage.

7. RÉCOMPENSER LES BONS COMPORTEMENTS, PAS SEULEMENT LE VOLUME

Le classement global ne compte que les KPI obligatoires (`ranking` appelle `statsFor` avec `requiredOnly: true`, l. 1318). Or "Sauvetage résiliations" est non obligatoire (`DEFAULT_KPIS`, l. 856). Sauver un client fait donc une célébration plein écran mais ne bouge pas le rang. Les appels de rétention (`S.loyalty`), les appels notés sur une résiliation (`resCallSave`, l. 3928) et les prospects (0 point, l. 858) ne rapportent rien. Les trophées (`allTrophies`, l. 1355) sont tous des classements de résultat. Le commercial qui fait 40 relances utiles dans la semaine reste invisible. Spinify et Ambition savent valoriser des "activités" pondérées, à condition de les plafonner pour éviter la triche.

PROMPT CLAUDE

Contexte : fitpulse.html. `DEFAULT_KPIS` (848), `ranking` (1315), `statsFor` (1269), `allTrophies` (1355), `allTime` (1344), `S.loyalty`, `resCallSave` (3928), `loyAct` (3016).

Objectif : que les relances utiles et les sauvetages comptent dans la motivation, sans encourager les saisies de complaisance.

À faire :

1. Sauvetage : le compter dans le score du classement global quand il a un objectif. Option dans Mes clubs > réglages : "Inclure les sauvetages dans le classement" (cochée par défaut).
2. Créer des "points d'action" mensuels, séparés du score : Joint OK ou RDV pris = 10 pts, Réglé = 20 pts, appel noté sur une résiliation = 10 pts, Pas de réponse ou Message laissé = 2 pts. Plafond de 300 points d'action par semaine et par personne. Ils s'ajoutent au cumul allTime (niveaux) mais pas au score du classement.
3. Nouveaux trophées hebdomadaires par club dans allTrophies : "Sauveur de la semaine" (le plus de clients sauvés, minimum 1), "Relanceur de la semaine" (le plus d'issues positives en rétention, minimum 3). Les afficher avec les autres dans Profil > Mes trophées.
4. Sur l'accueil membre, dans la carte "Ma journée", une ligne "Actions de la semaine : 12 relances, 2 issues positives".
5. Documenter ces règles dans une bulle d'aide sur le classement : "Comment gagner des points".
Règles :
 - Seules les actions enregistrées via les boutons d'issue existants comptent, jamais une saisie libre.
 - Une même fiche client ne rapporte des points d'action qu'une fois par jour.
 - Aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin dans les textes d'interface.Critères d'acceptation :
 - Marquer un client "Sauvée" fait évoluer le score global du commercial quand un objectif sauvetage existe.
 - 50 issues "Pas de réponse" saisies dans la semaine ne rapportent jamais plus de 300 points d'action au total.
 - Les trophées "Sauveur" et "Relanceur" apparaissent dans le profil des gagnants la semaine suivante.

8. NIVEAUX ET TROPHÉES ÉQUITABLES POUR LES PETITS COMME POUR LES GROS

Le calcul des points est juste : tout est rapporté à l'objectif individuel. Mais l'échelle des niveaux est démesurée. Le maximum mensuel est d'environ 3 550 points (somme des points de DEFAULT_KPIS, l. 849 à 857). Il faut donc 3 mois parfaits pour "Performer" (10 000) et environ 28 mois parfaits pour "Légende" (100 000, LEVELS, l. 875). Un nouveau reste Rookie un trimestre. Les étapes à 25 % créent des effets de seuil : 24 % rapporte 0 point, 99 % en rapporte 75 % (tierOf, l. 1267). Dépasser 100 % ne rapporte plus rien. Enfin les trophées vont au premier du club, toujours le même. Strava récompense aussi le record personnel et la régularité.

PROMPT CLAUDE

Contexte : fitpulse.html. LEVELS (870), levelOf (1353), allTime (1344), tierOf et TIERS (1186, 1267), statsFor (1269), allTrophies (1355), accomplishments (1399), PAGES.profile (2594).

Objectif : une progression que chaque commercial peut sentir chaque mois, quel que soit son volume.

À faire :

1. Recalibrer LEVELS sur la base d'environ 3 500 pts par mois parfait : Rookie 0, Performer 3 000, Warrior 10 000, Élite 25 000, Légende 50 000. Afficher dans le profil une barre "Prochain niveau dans X pts, soit environ N semaines au rythme actuel".
2. Ajouter un bonus de dépassement : au-delà de 100 % d'un KPI, +10 % des points du KPI par tranche de 10 % supplémentaires, plafonné à 150 % (cohérent avec SCORE_CAP). Ces points comptent pour allTime, pas pour le score du classement.
3. Nouveaux trophées personnels, non compétitifs, dans allTrophies : "Record personnel" (meilleur score

mensuel depuis l'arrivée), "Plus belle progression" du club (plus forte hausse du score par rapport au mois précédent), "Régularité" (au moins une saisie 5 jours de suite, réutiliser `accomplishments.streak`).

4. Profil : afficher les 3 prochains trophées atteignables avec la condition restante ("Encore 2 contrats pour ton record personnel").

5. Afficher le niveau (badge) à côté du nom dans le classement et le fil d'équipe.

Règles :

- Ne pas changer le calcul du score de classement (`statsFor.score`).

- Aucun emoji décoratif : les icônes de trophées passent en SVG (voir le prompt visuel associé). Aucun tiret cadratin ni demi-cadratin dans les textes d'interface.

Critères d'acceptation :

- Un commercial à 100 % de ses objectifs atteint Performer à la fin de son premier mois complet.

- Un commercial classé dernier mais en progression de 20 points de score sur le mois précédent obtient le trophée "Plus belle progression" s'il a la plus forte hausse.

- Le profil indique en semaines le temps estimé jusqu'au niveau suivant.

PROMPT CHATGPT (VISUEL)

Crée une planche de 5 badges de niveau pour une application de pilotage commercial de clubs de fitness, niveaux : ROOKIE, PERFORMER, WARRIOR, ÉLITE, LÉGENDE. Style : écussons géométriques plats, inclinés de 8 degrés vers la droite comme une typographie italique sportive, contour épais. Palette stricte : jaune #FFD600, noir #0A0A0A, blanc, un gris #2A2A2E. La richesse visuelle progresse avec le niveau (Rookie simple trait, Légende plein jaune avec couronne stylisée). Nom du niveau en capitales Barlow Condensed ExtraBold italique sous chaque badge. Fond noir uni. Pas de dégradé arc-en-ciel, pas d'emoji, pas de personnage, pas de texte autre que les noms. Livrable : une image 2000 x 600 px avec les 5 badges alignés, plus chaque badge isolé en 512 x 512 px sur fond transparent.

9. CLASSEMENT BIENVEILLANT : LE PODIUM, MOI ET MES VOISINS

PAGES. `leaderboard` (l. 2175) affiche la liste complète, noms et pastilles rouges compris, et le Top 5 de l'accueil met un point coloré devant chaque collègue (`hdot`, l. 4250). Le dernier se voit publiquement "en retard". Le bandeau "Vous êtes #6 ce mois-ci" (l. 2199) contient un tiret cadratin et ne dit pas l'écart avec le suivant. Spinify et Salesforce recommandent de montrer le podium, sa propre ligne et ses voisins immédiats, avec l'écart à combler : c'est ce qui motive le milieu de tableau. Strava ajoute les "kudos" : ici, les réactions du fil (`ACTIONS.react`, l. 2244) existent mais ne servent à rien ailleurs.

PROMPT CLAUDE

Contexte : `fitpulse.html`. PAGES. `leaderboard` (2175 à 2213), PAGES. `home` Top 5 (4249 à 4250), `ranking` (1315), `S.reactions`.

Objectif : un classement qui motive le milieu et le bas de tableau sans humilier.

À faire :

1. Pour un membre (non manager) : afficher le podium (3 premiers), puis ma ligne et mes voisins immédiats (rang - 1 et rang + 1), puis "Voir tout le classement" replié. Les managers gardent la liste complète par défaut.

2. Sur ma ligne : écart chiffré au rang du dessus dans l'unité la plus parlante ("2 contrats pour passer 4e"), calculé sur le KPI où l'écart pondéré est le plus faible.

3. Supprimer les pastilles de santé (`hdot`) devant les noms des collègues dans le Top 5 et le classement pour les membres. Garder la mienne.

4. Bandeau "Vous êtes #6 ce mois-ci" : remplacer le tiret cadratin par un point médian et ajouter la tendance par rapport à la semaine précédente ("en hausse de 2 places").

5. Ajouter un onglet "Progression" au classement : tri par hausse du score par rapport à la même date du mois précédent.

6. Kudos : sous chaque ligne du podium, un bouton "Bravo" (une fois par jour et par personne) qui crée une réaction et notifie la personne dans le fil d'équipe.

Règles :

- Aucun emoji décoratif (les boutons de réaction du fil passent en icônes SVG avec libellé). Aucun tiret cadratin ni demi-cadratin dans les textes d'interface.

- Les égalités restent départagées comme aujourd'hui.

Critères d'acceptation :

- Un membre classé 6e sur 9 voit sans ouvrir la liste complète : le podium, les rangs 5, 6 et 7, et l'écart pour passer 5e.

- Aucune pastille rouge ou orange n'est affichée à côté du nom d'un collègue pour un membre.

- Un "Bravo" envoyé apparaît dans le fil d'équipe du destinataire.

10. DÉFIS FLASH : LES RENDRE VISIBLES, LISIBLES ET RÉCOMPENSÉS

Les défis sont bien pensés côté équité (normalisation par l'objectif, `challengeRanking`, l. 1417). Mais un défi en cours n'apparaît ni sur l'accueil, ni dans la tabbar : il faut aller dans le menu "Défis flash". Aucune récompense n'est affichée, alors que les paliers en ont (`reward`). Le classement affiche un "%" de rythme peu parlant ("norm", l. 2320). Un membre sans objectif sur le KPI obtient 100 % dès sa première vente (`norm: value > 0 ? 1 : 0`, l. 1430). Enfin un défi ne peut porter que sur un KPI de vente, jamais sur les relances.

PROMPT CLAUDE

Contexte : `fitpulse.html`. `PAGES.challenges` (2309), `ACTIONS.newChallenge` et `createChallenge` (2329 à 2344), `challengeRanking` (1417), `PAGES.home` (4211).

Objectif : un défi en cours se voit partout, se comprend en un regard et dit ce qu'on gagne.

À faire :

1. Accueil : si un défi est en cours sur le club, afficher en tête de page un bandeau jaune "Défi en cours : Sprint avis Google. 3 h 12 restantes. Tu es 2e." Un tap ouvre le détail.

2. Formulaire de création : ajouter un champ "Récompense" (texte libre, facultatif) et l'afficher sur la carte du défi et dans le bandeau.

3. Classement du défi : afficher en premier la valeur réalisée ("3 avis"), et en second "rythme 140 %" avec une bulle d'aide qui explique la normalisation.

4. `challengeRanking` : un membre sans objectif sur le KPI ne doit plus obtenir `norm = 1` par défaut. Utiliser la moyenne des objectifs de l'équipe comme référence pour lui.

5. Autoriser des défis sur les points d'action de relance (voir le point 7) : "Défi relances 24 h".

6. Proposer un mode "Défi d'équipe" : objectif collectif chiffré, barre commune, réussite partagée.

7. Remplacer l'emoji éclair du titre de page ("Défis flash", ligne 2323) et du toast "Défi lancé" par l'icône SVG bolt existante.

Règles :

- Aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin dans les textes d'interface.

Critères d'acceptation :

- Pendant un défi, le bandeau est visible sans défiler sur l'accueil mobile et indique mon rang.
- Un membre sans objectif sur le KPI et une seule vente n'apparaît pas automatiquement à 100 %.
- La récompense saisie s'affiche sur la carte du défi et dans le bandeau.

11. WRAP MENSUEL : LE FAIRE DÉCOUVRIR ET LE RENDRE POSITIF POUR TOUS

Le wrap (`PAGES.wrap`, l. 2640) est la meilleure idée "plaisir" de l'app, mais il est caché dans Profil > Mes bilans (l. 2619). Seul le manager a un bandeau "récap prêt" les 5 premiers jours (l. 4240). La slide "Ton classement" affiche "#8" même au dernier rang, et la phrase finale peut être "Mois difficile" (l. 2651). Le canvas partageable utilise les polices Anton et Inter (l. 2686 à 2698), qui ne sont pas chargées (seules Barlow Condensed et Montserrat le sont, l. 5) : rendu en Impact, hors charte. Les slides défilent toutes les 6 s sans pause. Les zones de tap sont des div sans rôle ni clavier (l. 2664).

PROMPT CLAUDE

Contexte : `fitpulse.html`. `PAGES.wrap` (2640), `wrapShow` (2670), `drawWrapCard` (2682), `profPerf` (2606), `PAGES.home` (4211).

Objectif : chaque commercial voit son bilan au début du mois et en ressort motivé, quel que soit son rang.

À faire :

1. Accueil membre, du 1er au 5 du mois : bandeau "Ton bilan de septembre est prêt" vers `#/wrap/<mois précédent>/<mon id>`, masqué une fois vu (préférence `wrapSeen_<mois>`).
 2. Slide classement : si je suis dans la moitié basse, remplacer le rang par ma progression (" +12 points de score par rapport à août") ou mon meilleur KPI. Le rang reste affiché si je suis dans la moitié haute.
 3. Ajouter une slide "Tes actions" : clients sauvés, impayés récupérés, relances avec issue positive.
 4. Phrases de fin : toujours orientées vers le mois suivant, avec l'action concrète tirée du KPI le plus en retard ("Le mois prochain : 2 avis Google de plus par semaine").
 5. `drawWrapCard` : utiliser 'Barlow Condensed' italique 800 pour les titres et 'Montserrat' pour le texte, attendre `document.fonts.ready` avant de dessiner.
 6. Contrôles : appui long = pause, flèches gauche et droite au clavier, zones de tap en `<button>` avec `aria-label` "Précédent" et "Suivant", Échap pour fermer. Respecter `prefers-reduced-motion` (pas d'autoplay).
 7. Bouton "Partager" via `navigator.share` avec le PNG si disponible, sinon téléchargement.
- Règles :

- Aucun emoji décoratif (la slide trophées utilise des icônes SVG). Aucun tiret cadratin ni demi-cadratin dans les textes d'interface ni dans l'image générée.

Critères d'acceptation :

- Un membre qui se connecte le 2 du mois voit le bandeau du bilan sur l'accueil.
- Le dernier du classement ne voit pas son rang, mais sa progression ou son meilleur KPI.
- L'image téléchargée utilise Barlow Condensed (vérifiable visuellement) et ne contient ni emoji ni tiret cadratin.

- Le wrap se pilote entièrement au clavier.

PROMPT CHATGPT (VISUEL)

Crée un gabarit de story verticale 1080 x 1920 px pour le bilan mensuel d'un commercial de club de fitness, à partager dans l'équipe. Palette stricte : noir #0A0A0A, jaune #FFD600, blanc. Typographie : Barlow Condensed ExtraBold italique pour les chiffres et titres en capitales, Montserrat pour le texte courant. Composition : en haut "FIT PULSE" petit en jaune, puis un prénom en très grand (placeholder "PRÉNOM"), le mois ("SEPTEMBRE 2026"), un grand pourcentage jaune ("92 %"), trois lignes de KPI avec barres de progression fines jaunes sur fond gris #2A2A2E, une bande diagonale jaune inclinée en bas à droite rappelant l'énergie des affiches Fitness Park. Aucun logo de marque réel, aucun emoji, aucun tiret long, aucune photo. Livrer une version remplie avec des valeurs fictives et une version vide avec zones nommées.

12. PAGE "MES OBJECTIFS" : MOINS DE CHIFFRES, PLUS DE LECTURE

`dashObjectives` (l. 1970) affiche pour un membre : mascotte, % des points, points, retard en points, 4 étapes, bouton, rythme attendu, taux de conversion, score pondéré, astuce de glisser-déposer, puis jusqu'à 9 cartes KPI de 5 lignes chacune (`kpiCard`, l. 2017). L'astuce "réorganisez par glisser-déposer" s'affiche aussi sur mobile, où le drag and drop HTML5 (`bindKpiDrag`, l. 2027) ne fonctionne pas au doigt. L'onglet "Analyses" (comparaison N/N-1, CA, l. 2041) est un outil de manager. Le sélecteur "Vue perso / Vue club" est en tête alors que le membre veut d'abord sa vue.

PROMPT CLAUDE

Contexte : `fitpulse.html`. `PAGES.dashboard` (1940), `dashObjectives` (1970), `kpiCard` (2017), `bindKpiDrag` (2027), `dashAnalyses` (2041).

Objectif : une page Mes objectifs lisible en 5 secondes par un commercial sur téléphone.

À faire :

1. Pour un membre sur mobile : hero réduit à 3 informations (score du mois, statut de rythme, "J-12"). Mascotte masquée sous 860 px. Les cartes "Taux de conversion" et "Score pondéré" passent dans un bloc repliable "Détails".

2. Cartes KPI en version compacte sur mobile : libellé, "7 / 12", barre avec repère de rythme, et une seule phrase (`paceMessage`). Les points et le statut texte passent en petit sous la barre.

3. Trier les cartes par défaut : KPI obligatoires en retard d'abord, puis les autres, puis ceux à 100 %.

4. Réorganisation : masquer l'astuce et la poignée de glisser-déposer sur les écrans tactiles (`matchMedia('(pointer: coarse)')`). Proposer à la place un bouton "Réorganiser" qui active des flèches monter et descendre.

5. Onglet "Analyses" : visible pour les managers uniquement, ou renommé "Mon historique" pour le membre avec seulement l'évolution de son score sur 6 mois.

6. "Vue club" pour le membre : renommée "L'équipe" et placée en second.

Règles :

- Aucun emoji décoratif (retirer les emoji de KPI et la couronne des cartes, remplacer par icônes SVG et un libellé "Obligatoire"). Aucun tiret cadratin ni demi-cadratin dans les textes d'interface.

Critères d'acceptation :

- Sur 375 px, le hero et les 3 premières cartes KPI tiennent dans le premier écran et demi.

- Aucune mention de glisser-déposer n'apparaît sur un appareil tactile.

- Un membre ne voit plus la comparaison N/N-1 du club.

13. ACCESSIBILITÉ : CIBLES, TAILLES, COULEUR SEULE, LECTEURS D'ÉCRAN

Les petits boutons font environ 30 px de haut (`.btn.sm { padding: 5px 10px; font-size: 12.5px }`, l. 115), sous les 44 px recommandés pour un pouce. Ils sont utilisés pour les actions de relance clés (resCard, todoList). Plusieurs textes descendent à 10,5 ou 11 px (`.tb`, l. 446 ; `.tiers span` en #77777c sur #26262a, l. 175, contraste d'environ 3:1 ; `.badge`, l. 125). L'état de santé des collègues passe uniquement par la couleur (`hdot`). Les toasts n'ont pas de région live (`#toasts`, l. 702). Seuls 8 `aria-label` existent : les chevrons de période (`lbShift`, l. 2184) et les boutons corbeille n'en ont pas. Aucun style `:focus-visible` sur `.btn`. Le compte à rebours de la topbar se met à jour chaque seconde (`tickCountdown`, l. 1664).

PROMPT CLAUDE

Contexte : fitpulse.html, CSS en tête de fichier et rendus HTML des pages.

Objectif : conformité WCAG 2.2 AA sur les écrans du rôle membre.

À faire :

1. Cibles tactiles : sous 860 px, `.btn.sm`, `.btn.icon`, `.seg button`, `.tabs button`, boutons de réaction et liens d'outils du chat ont une zone cliquable de 44 x 44 px minimum (padding ou pseudo-élément), sans changer le rendu desktop.
2. Tailles : aucun texte informatif sous 12 px sur mobile (`.tb`, `.badge`, `.eyebrow`, `.tiers span`, `.kpi .tierlbl`, `.palier-mark small`, `.brand-sub`).
3. Contraste : `.tiers span` non atteint au moins 4,5:1 (par exemple #B9B9BE sur #26262a). Vérifier tous les textes en `var(--muted)` sur `var(--surface-2)`.
4. Couleur seule : partout où `hdot` ou `hchip` indique un état, ajouter un libellé texte visible ou une forme différente, et un texte accessible (span visuellement masqué).
5. Lecteurs d'écran : `#toasts` avec `role="status" aria-live="polite"`. `aria-label` sur tous les boutons icône sans texte (chevrons de période et de mois, corbeille, fermer, joindre une image, envoyer). Les div de tap du wrap deviennent des boutons.
6. Clavier : style `:focus-visible` visible sur `.btn`, `.tb`, `.tb-main`, liens de navigation (contour 2 px `var(--fp)` avec décalage 2 px, et contour noir sur fond jaune). Piège de focus dans `openModal` et retour du focus sur l'élément déclencheur à la fermeture.
7. Compte à rebours : afficher "J-12" (mise à jour toutes les minutes) au lieu de jours, heures, minutes, secondes. Le détail à la seconde reste disponible en infobulle.

Règles :

- Aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin dans les textes d'interface (y compris le libellé "aucune valeur" de `HEALTH.none`, à remplacer par "Sans objectif").

Critères d'acceptation :

- Audit Lighthouse Accessibilité ≥ 95 sur Accueil, Relances, Classement en émulation mobile.
- Toutes les cibles interactives mesurent au moins 44 x 44 px à 375 px de large.
- Navigation complète au clavier : saisir une vente depuis l'accueil avec Tab et Entrée uniquement.
- Le DOM de la topbar n'est plus modifié chaque seconde.

14. CONNEXION, PREMIÈRE FOIS, ÉTATS VIDES ET ERREURS

La connexion demande l'e-mail et un code de 16 caractères "FP-XXXX-XXXX-XXXX" (PAGES.login, l. 1889 à 1897), pénible au doigt. Le champ n'a ni `autocapitalize`, ni `autocorrect="off"`, ni `spellcheck="false"`, et une erreur ne produit qu'un toast éphémère "E-mail ou code incorrect." (l. 1915). Aucune aide "code perdu". Après la première connexion, un toast "Bienvenue" et rien d'autre : pas de visite guidée des 3 gestes clés. Les états vides du membre sont des impasses : "Votre manager n'a pas encore fixé vos objectifs du mois" (l. 1973) ou "Aucune tâche configurée" sans action possible. Les emoji sont partout dans l'interface (couronne, KPI, trophées, toasts, réactions, environ 30 occurrences) et une soixantaine de tirets cadratins traînent dans les libellés, hors charte.

PROMPT CLAUDE

Contexte : fitpulse.html. PAGES.login (1873), soumission #lgc (1900 à 1916), login (1742), dashObjectives état vide (1971 à 1974), tasksToday (2164), todoList état vide (4198).

Objectif : une première connexion sans friction et des états vides qui proposent toujours une action.

À faire :

1. Champ code : `autocapitalize="characters"`, `autocorrect="off"`, `spellcheck="false"`, `inputmode="text"`, `maxlength 17`. Insertion automatique des tirets pendant la frappe (FP-ABCD-EFGH-JKLM). Bouton "Afficher" pour voir le code.
2. Erreur de connexion : message inline sous le formulaire (`role="alert"`), focus remis sur le champ code, texte "E-mail ou code incorrect. Vérifiez les tirets ou demandez un nouveau code à votre manager." Lien "Code perdu ?" qui affiche cette même consigne.
3. Session : rester connecté par défaut (déjà le cas), et l'indiquer sous le bouton "Vous resterez connecté sur cet appareil".
4. Première connexion d'un membre (préférence onboarded absente) : visite en 3 écrans plein écran, passables, sur fond noir et jaune : "Saisis une vente en 2 taps" (pointe le bouton Saisir), "Vois où tu en es" (pointe la carte Ma journée), "Traite tes relances" (pointe l'onglet Relances). Bouton "C'est parti".
5. États vides avec action : sans objectif, "Tes objectifs arrivent. En attendant, chaque vente saisie compte déjà dans le fil et les paliers d'équipe" + bouton "Saisir une vente". Sans relance, bouton "Prendre un dossier sans responsable" qui ouvre Relances filtré sur "Sans responsable".
6. Nettoyage de charte sur toute l'interface : remplacer chaque emoji affiché à l'utilisateur (emoji des DEFAULT_KPIS, LOYALTY_TYPES, icônes de allTrophies, réactions du fil et du chat, toasts, titres) par une icône SVG de l'objet ICONS (en ajouter si besoin : étoile, gobelet, stylo, casquette, billet, bouée, aimant, téléphone, gâteau, banque, trophée, médaille, couronne). Remplacer chaque tiret cadratin ou demi-cadratin d'un texte d'interface par une virgule, un point médian ou deux-points. Valeur absente : "Aucune" ou "Sans objet" au lieu d'un tiret.

Règles :

- Ne pas toucher aux commentaires de code ni aux données importées.
 - Aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin dans les textes d'interface.
- Critères d'acceptation :
- Taper "fpabcdehghjklm" dans le champ code affiche "FP-ABCD-EFGH-JKLM" et permet la connexion si le code est bon.
 - Une erreur de connexion reste affichée tant que l'utilisateur ne modifie pas un champ.
 - Un nouveau membre voit la visite en 3 écrans une seule fois.
 - Une recherche des caractères U+2014 et U+2013 et des plages emoji (U+1F300 à U+1FAFF, U+2600 à U+27BF) dans les chaînes rendues à l'écran ne renvoie aucun résultat.

PROMPT CHATGPT (VISUEL)

Dessine un jeu de 16 icônes au trait pour une application commerciale de club de fitness, destinées à remplacer des emoji : étoile (avis), gobelet shaker (nutrition), stylo de signature (contrat), casquette (accessoires), billet (impayé récupéré), immeuble (B2B), ticket (invité), bouée (sauvetage), aimant (prospect), téléphone (appel), gâteau (anniversaire), boucle de flèches (renouvellement), banque (mandat), trophée, médaille, éclair (défi). Style : trait 2 px, coins arrondis, grille 24 x 24, légère inclinaison italique de 8 degrés pour l'énergie sportive, monochrome. Fournir deux déclinaisons : noir #0A0A0A sur fond jaune #FFD600, et jaune #FFD600 sur fond noir. Aucun dégradé, aucun texte, aucun emoji. Livrable : planche 4 x 4 en 2048 x 2048 px, puis chaque icône en SVG propre.

CE QU'IL FAUT PRENDRE CHEZ LES MEILLEURS

- **SPINIFY** : célébration à chaque action, écart chiffré au suivant, défis visibles en permanence sur l'écran principal. Repris dans les points 3, 9 et 10.
- **SALESFORCE MOBILE, HUBSPOT MOBILE** : accueil "Aujourd'hui" orienté tâches, barre basse à 4 ou 5 destinations, appel en un tap puis consignation de l'issue au retour. Repris dans les points 1, 5 et 6.
- **PIPEDRIVE** : une seule liste d'activités priorisée, un seul indicateur d'objectif lisible. Repris dans les points 4 et 6.
- **STRAVA** : records personnels, progression plutôt que rang absolu, kudos entre pairs, bilan partageable. Repris dans les points 8, 9 et 11.

3. RELANCES ET CRM DE RÉTENTION

Constat général. Fit Pulse a déjà trois briques de relance, mais elles ne se parlent pas :

- Action Rétention (`PAGES.loyalty`, l. 2979) : tâches calculées à la volée par `loyaltyTasks()` (l. 1454), résultats stockés dans `S.loyalty` (`clientId`, `type`, `outcome`, `userId`, `note`, `at`). Pas de responsable, pas de date de rappel.
- Impayés (`PAGES.impayés`, l. 3994) : statut, responsable, prochaine relance et note rangés dans `S.clients[id].dunning`, historique écrit dans `dunning.history` (l. 4049) mais jamais affiché.
- Résiliations (`PAGES.resiliations`, l. 3873) : `S.resiliations[id].actions[]`, avec le client stocké en texte libre (`r.client`), sans `clientId` ni téléphone.

La page Relances (`PAGES.relances`, l. 4273) n'est qu'une liste `todoList(99)` avec trois liens. Aucun lien `tel:`, `sms:`, `wa.me` ou `mailto:` n'existe dans le fichier. Il n'y a ni script, ni modèle de message, ni cadence, ni mesure de joignabilité.

Les points ci-dessous vont du plus impactant au moins impactant. Chaque prompt est autonome.

1. UN SEUL MODÈLE DE RELANCE ET UN SEUL JOURNAL D'INTERACTIONS

Trois vocabulaires coexistent pour le même geste : `OUTCOMES` (l. 1443), `RES_CALLS` (l. 3864), `DUN_STATUS` (l. 3980). Un même client peut être appelé pour un impayé dans Action Rétention (`loyAct`, l. 3016) et dans Impayés (`dunSet`, l. 4052) sans que l'un voie l'autre. Les résiliations ne pointent vers aucune fiche client. Sans socle commun, ni timeline, ni anti-doublon, ni mesure ne sont possibles. C'est le chantier qui débloque tous les autres.

PROMPT CLAUDE

Contexte : Fit Pulse est une appli web en un seul fichier HTML (`fitpulse.html`). L'état global est `S`, modifié uniquement via `db.set(path, value)` et `db.batch([[path, value], ...]).emptyState()` (vers l. 878) liste les collections. `normalizeState()` (vers l. 983) complète un état ancien. `hkey()` (vers l. 3199) produit un identifiant stable à partir d'une chaîne.

Objectif : créer un socle unique pour toutes les relances, sans casser Action Rétention, Impayés et Résiliations.

1. Modèle de données. Ajoute dans `emptyState()` et `normalizeState()` :

```
- S.relances = {} ; chaque relance :
  { id, clubId, clientId, kind, refId, anchor, status, ownerId, claimedBy, claimedUntil,
    dueDate, nextAt, step, attempts, amount, createdAt, closedAt, closedBy, result, lostReason }
  kind parmi :
    'suivi15', 'suivi30', 'anniversaire', 'fincontrat', 'impaye', 'mandat', 'resiliation', 'prospect', 'invite', 'libre'
    status parmi : 'todo', 'encours', 'attente', 'gagne', 'perdu', 'annule'.
    anchor = date qui définit l'occurrence (date de début de contrat, anniversaire de l'année, date de fin, balanceAt, date de la demande de résiliation).
    id détermine : 'rl' + hkey(clubId + '|' + clientId + '|' + kind + '|' + anchor).
- S.touches = {} ; chaque interaction :
  { id, clubId, clientId, relanceId, at, by, channel, outcome, reason, note, callbackAt,
    promiseAmount, promiseDate, rdvAt, templateId }
  channel parmi : 'call', 'sms', 'whatsapp', 'email', 'visite', 'note'.
  outcome parmi :
    'joint', 'messengerie', 'pasreponse', 'mauvaisnumero', 'rappeler', 'promesse', 'rdv', 'paye', 'sauve', 'refus', 'ok',
    'envoye'.
- S.relanceCfg = {} ; par club : { lockMinutes: 15, maxAttempts: 4, quietFrom: '09:00', quietTo: '20:00', sundayOff: true }.
```

- S.templates = {} (rempli au point des scripts).

2. Une constante unique TOUCH_OUTCOMES = { joint:{label:'Joint',reached:true}, messagerie:{label:'Messagerie',reached:false}, pasreponse:{label:'Pas de réponse',reached:false}, mauvaisnumero:{label:'Mauvais numéro',reached:false}, rappeler:{label:'Rappeler le',reached:true}, promesse:{label:'Promesse de paiement',reached:true}, rdv:{label:'Rendez-vous',reached:true}, paye:{label:'Payé',reached:true,win:true}, sauve:{label:'Sauvé',reached:true,win:true}, refus:{label:'Refus',reached:true,lose:true}, ok:{label:'Fait',reached:true,win:true}, envoye:{label:'Message envoyé',reached:false} }.

3. Fonction syncRelances(clubId) : reprend les règles de loyaltyTasks() (l. 1454), y ajoute 'resiliation' (une relance par dossier ouvert de S.resiliations, refId = id du dossier) et 'impaye' (refId = clientId, amount = balance). Elle crée les relances manquantes, ne modifie jamais une relance déjà touchée, et passe en 'annule' (result: 'auto') une relance 'todo' dont la condition a disparu. Appelle-la au boot, après rsmCommit, après wizCommit, et une fois par jour (clé safeLS 'parkpulse.relSync.<club>').

4. Lien résiliation vers client : ajoute r.clientId. Rapprochement par tokensKey(r.client) sur S.clients du club ; si 0 ou plusieurs résultats, laisse null et affiche un bouton « Lier à une fiche » dans resOpen.

5. Migration non destructive : fonction migrateTouches() appelée une fois (drapeau S.meta.touchesV1) qui recopie S.loyalty, S.resiliations[].actions et S.clients[].dunning.history en S.touches avec la bonne correspondance d'issue. Les anciennes collections restent lisibles et ne sont plus écrites par le nouveau code.

6. Toute action existante (loyAct, resCallSave, resSaveIt, resLose, dunSet, dunTake, dunPaidSave) écrit en plus un S.touches et met à jour la relance liée.

Règles d'interface : aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin dans les textes.

Critères d'acceptation :

- Après chargement de la démo, Object.keys(S.relances).length > 0 et chaque relance a un clientId existant.
- Appeler syncRelances deux fois de suite ne crée aucune relance en plus.
- Noter « Pas de réponse » dans Résiliations crée un S.touches avec channel 'call', outcome 'pasreponse', relanceId du dossier.
- Les pages Action Rétention, Impayés et Résiliations s'affichent comme avant.
- grep -P "[\x{2013}\x{2014}]" sur le code ajouté ne renvoie rien.

2. UNE VRAIE FILE D'APPELS PRIORISÉE, MODE SESSION

Aujourd'hui PAGES.relances (l. 4273) affiche myToDo() (l. 4185) : les résiliations et impayés à mon nom, plus les appels de fidélisation de tout le club (pas les miens), et jamais les tâches impaye ou mandat d'Action Rétention. Le bouton d'une ligne impayé ouvre directement « Récupéré » (dunPaid, l. 4195) au lieu d'un appel. Il faut un écran unique, trié par un score, qui enchaîne les appels comme un power dialer (Aircall, Ringover).

PROMPT CLAUDE

Contexte : fitpulse.html, page PAGES.relances (vers l. 4273), onglet « Relances » de la barre mobile (tabBar, vers l. 4307). Le socle S.relances / S.touches / TOUCH_OUTCOMES existe (sinon crée-le d'abord).

Objectif : remplacer la page Relances par une file d'appels priorisée et un mode session.

Score de priorité, fonction relScore(rl) :

- base par kind : resiliation 60, impaye 50, fincontrat 40, mandat 35, suivi15 32, suivi30 30, prospect 25, invite 25, anniversaire 15, libre 20 ;

- impaye : + min(30, amount / 10) ;
- resiliation : + 40 si date effective dans 7 jours ou moins ;
- rappel planifié (nextAt) atteint : + 40 ; nextAt dans le futur : la relance sort de la file jusqu'à l'heure ;
- retard : + 3 par jour après dueDate (plafond 30) ;
- - 5 par tentative déjà faite.

Écran :

1. En-tête : « À appeler maintenant » (nombre), « Rappels du jour », « Sans téléphone », bouton « Démarrer la session ».
 2. Filtres en segments : Mes relances / Non attribuées / Tout le club, puis par kind avec compteurs.
 3. Liste triée par score. Chaque ligne : nom, kind en badge texte (pas d'icône emoji), raison en une ligne (ex. « 87,40 € dus depuis le 12/09 », « fin de contrat le 18/10 », « J+15, adhérent depuis le 20/09 »), responsable, nombre de tentatives, dernière interaction.
 4. Boutons par ligne : Appeler, SMS, WhatsApp, E-mail (voir le point « un tap »), Noter, Plus tard.
 5. Section repliée « Sans téléphone » : relances dont le client n'a pas de phone valide, avec bouton « Ajouter le numéro ».
 6. Mode session : plein écran mobile, une fiche à la fois. Affiche le script du kind, les infos utiles (offre, montant, dates, dernière note), les boutons de contact, puis la feuille de résultat. Après enregistrement, passe automatiquement à la suivante. Boutons Passer et Terminer. Compteur « 7 sur 23 » et minuteur de session.
 7. Sur l'accueil, todoList() pointe vers la relance (ouvre la fiche relance), jamais directement vers « Récupéré ».
- Règles : aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin. Mobile d'abord, cibles tactiles de 44 px minimum, aucun défilement horizontal à 360 px.
- Critères d'acceptation :
- Avec la démo, la première ligne de la file est une résiliation à moins de 7 jours ou le plus gros impayé.
 - Une relance avec nextAt dans 2 heures n'apparaît pas dans « À appeler maintenant » mais dans « Rappels du jour ».
 - Les tâches impayé et sans mandat apparaissent dans la file.
 - En mode session, enregistrer un résultat affiche la fiche suivante sans retour à la liste.

3. LE TÉLÉPHONE MANQUE : LA BASE RESAMANIA N'EN APPORTE PAS

Aucune définition Resamania ne lit de téléphone : `clients` (l. 3378) ne prend que numéro, nom, anniversaire, état et commercial ; `ventes` (l. 3273), `incidents` (l. 3336), `sans-mandat` (l. 3369) non plus. Seul l'assistant générique « Résumé clients » (`impNew`, l. 2795) mappe une colonne téléphone. Pire, les deux imports créent des doublons : l'assistant crée des fiches sans `num` (rapprochement par nom ou e-mail, l. 2882), `rsmCommit` crée des fiches par `num` (l. 3688). On obtient une fiche avec téléphone et une fiche avec impayé. Résultat : « pas de téléphone » partout (l. 3010).

PROMPT CLAUDE

Contexte : fitpulse.html. RSM_DEFS (vers l. 3271) décrit les exports Resamania ; chaque parse(c) appelle c.client(num, {...}). rsmCommit (vers l. 3681) fusionne dans S.clients via clientIdx ('n:'+num, 't:'+tokensKey(nom)). L'assistant générique (wizNext/wizCommit, vers l. 2853-2928) importe « Résumé clients » avec phone et email mais sans num.

Objectif : que chaque client à relancer ait un téléphone fiable, sans doublon de fiche.

1. Lecture : dans les parse 'clients', 'ventes', 'incidents', 'clients-incident', 'sans-mandat', cherche les colonnes par c.find(h => /portable|mobile|telephone|tel/.test(h)) et /mail/. Passe phone et email à c.client(). Ne remplace jamais un téléphone saisi à la main (phoneSrc === 'manual').

2. Normalisation : fonction phoneE164(raw) : retire espaces, points, tirets ; '0X.....' devient '+33X.....' ; '0033' devient '+33' ; garde '+262', '+590' etc. ; renvoie null si moins de 9 chiffres utiles. Stocke S.clients[id].phone (E.164), phoneRaw, phoneSrc ('rsm','wizard','manual'), phoneBad (true si signalé « mauvais numéro »). Affichage : fonction phoneFmt() en « 06 12 34 56 78 ».

3. Assistant générique : ajoute une colonne optionnelle « N° client Resamania » au mapping de 'clients' et 'soldes'. findC rapproche d'abord par num, puis e-mail, puis nom + naissance.

4. Fusion : fonction mergeClients(keepId, dropId) qui fusionne les champs (le non vide gagne, phone manuel prioritaire), réécrit clientId dans S.relances, S.touches, S.loyalty, S.resiliations, puis supprime dropId. Écran Membres > Doublons clients : liste les paires de même tokensKey dans un club, dont l'une a num et l'autre non, avec bouton « Fusionner ».

5. Saisie rapide : sur toute relance sans téléphone, bouton « Ajouter le numéro » (champ tel, validation phoneE164, phoneSrc 'manual').

6. Indicateur sur la page Imports : « Clients avec téléphone : 312 sur 540 (58 %) » et lien vers la liste des relances sans téléphone.

7. Aide d'export : dans la description de la définition 'clients', ajoute la consigne d'ajouter la colonne Téléphone portable dans l'export Resamania si la liste le permet ; sinon recommander l'export de gestion qui la contient.

Règles : aucun emoji, aucun tiret cadratin ni demi-cadratin dans les textes.

Critères d'acceptation :

- phoneE164('06 12 34 56 78') === '+33612345678' ; phoneE164('0033612345678') === '+33612345678' ; phoneE164('12') === null.
- Importer un CSV clients avec colonne « Téléphone portable » renseigne phone sur les fiches par num.
- Importer « Résumé clients » avec num puis « Clients en incident » ne crée qu'une fiche par client.
- Fusionner deux fiches conserve l'historique d'interactions des deux.

4. CONTACTER EN UN TAP ET REVENIR NOTER LE RÉSULTAT

Le téléphone est affiché en texte gris (l. 3010, l. 4035, l. 4196). Sur mobile, le commercial doit le recopier. Les outils de référence ouvrent l'appel en un tap puis proposent la feuille de résultat au retour dans l'appli. C'est ce retour automatique qui fait que le résultat est réellement noté.

PROMPT CLAUDE

Contexte : fitpulse.html. Les clients ont S.clients[id].phone en E.164 (sinon ajoute phoneE164()). S.touches et S.templates existent. ACTIONS est le registre des data-act ; openModal() ouvre une modale.

Objectif : boutons Appeler, SMS, WhatsApp, E-mail fonctionnels sur toute relance, avec journalisation et feuille de résultat au retour.

1. Fonction contactLinks(client, relance, templateId) qui renvoie :

- tel:+33612345678
- sms:+33612345678?&body=<texte encodé> (forme compatible iOS et Android)
- https://wa.me/33612345678?text=<texte encodé> (sans le +)
- mailto:adresse?subject=...&body=...

Le texte vient du modèle choisi, variables remplacées (voir le point des modèles).

2. Boutons rendus en vraies balises <a href> (pas de window.open) pour que le système ouvre l'appli native. Bouton grisé avec info-bulle « Pas de numéro » si phone absent ou phoneBad, « Pas d'e-mail » si email absent, « Opposé aux SMS » si client.optOutSms.

3. SMS, WhatsApp, E-mail : avant d'ouvrir, une mini modale propose les modèles du kind et un aperçu modifiable. Au clic sur Envoyer, écrire un S.touches { channel, outcome: 'envoye', templateId, note: texte final } puis ouvrir le lien.

4. Appel : au clic, mémoriser UI.pendingCall = { relanceId, clientId, at: Date.now() }. Écouter document.visibilitychange : quand la page redevient visible et que pendingCall a moins de 2 heures, ouvrir la feuille de résultat (point suivant) pré-remplie avec channel 'call'. Si l'utilisateur ferme sans rien choisir, ne rien écrire.

5. Bureau : sur écran large, le bouton Appeler affiche aussi le numéro en gros avec « Copier ».

6. Respect des horaires : si l'heure locale est hors S.relanceCfg[club].quietFrom / quietTo, ou dimanche avec sundayOff, afficher une confirmation « Il est 21 h 10. Appeler quand même ? » (appel) ou « Programmer l'envoi demain à 9 h ? » (le message est alors gardé en relance avec nextAt).

Règles : aucun emoji, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- Sur un client +33612345678, le lien WhatsApp est exactement https://wa.me/33612345678?text=... .
- Après un tap sur Appeler puis retour dans l'onglet, la feuille de résultat s'ouvre seule.
- Un SMS envoyé apparaît dans la timeline du client avec le texte envoyé.
- Aucun bouton actif pour un client sans numéro.

5. FEUILLE DE RÉSULTAT ET PROCHAINE ACTION DATÉE

loyAct (l. 3016) n'enregistre qu'une issue, avec une note libre seulement pour « RDV » et « Refus ». « Rappeler jeudi » n'existe que comme texte (placeholder l. 3925). Il n'y a pas de date de rappel dans S.loyalty ni dans S.resiliations. Le motif de refus est du texte libre, donc inexploitable. Une relance sans prochaine action datée est une relance perdue.

PROMPT CLAUDE

Contexte : fitpulse.html avec S.relances, S.touches, TOUCH_OUTCOMES.

Objectif : une feuille de résultat unique, ouverte après chaque contact, qui impose une suite.

- Feuille (modale tiroir sur mobile) : choix de l'issue en gros boutons selon le kind :
 - tous : Joint, Messagerie, Pas de réponse, Mauvais numéro, Rappeler le, Refus ;
 - impaye : + Promesse de paiement, Payé ;
 - resiliation : + Sauvé, et liste « Solution proposée » (reprend RES_OFFERS) ;
 - suivi15, suivi30, prospect, invite : + Rendez-vous ;
 - anniversaire, mandat, fincontrat : + Fait.
- Champs conditionnels :
 - Rappeler le : date et heure obligatoires, raccourcis « Ce soir 18 h », « Demain 10 h », « Lundi 10 h » ;
 - Promesse : montant (défaut = solde) et date obligatoires ;
 - Rendez-vous : date, heure, objet (Bilan coach, Visite, Signature) ;

- Refus : motif obligatoire dans une liste par kind (resiliation : RES_REASONS ; impaye : Conteste la dette, Difficultés financières, Ne répond plus, Déjà payé ; fincontrat : Prix, Déménagement, Concurrence, Ne vient plus, Santé, Autre) ;
- Note : facultative partout, 280 caractères.

3. Effets sur la relance :

- Messagerie, Pas de réponse : attempts + 1, nextAt calculé par la cadence (point suivant) ;
- Mauvais numéro : client.phoneBad = true, la relance passe dans « Sans téléphone » ;
- Rappeler le, Promesse, Rendez-vous : status 'attente', nextAt = date saisie (promesse : lendemain de la date promise à 10 h pour vérifier) ;
- Payé, Sauvé, Fait, Joint sur kind anniversaire : status 'gagne', closedAt, closedBy ;
- Refus : status 'perdu', lostReason ;
- attempts >= maxAttempts sans contact joint : status 'perdu', lostReason 'Injoignable'.

4. Payé déclenche le flux existant dunPaidSave (canal, montant, KPI). Sauvé déclenche resSaveIt.

5. Le bouton Enregistrer reste désactivé tant qu'un champ obligatoire manque.

6. Toast de confirmation qui annonce la suite : « Noté. Prochaine relance jeudi 10 h. »

Règles : aucun emoji, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- Impossible d'enregistrer « Rappeler le » sans date.
- Une promesse de 50 € au 15/10 met la relance en attente jusqu'au 16/10 10 h.
- Trois « Pas de réponse » enregistrés la même minute ne ferment pas la relance (voir cadence).
- Les motifs de refus sont comptables par valeur dans le tableau de bord.

6. CADENCES AUTOMATIQUES ET DEUX BUGS DE LOGIQUE DANS LES SUIVIS

loyaltyTasks() regroupe J+15 et J+30 sous le même type suivi avec since = c.start (l. 1464). Un J+15 réussi (ok, done: true) ferme donc aussi le J+30, qui n'apparaît jamais. Ensuite, MAX_ATTEMPTS = 3 (l. 1452) compte les tentatives sans espacement : trois taps « Pas de réponse » en une minute classent le client en Perdus. Enfin, « Joint, OK » ferme une tâche impayé alors que le solde reste dû (l. 1474 à 1478). Une cadence type HubSpot Sequences règle les trois.

PROMPT CLAUDE

Contexte : fitpulse.html. loyaltyTasks() (vers l. 1454) et syncRelances() produisent les relances. S.relanceCfg[club] existe.

Objectif : séparer J+15 et J+30, et automatiser la relance suivante selon une cadence par type.

1. Corriger la génération : kind 'suivi15' (fenêtre start + 13 à start + 20) et 'suivi30' (start + 28 à start + 40), deux relances indépendantes, anchor = c.start. Un J+15 gagné ne ferme pas le J+30.
2. Impayé : seuls Payé ou la disparition du solde à l'import ferment la relance. Joint passe la relance en 'attente' avec nextAt + 3 jours si aucune promesse n'est saisie.
3. Cadences par défaut dans S.relanceCfg[club].cadences, éditables par le manager (Membres > Réglages relances) :
 - impaye : J0 appel ; J0 SMS si non joint ; J+2 appel ; J+5 WhatsApp ; J+9 appel ; J+14 e-mail récapitulatif ; puis perdu ou escalade manager.
 - mandat : J0 appel ; J+2 SMS avec lien ou consigne d'accueil ; J+7 appel ; J+14 appel.
 - resiliation : J0 appel ; J+1 appel ; J+3 SMS ; puis tous les 2 jours jusqu'à la date effective.
 - fincontrat : J-30 appel ; J-21 SMS ; J-14 appel ; J-7 appel ; J-2 SMS.

- suivi15, suivi30 : J0 appel ; J+1 SMS si non joint ; J+3 appel.
 - anniversaire : le jour J, SMS ou WhatsApp, une seule étape.
 - prospect, invite : J+1 appel ; J+2 SMS ; J+5 appel ; J+12 appel.
- Format : [{ offsetDays, channel, templateId }].

4. Après une issue non jointe, step + 1 et nextAt = anchor ou date de la tentative + offset de l'étape suivante, ramené dans les horaires autorisés (quietFrom, sundayOff). Minimum 4 heures entre deux tentatives d'appel.

5. Une étape SMS ou WhatsApp apparaît dans la file avec le modèle prêt : un tap ouvre le message pré-rempli (pas d'envoi automatique, l'appli n'a pas de passerelle SMS).

6. Fin de cadence sans contact : status 'perdu', lostReason 'Injoignable', visible dans l'onglet Perdus avec bouton Relancer (qui remet step à 0).

Règles : aucun emoji, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- Un client inscrit il y a 30 jours avec un J+15 gagné a une relance suivi30 à faire.
- Après « Pas de réponse » sur un impayé un lundi à 11 h, la relance réapparaît le lundi même pour le SMS puis le mercredi pour l'appel.
- Aucune relance n'est planifiée un dimanche si sundayOff est vrai.
- Un « Joint » sur un impayé ne retire pas le dossier de la file.

7. SCRIPTS D'APPEL ET MODÈLES DE MESSAGES RÉDIGÉS

Aucun texte d'aide n'existe : le hint de LOYALTY_TYPES (l. 1436) tient en cinq mots. Les nouveaux commerciaux improvisent. Il faut un script court par type, affiché pendant l'appel, et des modèles SMS, WhatsApp et e-mail avec variables. Les textes ci-dessous sont prêts à intégrer.

PROMPT CLAUDE

Contexte : fitpulse.html. Créer S.templates et les scripts d'appel. Variables disponibles : {prenom}, {nom}, {club}, {commercial}, {montant}, {date_fin}, {offre}, {date_rdv}, {date_promesse}, {lien_paiement}, {horaires_accueil}. Fonction fillTemplate(body, ctx) : remplace les variables connues, laisse un repère visible [variable manquante] sinon, et bloque l'envoi tant qu'il en reste.

1. Modèle : S.templates[id] = { id, clubId (null = tous les clubs), kind, channel ('sms', 'whatsapp', 'email', 'script'), label, subject, body, active, updatedBy, updatedAt }. Le manager édite les modèles dans Membres > Modèles de messages (liste par kind, aperçu avec un client exemple, compteur de caractères SMS : 160 par segment).

2. Scripts (channel 'script'), affichés en mode session en 3 blocs : Ouverture, Questions, Conclusion. Textes par défaut :

suivi15 :

Ouverture : « Bonjour {prenom}, c'est {commercial} du Fitness Park {club}. Je vous appelle juste pour savoir comment se passent vos deux premières semaines. Vous avez deux minutes ? »

Questions : « Vous venez combien de fois par semaine ? Vous avez trouvé vos repères sur les machines ? Vous avez déjà fait votre séance avec un coach ? »

Conclusion : « Je vous propose un créneau avec un coach pour caler un programme. Plutôt en semaine ou le week-end ? » Si tout va bien : « Parfait. Si vous avez la moindre question, demandez-moi à l'accueil. »

suivi30 :

Ouverture : « Bonjour {prenom}, {commercial} du Fitness Park {club}. Ça fait un mois que vous êtes avec nous, je voulais faire le point. »

Questions : « Vous atteignez ce que vous visiez en vous inscrivant ? Qu'est-ce qui vous aiderait à venir plus souvent ? Vous connaissez quelqu'un qui aimerait essayer ? »

Conclusion : « Je peux lui offrir une séance découverte. Vous me donnez son prénom et son numéro ? »

anniversaire :

Ouverture : « Bonjour {prenom}, c'est {commercial} du Fitness Park {club}. On voulait simplement vous souhaiter un bon anniversaire de la part de toute l'équipe. »

Conclusion : « Passez nous voir à l'accueil cette semaine, on a une petite attention pour vous. »

fincontrat :

Ouverture : « Bonjour {prenom}, {commercial} du Fitness Park {club}. Votre engagement {offre} arrive à échéance le {date_fin}, je voulais anticiper avec vous. »

Questions : « Comment s'est passée cette année ? Vous comptez continuer ? Votre formule correspond toujours à votre pratique ? »

Conclusion : « Si vous renouvelez avant le {date_fin}, je vous garde les conditions actuelles. On le fait ensemble à l'accueil ou par téléphone ? »

impaye :

Ouverture : « Bonjour {prenom}, {commercial} du Fitness Park {club}. Je vous appelle au sujet d'un prélèvement qui n'est pas passé, pour un montant de {montant}. Ça arrive souvent, je voulais simplement régler ça avec vous. »

Questions : « Vous étiez au courant ? Votre carte ou votre compte a changé récemment ? »

Conclusion : « Vous pouvez régler à l'accueil lors de votre prochaine séance ou en ligne. À quelle date je peux noter le règlement ? » Noter la promesse avec date et montant.

mandat :

Ouverture : « Bonjour {prenom}, {commercial} du Fitness Park {club}. Il manque le mandat de prélèvement sur votre abonnement, ce qui peut bloquer votre accès. »

Conclusion : « Ça prend deux minutes à l'accueil avec votre RIB. Vous passez quand cette semaine ? »

resiliation :

Ouverture : « Bonjour {prenom}, {commercial} du Fitness Park {club}. J'ai bien reçu votre demande de résiliation. Avant de la traiter, je voulais comprendre ce qui vous amène à arrêter. »

Questions : « Qu'est-ce qui a changé pour vous ? Si on trouvait une solution à ce point, vous resteriez ? »

Conclusion selon le motif : Prix : « Je peux regarder une formule plus adaptée. » Manque de temps ou santé : « On peut suspendre votre abonnement plutôt que l'arrêter. » Insatisfaction : « Je vous propose une séance avec un coach pour relancer votre programme. » Toujours finir par : « Je note votre décision et je vous confirme par SMS. »

prospect / invite :

Ouverture : « Bonjour {prenom}, {commercial} du Fitness Park {club}. Vous êtes passé nous voir récemment, je voulais savoir ce que vous en avez pensé. »

Questions : « Qu'est-ce qui vous a plu ? Qu'est-ce qui vous retient pour vous inscrire ? »

Conclusion : « Je vous propose de revenir pour une séance offerte avec un coach. Jeudi ou samedi, qu'est-ce qui vous arrange ? »

3. Modèles SMS (channel 'sms', identiques par défaut en 'whatsapp') :

- suivi15 : « Bonjour {prenom}, c'est {commercial} du Fitness Park {club}. J'ai essayé de vous joindre pour savoir comment se passent vos débuts. Une question, un besoin ? Répondez ici, je vous rappelle. »

- suivi30 : « Bonjour {prenom}, déjà un mois au Fitness Park {club}. Envie d'un point avec un coach pour garder le rythme ? Répondez OUI et je vous propose un créneau. {commercial} »

- anniversaire : « Joyeux anniversaire {prenom} ! Toute l'équipe du Fitness Park {club} vous souhaite une belle journée. Une petite attention vous attend à l'accueil cette semaine. »

- fincontrat : « Bonjour {prenom}, votre engagement {offre} se termine le {date_fin}. Renouvelez avant cette date pour garder vos conditions. Je reste dispo pour en parler. {commercial}, Fitness Park {club} »

- impaye (1er message) : « Bonjour {prenom}, un prélèvement de {montant} n'a pas pu être effectué sur votre abonnement Fitness Park {club}. Vous pouvez régulariser à l'accueil ou en ligne : {lien_paiement}. Merci, {commercial} »

- impaye (rappel) : « Bonjour {prenom}, sauf erreur, le montant de {montant} reste à régler. Passez à l'accueil ou répondez à ce message pour convenir d'une date. Merci. {commercial}, Fitness Park {club} »

- impaye (promesse) : « Merci {prenom}, c'est noté : règlement de {montant} le {date_promesse}. Bonne séance ! {commercial} »

- mandat : « Bonjour {prenom}, il manque le mandat de prélèvement sur votre abonnement Fitness Park {club}. Passez à l'accueil avec votre RIB, cela prend deux minutes. {horaires_accueil} »
- resiliation : « Bonjour {prenom}, j'ai bien reçu votre demande. Avant de la traiter, j'aimerais en parler deux minutes avec vous. Quand puis-je vous appeler ? {commercial}, Fitness Park {club} »
- rdv (confirmation) : « C'est noté {prenom} : rendez-vous le {date_rdv} au Fitness Park {club}. À bientôt ! {commercial} »
- prospect : « Bonjour {prenom}, merci de votre visite au Fitness Park {club}. Une séance découverte avec un coach vous est offerte. Répondez avec le jour qui vous arrange. {commercial} »

4. E-mail impayé récapitulatif (J+14) : objet « Votre abonnement Fitness Park {club} : régularisation de {montant} », corps de 5 lignes reprenant le montant, les moyens de règlement et le contact du club.

Règles : aucun emoji, aucun tiret cadratin ni demi-cadratin dans les textes et l'interface. Les points d'exclamation restent rares (anniversaire, confirmation).

Critères d'acceptation :

- Chaque kind a au moins un script et un modèle SMS actifs après normalizeState().
- fillTemplate('Bonjour {prenom}', {prenom:'Léa'}) === 'Bonjour Léa'.
- Un modèle avec {lien_paiement} vide bloque l'envoi et affiche « [lien_paiement manquant] ».
- grep sur les modèles : aucun caractère emoji, aucun caractère U+2013 ou U+2014.

8. ATTRIBUTION ET ANTI-DOUBLON ENTRE COLLÈGUES

Action Rétention n'a aucun responsable : `S.loyalty` stocke seulement l'auteur de l'action. Deux commerciaux peuvent appeler le même adhérent le même matin, ce qui agace le client et fausse les stats. Impayés et Résiliations ont un `ownerId` mais pas de verrou pendant l'appel. Les outils pro posent prise en charge, verrou temporaire et alerte « déjà contacté aujourd'hui ».

PROMPT CLAUDE

Contexte : fitpulse.html avec `S.relances` (`ownerId`, `claimedBy`, `claimedUntil`) et `S.touche`s. Synchronisation possible entre appareils (backend Firebase ou local).

Objectif : un client n'est jamais relancé deux fois par erreur.

1. Attribution :

- Bouton « Je m'en occupe » sur toute relance non attribuée (comme `dunTake`, l. 4058 et `resTake`, l. 3919) : `ownerId = ME.id`.
- Manager : sélection multiple et « Attribuer à » ; bouton « Répartir équitablement » qui distribue les relances non attribuées du filtre courant entre les membres actifs du club au prorata du nombre déjà en cours.
- Règle par défaut : une relance impayé reprend le responsable existant de `dunning.ownerId` ; une relance `suivi15` / `suivi30` / `fincontrat` reprend le commercial vendeur (`S.clients[id].seller` résolu par `resolveSeller`) s'il est membre actif du club.

2. Verrou : à l'ouverture d'une fiche en mode session ou au tap sur Appeler, `claimedBy = ME.id` et `claimedUntil = now + lockMinutes`. Les autres voient « En cours par Julie, 12 min » et les boutons de contact sont désactivés, sauf « Prendre la main » (confirmation). Le verrou se libère à l'enregistrement du résultat ou à expiration.

3. Garde-fou client : si le client a un `S.touche`s de moins de 24 heures, tout contact affiche « Déjà contacté aujourd'hui par Marc à 10 h 12 : Messagerie. Continuer ? ». Valable entre types différents (un client impayé et en fin de contrat n'est appelé qu'une fois : afficher les deux motifs sur la même fiche).

4. Regroupement : la file affiche une ligne par client, pas par relance. Si un client a plusieurs relances ouvertes, la ligne liste les motifs et la feuille de résultat s'applique à toutes, avec une case par relance pour l'issue spécifique.

5. Absence : si un membre est archivé ou n'a pas ouvert l'appli depuis 3 jours, ses relances en retard remontent dans « Non attribuées » avec l'étiquette « Réattribuer ».

Règles : aucun emoji, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- Deux sessions ouvertes sur deux navigateurs : quand A appelle un client, B voit le verrou après synchronisation.
- Un client avec impayé et fin de contrat n'apparaît qu'une fois dans la file.
- « Répartir équitablement » sur 10 relances et 2 membres sans charge donne 5 et 5.

9. IMPAYÉS : DOUBLE COMPTAGE, PROMESSES ET CLÔTURE AUTOMATIQUE

Trois chemins créent le KPI « Impayés récupérés » pour le même encaissement : `loyAct` avec issue `paid` (l. 3020, id aléatoire), `dunPaidSave` (l. 4070, id `dn_`), puis l'import Incidents qui crée une saisie `r+hkey` quand le canal est équipe (l. 3350 et l. 3696). La prime peut donc être comptée deux ou trois fois. En plus, `loyAct paid` remet le solde à 0 sans renseigner `dunning`, donc le dossier disparaît d'Impayés sans trace « Récupéré ». La promesse de paiement est un simple statut sans montant ni date.

PROMPT CLAUDE

Contexte : `fitpulse.html`. KPI 'impayes' (l. 853). Saisies manuelles : `loyAct` (vers l. 3016) et `dunPaidSave` (vers l. 4066). Import : `parse 'incidents'` (vers l. 3336) crée `c.entry kpiId 'impayes'` pour canal 'equipe', écrit en entries `'r'+hkey` dans `rsmCommit` (vers l. 3696). `S.recov` garde chaque régularisation avec `clientNum`.

Objectif : un euro récupéré compte une seule fois, et la promesse devient un engagement suivi.

1. Un seul chemin manuel : l'issue Payé (feuille de résultat, Action Rétention ou Impayés) appelle une fonction unique `markPaid(clientId, amount, canal, userId)` qui écrit `dunning { status: 'recupere', recoveredAt, amount, canal, by }` et la saisie d'id `'dn_' + clientId + '_' + date`. Supprime la création d'entrée aléatoire dans `loyAct`.

2. Dédoublonnage à l'import : dans `rsmCommit`, avant d'écrire une entrée 'impayes' issue d'Incidents, cherche une saisie `'dn_' + clientId` (client trouvé par num) entre date - 7 jours et date + 7 jours avec un montant à 1 € près. Si elle existe : remplace-la par l'entrée importée (même `userId`, source 'import', champ `manualId` pour trace) au lieu d'ajouter. Affiche dans `rsmDoneCard` « 3 encaissements déjà saisis à la main, rapprochés ».

3. Promesse : champs `dunning.promiseAmount`, `dunning.promiseDate`, et `S.touches outcome 'promesse'`. La relance passe en 'attente' jusqu'au lendemain de `promiseDate`. Ce jour-là, si le solde a baissé d'au moins `promiseAmount` (import ou saisie), la relance est gagnée automatiquement ; sinon elle revient en tête de file avec l'étiquette « Promesse non tenue ».

4. Statut Impayés : remplace le select libre de statut (l. 4036) par l'état dérivé de la relance (À relancer, Relancé, Promesse au 15/10, Récupéré, Perdu) ; la date « Prochaine relance » est `nextAt` de la relance.

5. Historique : afficher `dunning.history` et les `S.touches` du client dans un tiroir au clic sur le nom (aujourd'hui l'historique est écrit mais jamais affiché).

Règles : aucun emoji, aucun tiret cadratin ni demi-cadratin. Retirer l'emoji du toast « Réglé : ajouté à

vos impayés récupérés ».

Critères d'acceptation :

- Marquer Payé 80 € à la main puis importer Incidents avec la même régularisation par l'équipe : une seule entrée impayés de 80 € pour le mois.
- Marquer Payé depuis Action Rétention fait apparaître le client dans Impayés > Récupérés.
- Une promesse non tenue remonte en tête de file le lendemain de la date.

10. FINS DE CONTRAT : LE RAPPROCHEMENT PAR NOM EST FRAGILE

La liste Abonnements n'a pas de numéro client (abonnements, l. 3387). `rsmCommit` rapproche par `tokensKey(nom)` et ne garde que la première fiche de chaque clé (l. 3688 : `clientIdx['t:'+...] || c`). Avec deux « Thomas Martin », la date de fin va sur le mauvais. Si personne ne correspond, une fiche fantôme sans numéro est créée (l. 3720), qui génère une relance renouvellement sans téléphone. Enfin `c.end` n'est jamais effacé quand l'abonné renouvelle.

PROMPT CLAUDE

Contexte : fitpulse.html. RSM_DEFS 'abonnements' (vers l. 3387) appelle `c.clientByName(contact, {end, offer, start})`. `rsmCommit` (vers l. 3719) rapproche via `clientIdx['t:'+tokensKey(nom)]`.

Objectif : rapprocher les fins de contrat sans erreur silencieuse.

1. Index multiple : `clientIdx['t:'+clé]` devient une liste de fiches. Résolution :
 - 1 candidat : rattachement direct ;
 - plusieurs : départager par offre (offer contient le libellé), puis par date de début (start égal à 3 jours près) ; si encore ambigu, ne rien écrire ;
 - 0 candidat : ne pas créer de fiche. Garder la ligne en attente.
2. File de doutes : `S.rsm.pendingMatches[club][key] = { name, end, offer, start, candidates: [clientId], at }`. Écran Imports > Resamania > « À rapprocher (n) » : pour chaque ligne, choisir la fiche parmi les candidats, chercher une autre fiche, ou « Ignorer ». Le choix est mémorisé (`S.rsm.nameLinks[club][tokensKey] = clientId`) et réutilisé aux imports suivants.
3. Normalisation du nom de contact : retirer civilité (M., Mme, Mlle), contenu entre parenthèses, e-mail entre chevrons, avant `tokensKey`.
4. Renouvellement détecté : si une vente d'abonnement (parse 'ventes') pour le même num a une date $\geq$ end - 45 jours, la relance fincontrat passe en 'gagne' avec result 'renouvele' et `c.end` est mis à jour.
5. Fiches fantômes existantes : script de nettoyage qui liste les `S.clients` sans num, sans phone, créées par import Abonnements, et propose fusion ou suppression (aucune suppression automatique).
6. Le message d'avertissement actuel est remplacé par un décompte : « 42 fins de contrat rattachées, 3 à vérifier, 1 sans correspondance ».

Règles : aucun emoji, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- Avec deux fiches « Thomas Martin » d'offres différentes, la ligne « MARTIN Thomas / Premium » va sur la fiche Premium.
- Avec deux fiches identiques, la ligne part dans « À rapprocher » et aucune date n'est écrite.
- Une ligne sans correspondance ne crée aucune fiche.
- Un choix manuel est réappliqué automatiquement au prochain import.

11. FICHE CLIENT ET TIMELINE COMPLÈTE

Il n'existe pas de fiche client. `loyHistory` (l. 3044) montre les 100 dernières actions du club, tous clients confondus. L'historique d'une résiliation est dans `resOpen` (l. 3945), celui d'un impayé n'est affiché nulle part. Avant d'appeler, le commercial doit voir en un écran qui est le client, ce qui a été dit, et par qui.

PROMPT CLAUDE

Contexte : `fitpulse.html` avec `S.clients`, `S.relances`, `S.touches`, `S.resiliations` (`clientId`), `S.recov` (`clientNum`), `S.entries`.

Objectif : une fiche client accessible depuis toute liste (nom cliquable) et depuis une recherche globale.

1. Route `#/client/<id>` (`PAGES.client`), tiroir sur mobile.
 2. En-tête : nom, numéro Resamania, offre, ancienneté (« adhérent depuis 8 mois »), date de fin, solde dû en rouge s'il existe, badges Sans mandat, Opposé aux SMS. Boutons de contact en un tap. Responsable actuel.
 3. Bloc « Relances ouvertes » : chaque relance avec kind, étape de cadence, prochaine action, bouton Noter.
 4. Timeline antichronologique fusionnant : `S.touches` (canal, issue, note, auteur), création et clôture des relances, actions de résiliation, régularisations `S.recov` (montant, canal), changements de dunning, ventes importées (contrat signé), modifications de coordonnées. Chaque ligne : date et heure, auteur, libellé court. Filtres : Tout, Appels, Messages, Paiements, Résiliation.
 5. Champ « Ajouter une note » en bas (`S.touches` channel 'note').
 6. Édition des coordonnées : téléphone, téléphone 2, e-mail, `optOutSms`, `optOutCall` ; chaque modification écrit une ligne de timeline.
 7. Recherche globale dans l'en-tête (nom, numéro, téléphone), résultats limités au club courant.
- Règles : aucun emoji, aucun tiret cadratin ni demi-cadratin.
- Critères d'acceptation :
- Sur un client avec impayé régularisé et résiliation sauvée, la timeline montre les deux, dans l'ordre.
 - Taper « 0612 » dans la recherche trouve le client au +33612345678.
 - Ajouter une note la fait apparaître immédiatement en haut de la timeline.

12. MESURE : JOIGNABILITÉ, CONVERSION PAR TYPE, EUROS RÉCUPÉRÉS

`loyPerf` (l. 3028) compte des actions et un « taux de succès » par personne. On ne sait pas combien d'appels aboutissent, quel type de relance convertit, ni combien d'euros les relances ont fait rentrer. La page Résiliations calcule déjà un délai de prise en charge (l. 3887) : il faut généraliser.

PROMPT CLAUDE

Contexte : `fitpulse.html` avec `S.touches`, `S.relances`, `S.recov`, `TOUCH_OUTCOMES` (`reached`, `win`, `lose`). `monthNav()`, `barChart()`, `progressBar()` existent.

Objectif : un onglet « Performance » dans la page Relances, par mois, pour le club et par commercial.

Indicateurs (définitions exactes) :

- Tentatives d'appel : touches channel 'call' du mois.
- Taux de joignabilité : touches 'call' dont TOUCH_OUTCOMES[outcome].reached / touches 'call'.
- Messages envoyés : touches 'sms' + 'whatsapp' + 'email'.
- Taux de conversion par kind : relances closes 'gagne' / (closes 'gagne' + closes 'perdu') du mois, par kind.
- Délai de premier contact : moyenne en heures entre createdAt de la relance et son premier touch.
- Relances en retard : status 'todo' ou 'attente' avec nextAt ou dueDate dépassé de plus de 2 jours.
- Euros récupérés par l'équipe : S.recov canal 'equipe' du mois (déjà calculé, réutiliser recovList).
- Euros influencés : S.recov tous canaux dont le clientNum correspond à un client ayant un touch impaye 'joint', 'promesse' ou 'envoyé' dans les 14 jours précédant la date de régularisation. Afficher séparément de « récupérés par l'équipe ».
- Taux de tenue des promesses : promesses dont le solde a baissé à la date / promesses arrivées à échéance.
- Sauvetages : relances resiliation gagnées / closes.
- Motifs de refus : répartition de lostReason par kind.

Affichage :

1. Rangée de tuiles : joignabilité, conversion globale, euros récupérés, euros influencés, délai de premier contact.
2. Tableau par kind : tentatives, joignabilité, conversion, délai.
3. Tableau par commercial : tentatives, joignabilité, conversions, euros, relances en retard. Le membre voit sa ligne et le total club ; le manager voit tout.
4. Meilleures heures d'appel : joignabilité par tranche horaire (9 h à 20 h) sur les 90 derniers jours, en barres.
5. Export CSV des touches du mois.

Règles : aucun emoji, aucun tiret cadratin ni demi-cadratin. Valeurs vides affichées « n.d. ».

Critères d'acceptation :

- 10 appels dont 4 joints donnent 40 % de joignabilité.
- Un paiement en ligne 5 jours après un SMS impayé compte en « influencés », pas en « récupérés par l'équipe ».
- Les chiffres changent avec monthNav.

13. PROSPECTS ET INVITÉS : AUJOURD'HUI DE SIMPLES COMPTEURS

Le parse prospects (l. 3397) ne crée que des saisies KPI prospects : aucune fiche personne, aucun téléphone, donc aucune relance possible. loyaltyTasks exclut d'ailleurs tout statut contenant « prospect » (l. 1460). Les invités (séances découverte, parrainages) n'existent pas du tout. C'est pourtant la relance qui convertit le plus vite en contrat.

PROMPT CLAUDE

Contexte : fitpulse.html. RSM_DEFS 'prospects' (vers l. 3397) produit c.entry kpiId 'prospects'. S.clients porte les adhérents. S.relances accepte kind 'prospect' et 'invite'.

Objectif : relancer les prospects et invités comme les adhérents.

1. Fiches : le parse 'prospects' appelle aussi c.client(cle, { name, phone, email, status: 'prospect', createdAt: date, seller, prospectValue }); clé = num si présent, sinon 'p:' + tokensKey(nom) + ':' + date. Lire les colonnes téléphone, e-mail, statut de prospection, valeur du prospect.

2. Saisie rapide : bouton « Nouveau prospect » ou « Nouvel invité » (accueil et barre mobile Saisir) : prénom, nom, téléphone obligatoire, source (Passage, Site, Parrainage, Réseaux, Invité d'un adhérent), parrain (recherche client) pour un invité, date de la séance découverte. Crée la fiche, la relance (ownerId = ME.id) et la saisie KPI prospects.

3. Génération : syncRelances crée kind 'prospect' (anchor createdAt) ou 'invite' (anchor date de séance), cadence du point des cadences.

4. Conversion : quand une vente d'abonnement importée (parse 'ventes') a le même téléphone, e-mail ou tokensKey de nom dans les 60 jours, la relance passe en 'gagne', result 'inscrit', status du client 'Client', et la fiche prospect est fusionnée avec la fiche adhérent (mergeClients).

5. Parrainage : la relance invite affiche le nom du parrain ; à la conversion, ajouter une ligne dans la timeline du parrain.

Règles : aucun emoji, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- Importer un fichier Prospects de 20 lignes crée 20 fiches et 20 relances, sans doubler le KPI au second import.
- Un invité saisi apparaît dans la file le lendemain de sa séance.
- Une vente importée pour ce prospect ferme sa relance en gagné et fusionne les fiches.

14. CONFORMITÉ, CONSENTEMENT ET PROPRIÉTÉ DE L'INTERFACE

Envoyer des SMS ou WhatsApp demande de respecter l'opposition du client, des horaires raisonnables et une trace. Rien de cela n'existe. Par ailleurs l'interface actuelle utilise des emojis comme icônes métier (LOYALTY_TYPES , l. 1431 à 1437 ; todoList , l. 4194 et 4195 ; toasts l. 3021 et 3936), contraire à la charte demandée.

PROMPT CLAUDE

Contexte : fitpulse.html. LOYALTY_TYPES (vers l. 1431) et todoList (vers l. 4191) utilisent des emojis comme icônes. ico(name) rend des icônes SVG.

Objectif : relances conformes et interface sans emoji.

1. Consentement : champs S.clients[id].optOutSms, optOutCall, optOutEmail, optOutAt, optOutBy. Une réponse « STOP » rapportée par le commercial se saisit par l'issue « Ne plus contacter » (canal précisé) dans la feuille de résultat. Les boutons concernés sont désactivés avec la mention « Opposé ». Les modèles marketing (anniversaire, prospect, suivi30) ajoutent une mention de désinscription (ex. « Répondez STOP pour ne plus recevoir ces messages ») ou la mention choisie par le club dans S.relanceCfg[club].smsFooter ; les messages de gestion (impayé, mandat) n'en ont pas besoin.

2. Horaires : quietFrom, quietTo, sundayOff appliqués à la planification et confirmés au clic (voir le point « un tap »). Pas de relance impayé plus de 2 fois par semaine (contrôle dans la cadence).

3. Trace : chaque S.touches garde by, at, channel, templateId et texte envoyé. Export CSV par client depuis sa fiche (droit d'accès).

4. Rétention des données : S.touches de plus de 3 ans supprimés au boot par un manager (avec confirmation

et compteur).

5. Emojis : remplacer les champs icon de LOYALTY_TYPES par des noms d'icônes SVG (phone, repeat, gift, euro, bank) rendus via ico(). Retirer les emojis des toasts et libellés (« Client sauvé », « Réglé : ajouté à vos impayés récupérés »). Ajouter les icônes manquantes au jeu ICONS si besoin.

6. Typographie : remplacer dans les textes d'interface les tirets cadratins (U+2014) et demi-cadratins (U+2013) par une virgule, deux-points ou un point, et le placeholder tiret cadratin des cellules vides par « n.d. » ou une cellule vide.

Règles : aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- Un client optOutSms n'a ni bouton SMS ni bouton WhatsApp actif, et aucune étape SMS de cadence ne lui est proposée.

- grep -P "[\x{1F300}-\x{1FAFF}\x{2600}-\x{27BF}]" sur les chaînes d'interface des pages Relances, Action Rétention, Impayés et Résiliations ne renvoie rien.

- grep -P "[\x{2013}\x{2014}]" sur ces mêmes pages ne renvoie rien dans les textes affichés.

ORDRE DE RÉALISATION CONSEILLÉ

1. Socle (point 1), puis téléphone (point 3) : sans eux, rien n'est appelable.
2. Un tap (4), feuille de résultat (5), file d'appels (2) : la valeur visible pour les équipes.
3. Cadences et correctifs J+15 / J+30 (6), impayés (9), anti-doublon (8).
4. Scripts et modèles (7), fiche client (11), fins de contrat (10).
5. Mesure (12), prospects (13), conformité (14).

4. ALERTES ET NOTIFICATIONS

Constat de départ : Fit Pulse ne prévient personne. Tout ce qui ressemble à une alerte n'existe que si l'appli est ouverte, au premier plan, sur le bon écran. Le seul réglage « Bilan hebdomadaire du club (lundi matin) » est un interrupteur sans moteur derrière. L'objectif de ce chantier : chaque événement qui rapporte ou fait perdre de l'argent déclenche un message court, envoyé au bon moment, à la bonne personne, avec un lien direct vers l'action.

1. CE QUI EXISTE AUJOURD'HUI : TOUT EST LOCAL ET PASSIF

- `toast()` (l. 1510) affiche un message 3,2 s dans `#toasts`. `detectLive()` (l. 994) ne réagit qu'aux nouvelles `entries` manuelles d'un collègue de moins de 60 s, et seulement en mode Firebase, onglet ouvert. Son texte contient un emoji et un tiret cadratin (l. 1003).
- Les compteurs sont des pastilles recalculées au rendu : `shell()` (l. 1637) pour Feed, Chat, Rétention, Résiliations, Impayés, et `tabBar()` (l. 4309) pour Relances. `managerCockpit()` (l. 4258) calcule « Sans saisie aujourd'hui » et les résiliations J-7, sans jamais prévenir le manager.
- Les bandeaux `alerts` de `PAGES.loyalty` (l. 2986 à 2991) et le contrôle impayés supérieurs à 3 fois le CA Pack de `impManual()` (l. 2969) restent visibles seulement sur la page. `celebrate()` (l. 4130) et `checkPalierCrossed()` (l. 4138) ne se déclenchent que chez la personne qui a saisi. Le reste de l'équipe ne voit jamais le palier franchi.
- `profAccount()` (l. 2622 à 2631) propose `liveBanner` et `digest`, mais `digest` n'est lu nulle part : la promesse affichée est fausse.
- Le mode local (`localBackend`, l. 916) rend tout envoi serveur impossible. Le push suppose le mode partagé Firebase (`firebaseBackend`, l. 927) et un hébergement HTTPS.

Idée : commencer par des corrections rapides côté client, sans serveur. Elles servent de socle au centre de notifications.

PROMPT CLAUDE

Contexte : fichier unique `fitpulse.html` (application Fit Pulse). Ne casse aucun comportement existant.

Objectif : corrections rapides des alertes côté client, sans serveur.

1. `detectLive` (vers la ligne 994) :
 - Retire l'emoji et le tiret cadratin du toast. Nouveau format exact :
"`{Prénom Nom}` : `#{valeur}` `{KPI}`, `{club}`"
 - Étends la détection aux nouveautés suivantes, pour les clubs de ME uniquement :
 - a) nouvelle résiliation (`S.resiliations`, clé absente de `before`) :
"Nouvelle résiliation reçue. Ouvrez Résiliations pour la prendre."
 - b) nouveau défi flash (`S.challenges`) :
"Défi flash lancé : `{titre}`, `{durée}` h. Ouvrez Défis."
 - c) palier collectif franchi (comparer `palierState(clubId, mk, k).reached` entre `before` et `after`) :
"Palier `{n}` atteint en `{KPI}` pour l'équipe."
Appelle `celebrate()` aussi chez les collègues, pas seulement chez l'auteur de la saisie.
 - N'affiche jamais de nom de client dans un toast.
2. `profAccount` (vers la ligne 2622) : la case "Bilan hebdomadaire du club (lundi matin)" ne fait rien. Remplace son libellé par "Bilan hebdomadaire (disponible avec les notifications)" et désactive-la tant que le module de notifications n'est pas branché (pref `'notifReady'` absente).
3. Ajoute une fonction `notify(kind, text, href)` qui centralise toast + futur centre in-app. Tous les nouveaux messages passent par elle.

Critères d'acceptation :

- grep sur le fichier : aucun caractère U+2014 ni U+2013 dans les chaînes passées à toast() ou notify().
- En mode Firebase, ouvrir deux navigateurs : créer une résiliation dans l'un fait apparaître le toast dans l'autre en moins de 3 s.
- Franchir un palier depuis le navigateur A déclenche celebrate() dans le navigateur B.
- Mode local : aucun changement visible hors du libellé du réglage.

2. ARCHITECTURE CIBLE : FIREBASE DE BOUT EN BOUT, SANS SERVEUR À GÉRER

L'appli parle déjà à Firebase Realtime Database sous `/pulse` (`attach()`, l. 941) et identifie l'utilisateur par son e-mail (`afterFirebaseLogin()`, l. 1930). La voie la plus courte : Firebase Hosting (HTTPS obligatoire pour le service worker), Cloud Functions v2 (déclencheurs RTDB pour l'instantané, `onSchedule` pour l'heure fixe, fuseau Europe/Paris), FCM pour le Web Push, extension « Trigger Email » ou Brevo pour l'e-mail. Le plan Blaze est requis mais le volume d'un club tient dans la gratuité.

Plan B sans Blaze : un Cloudflare Worker avec Cron Triggers lit la base en REST (compte de service), envoie le Web Push en VAPID et l'e-mail via Brevo. On perd l'instantané (minimum 1 minute de cron) mais le coût reste nul. Google Apps Script convient seulement pour l'e-mail du soir, pas pour le push.

Point critique : le client écoute `/pulse` en entier (`root.on('value')`, l. 945). Toute écriture sous `/pulse` renvoie l'arbre complet à chaque appareil et relance `normalizeState` puis le rendu. Les journaux, files d'attente et jetons doivent donc vivre hors de `/pulse`.

PROMPT CLAUDE

Contexte : Fit Pulse est un fichier unique `fitpulse.html`. Données dans Firebase RTDB sous `/pulse` (`firebaseBackend` vers la ligne 927). Le client écoute `/pulse` en entier : n'écris RIEN de nouveau sous `/pulse` sauf mention contraire.

Objectif : poser le socle serveur des notifications.

Fichiers à créer :

- `firebase.json` (`hosting public = "public"`, `functions = "functions"`, `database rules = "database.rules.json"`)
- `public/index.html` (copie de `fitpulse.html`), `public/sw.js` (vide pour l'instant), `public/manifest.webmanifest` (vide)
- `functions/package.json` (Node 20, `firebase-functions v5+`, `firebase-admin v12+`)
- `functions/src/index.ts` : exporte les fonctions, région `eu-west-1`, `setGlobalOptions({ region: 'eu-west-1', maxInstances: 3 })`
- `functions/src/time.ts` : `helpers todayParis(), nowParis(), isoParis(date), hhmmParis(), dayOfWeekParis()`.
Toutes les dates métier (`today()`, `dateOf()` du client, ligne 775) sont en heure locale : le serveur DOIT calculer en Europe/Paris.
- `functions/src/state.ts` : `loadClub(clubId)` qui lit seulement les sous-arbres utiles (`/pulse/users`, `/pulse/clubs`, `/pulse/clients` filtré par `clubId`, `/pulse/resiliations`, `/pulse/entries` du mois, `/pulse/targets/{mois}`, `/pulse/paliers/{club}`).
Porte côté serveur, à l'identique, les fonctions pures du client : `sumRange`, `rangeOf`, `elapsed`, `statusOf`, `palierState`, `loyaltyTasks`, `dunOf`, `dunDue`, `resOpen`, `daysTo`, `clubMembers`. Copie-les dans `functions/src/core.ts` avec un

commentaire

"copie de fitpulse.html ligne N" au-dessus de chacune.

Nouveaux chemins RTDB hors /pulse :

- /notif/tokens/{authUid}/{tokenId} : { token, userId, ua, createdAt, lastSeen }
- /notif/inbox/{userId}/{notifId} : { ruleId, title, body, url, at, readAt, count, groupKey }
- /notif/log/{dedupKey} : { at, userId, ruleId, channels }
- /notif/queue/{id} : { userId, payload, notBefore } (messages retenus par les heures calmes)
- /notif/prefs/{userId} : préférences (voir point 9)
- /notif/settings/{clubId} : réglages manager (seuils, heures, règles actives)
- /notif/private/{userId} : { phone } (SMS optionnel, jamais sous /pulse)

Critères d'acceptation :

- firebase deploy --only hosting,functions,database réussit sur un projet vierge.
- Une fonction de test "ping" (callable) renvoie todayParis() correct à 23 h 30 et à 00 h 30 heure de Paris.
- Les tests unitaires (vitest) de core.ts donnent les mêmes résultats que les fonctions du client sur l'état de démonstration (demoState, ligne 1009, exporté en JSON fixture).
- Aucune écriture serveur sous /pulse.

3. PWA INSTALLABLE : MANIFEST, SERVICE WORKER, ICÔNE SUR L'ÉCRAN D'ACCUEIL

Le fichier n'a ni manifest, ni `theme-color`, ni service worker (l. 1, seul un `meta viewport`). Sans installation, iOS refuse le Web Push (iOS 16.4 et plus, appli ajoutée à l'écran d'accueil uniquement). Le commercial ouvre déjà Fit Pulse sur son téléphone (commentaire l. 4092, `tabBar()` l. 4307) : l'installer en plein écran est cohérent.

Le service worker doit rester minimal : afficher le push, ouvrir la bonne route au clic (`#!/relances`, `#!/resiliations`, `#!/impayes`, `#!/loyalty`, `#!/challenges`), mettre à jour le badge de l'icône. Pas de cache agressif de `index.html`, sinon les mises à jour ne passent plus.

PROMPT CLAUDE

Contexte : fitpulse.html devient public/index.html servi par Firebase Hosting.

Fichiers à créer :

1. public/manifest.webmanifest

```
{ "name": "Fit Pulse", "short_name": "Fit Pulse", "start_url": "/#/home", "scope": "/",  
  "display": "standalone", "background_color": "#141413", "theme_color": "#FFD200", "lang": "fr",  
  "icons": [192, 512 en PNG, plus une 512 "purpose": "maskable"] }
```

Génère les PNG à partir de `window.PARKPULSE_ASSETS.icon` si présent, sinon un carré jaune #FFD200 avec "FP".

2. Dans `<head>` : `<link rel="manifest">`, `<meta name="theme-color" content="#FFD200">`,
`<link rel="apple-touch-icon" href="/icons/180.png">`, `<meta name="apple-mobile-web-app-capable"`
`content="yes">`.

3. public/sw.js :

- importScripts firebase-app-compat et firebase-messaging-compat 10.12.2 (même version que loadSdk, ligne 930).
- messaging.onBackgroundMessage : lit uniquement payload.data { title, body, url, tag, badge }.
self.registration.showNotification(title, { body, tag, renotify: false, icon, badge: '/icons/badge-72.png', data: { url } })).
- Si 'badge' est fourni : navigator.setAppBadge(Number(badge)) quand disponible.
- notificationclick : focus d'un client existant puis client.navigate(url), sinon clients.openWindow(url).
- Envoie aussi postMessage({ type: 'notif-click', id }) pour marquer lu dans l'inbox.
- Cache : stratégie network-first pour index.html, cache-first pour /icons et les polices. Version de cache dans une constante.

4. Dans le script de l'appli : enregistrement du SW seulement si location.protocol === 'https:' et backend.mode === 'firebase'.

5. Bouton "Installer Fit Pulse" dans Mon profil (profAccount, ligne 2622) :

- Android/desktop : capture beforeinstallprompt, bouton visible, prompt() au clic.
- iOS Safari non installé (navigator.standalone false) : affiche le texte exact "Pour recevoir les alertes sur iPhone : touchez Partager, puis Sur l'écran d'accueil, puis ouvrez Fit Pulse depuis l'icône."

Critères d'acceptation :

- Lighthouse : critère "Installable" vert.
- Android Chrome : l'appli s'installe, s'ouvre sans barre d'adresse sur #/home.
- iPhone iOS 17 : après ajout à l'écran d'accueil, Notification.permission est demandable depuis un bouton.
- Une nouvelle version de index.html déployée est servie au deuxième chargement au plus tard.
- Fichier local ouvert en file:// : aucun appel à serviceWorker.register, aucune erreur console.

4. ABONNEMENT PUSH, JETONS ET IDENTITÉ : LE BON TÉLÉPHONE POUR LA BONNE PERSONNE

Piège repéré : les deux comptes de départ partagent la même adresse (PARKPULSE_ACCOUNTS , l. 731 et 732) et afterFirebaseLogin() prend le premier utilisateur trouvé par e-mail (l. 1933). Le jeton doit donc être rattaché à ME.id au moment de l'abonnement, pas déduit de l'e-mail côté serveur.

La permission se demande après un geste utilisateur et après une explication, jamais au chargement. Un refus sur iOS est quasi définitif. Un jeton qui renvoie messaging/registration-token-not-registered est supprimé à l'envoi.

PROMPT CLAUDE

Contexte : fitpulse.html en mode Firebase, sw.js déjà en place (firebase-messaging-compat). Les chemins /notif/* existent hors de /pulse.

À faire côté client :

1. Charge firebase-messaging-compat dans loadSdk (ligne 930) quand 'serviceWorker' in navigator.
2. Carte "Notifications" de profAccount (ligne 2628), remplace par :
 - Statut : "Activées sur cet appareil" / "Désactivées" / "Bloquées dans le navigateur".
 - Bouton "Activer les alertes sur cet appareil". Au clic seulement : Notification.requestPermission(), puis messaging.getToken({ vapidKey: window.PARKPULSE_VAPID, serviceWorkerRegistration }).
 - Écris /notif/tokens/{auth.uid}/{sha256(token).slice(0,16)} = { token, userId: ME.id, ua: navigator.userAgent.slice(0,120), createdAt, lastSeen }.
 - À chaque ouverture : met à jour lastSeen ; si le jeton a changé, remplace l'entrée.

- `logout()` (ligne 1743) : supprime le jeton de cet appareil avant `signOut`.

3. Bandeau doux sur l'accueil (`PAGES.home`, ligne 4211), une seule fois par appareil, si `permission === 'default'` :

"Recevez vos relances du jour sur ce téléphone." Boutons "Activer" et "Plus tard" (pref locale, 14 jours).

À faire côté serveur (`functions/src/send.ts`) :

- `sendPush(userId, { title, body, url, tag, badge })` : lit tous les tokens dont `userId` correspond (requête sur `/notif/tokens`, index sur `userId`), envoie avec `admin.messaging().sendEachForMulticast` en message "data only" + `webpush.headers { Urgency, TTL }`.
TTL : 4 h pour une relance, 12 h pour un résumé, 1 h pour un défi flash.

- Supprime les jetons en erreur `not-registered` ou `invalid-argument`.

- Purge planifiée : jetons avec `lastSeen` de plus de 60 jours.

Règles `database.rules.json` pour `/notif` :

- `tokens/$uid` : `.read` et `.write` si `auth.uid === $uid`.

- `inbox/$userId` : lecture si l'utilisateur authentifié a cet id (via `/pulse/users/$userId/email === auth.token.email`),
écriture limitée au champ `readAt`.

- `log`, `queue`, `settings` (écriture), `private` : aucun accès client, sauf `settings` en lecture pour l'équipe et écriture pour un manager.

Critères d'acceptation :

- Le compte créateur et le compte manager de la même adresse reçoivent chacun seulement les alertes de leur rôle,
selon le compte choisi au login.

- Deux appareils pour un même utilisateur : un push arrive sur les deux, une seule fois chacun.

- Révoquer la permission dans le navigateur, puis envoyer : le jeton est supprimé de la base au premier échec.

- Aucun appel à `requestPermission` sans clic.

5. LE MOTEUR DE RÈGLES : DÉCLENCHEUR, CIBLE, CANAL, FRÉQUENCE, HEURES CALMES, REGROUPEMENT

Chaque alerte est une règle déclarative, pas du code dispersé. Le moteur reçoit des « candidats » (utilisateur, règle, entité, compteur), applique dans l'ordre : préférences, déduplication, regroupement, heures calmes, plafond quotidien, puis envoie sur les canaux autorisés et écrit l'inbox. C'est la même logique pour un déclencheur base de données et pour un cron.

Valeurs proposées : heures calmes 20 h 30 à 8 h 00 et dimanche entier. Plafond de 6 push par jour et par personne, hors priorité « urgente ». Fenêtre de regroupement de 10 minutes pour les événements, groupe fixe pour les créneaux planifiés. Les messages retenus partent à la fin des heures calmes, fusionnés.

PROMPT CLAUDE

Contexte : `functions/` du projet Fit Pulse (Cloud Functions v2, TypeScript). `core.ts` porte les calculs du client.

Crée `functions/src/engine.ts`.

Modèle d'une règle (functions/src/rules/*.ts) :

```
type Rule = {
  id: string; // 'res_new', 'dun_new', ...
  kind: 'event' | 'schedule';
  priority: 'urgent' | 'normal' | 'low';
  roles: Array<'membre' | 'manager' | 'createur'>; // rôles éligibles par défaut
  channels: Array<'push' | 'inbox' | 'email' | 'sms'>; // canaux par défaut, inbox toujours inclus
  dedup: (c: Candidate) => string; // clé unique, ex. `res_new|${userId}|${resId}`
  group?: { key: (c) => string; windowMin: number; text: (cs: Candidate[]) => Msg };
  maxPerDay?: number; // par utilisateur et par règle
  text: (c: Candidate) => Msg; // { title, body, url }
};
type Candidate = { ruleId, userId, clubId, entityId?, count?, data: Record<string, string|number> };
```

Pipeline dispatch(candidates: Candidate[]) :

1. Filtre rôle et statut : utilisateur actif (status !== 'archived' et !== 'pending'), club dans user.clubs.
2. Préférences (/notif/prefs/{userId}) : règle désactivée => stop ; canaux = intersection(règle.channels, prefs.channels).
Un membre ne peut pas désactiver une règle marquée locked par le manager dans /notif/settings/{clubId}.
3. Déduplication : transaction sur /notif/log/{dedupKey encodé}. Si existe => stop.
4. Regroupement : candidats de même group.key dans windowMin => un seul message via group.text, count = taille.
Implémentation : écrire dans /notif/queue avec notBefore = now + windowMin, puis une fonction onSchedule toutes les 5 minutes
vide la queue et fusionne par (userId, groupKey).
5. Heures calmes (prefs.quiet, défaut { from: '20:30', to: '08:00', sunday: true }) : sauf priority 'urgent', repousse notBefore à la fin de la plage.
6. Plafond : compte les push du jour (log). Au-delà de prefs.maxPush (défaut 6), canal push retiré, inbox conservé.
7. Envoi : inbox toujours ; push via sendPush ; email via sendEmail ; sms via sendSms si opt-in.
Badge = nombre d'items non lus dans l'inbox après écriture.
8. Journal : /notif/log/{key} = { at, userId, ruleId, channels }. Purge à 45 jours.

Contraintes de texte (vérifiées par un test) :

- title <= 40 caractères, body <= 120 caractères.
- Aucun emoji (regex \p{Extended_Pictographic}), aucun U+2014, aucun U+2013.
- Aucun nom de client, téléphone, montant individuel lié à un nom (voir point 11).

Critères d'acceptation (tests vitest sur engine.ts avec horloge simulée Europe/Paris) :

- 3 relances dues pour le même commercial à 10 h 00 => un seul push "3 relances à faire aujourd'hui".
- Événement à 22 h 10 priorité normale => push reçu à 8 h 00, pas avant. Priorité urgente => immédiat.
- Même dedupKey envoyée deux fois => un seul envoi.
- 7e push normal de la journée => inbox seulement.
- Règle coupée dans les préférences => aucun envoi, aucune ligne inbox.

6. CATALOGUE DES ALERTES : DÉCLENCHEURS, DESTINATAIRES, TEXTES EXACTS

Chaque ligne s'appuie sur une donnée qui existe déjà : `S.resiliations` et `resUrgent()` (l. 3869), `clients.balance` et `dunning` (l. 3987 à 3992), `loyaltyTasks()` et ses types `suivi`, `renouvellement`, `anniversaire` (l. 1454 à 1471), `statusOf()` (l. 1291), `palierState()` (l. 4106), `S.challenges` (l. 2342), `rsm.routine` écrit à chaque import (l. 3698), `ROUTINE_WEEK` (l. 3553). Aucun texte ne contient de nom de client : le nom n'apparaît qu'une fois l'appli ouverte.

Id	Déclencheur	Destinataire	Canal	Texte (titre / corps)	Moment
res_new	Création de <code>/pulse/resiliations/{id}</code> (saisie ou import Resamania)	Membres du club + manager ; responsable seul si <code>ownerId</code> déjà posé	push, inbox	Résiliation reçue / Un client veut partir. Effective dans {n} jours. Prenez le dossier.	Immédiat (groupé 10 min)
res_noowner	Dossier <code>nouvelle</code> sans <code>ownerId</code> depuis 24 h	Manager du club	push, inbox	Dossier sans responsable / {n} résiliation(s) attendent depuis 24 h. Attribuez-les.	10 h 15
res_j7	<code>resOpen</code> et <code>daysTo(effective)</code> = 7, 3 ou 1	Responsable, sinon manager	push, inbox (urgent à J-1)	Sauvetage à J-{n} / Dernier délai pour appeler. Notez l'appel dans Fit Pulse.	9 h 30
dun_new	<code>clients/{id}/balance</code> passe de 0 à plus de 0 (import)	Manager ; responsable si déjà attribué	push, inbox	Nouvel impayé / {n} nouveau(x) dossier(s), {total} € au total. À attribuer.	Après import (groupé)
dun_promise	<code>dunning.status</code> = <code>promesse</code> et <code>dunning.next</code> dépassé, solde toujours dû	Responsable du dossier	push, inbox	Promesse non tenue / Le paiement promis n'est pas arrivé. Relancez aujourd'hui.	9 h 45
dun_due	<code>dunDue()</code> vrai et <code>dunning.next</code> = aujourd'hui	Responsable	push, inbox (groupé avec <code>loy_*</code>)	Relances du jour / {n} relance(s) à faire aujourd'hui. Ouvrez Mes relances.	10 h 00
loy_bday	Anniversaire le jour même (<code>c.birth</code>)	Membres du club (tâche partagée)	push, inbox (groupé)	Anniversaires du jour / {n} adhérent(s) fêtent leur anniversaire. Un message suffit.	10 h 00
loy_end30	<code>c.end</code> = aujourd'hui + 30	Membres du club	inbox, résumé du matin	Fins de contrat à J-30 / {n} contrat(s) à renouveler. Proposez le réengagement.	Résumé 7 h 45
loy_end7	<code>c.end</code> = aujourd'hui + 7	Membres du club	push, inbox	Fins de contrat à J-7 / {n} adhérent(s) sans réengagement. Appelez avant la fin.	10 h 00
loy_j15	<code>c.start</code> = aujourd'hui - 15	Membres du club	push, inbox (groupé)	Appels J+15 / {n} nouvel(s) adhérent(s) à appeler. Vérifiez qu'ils reviennent.	10 h 00
loy_j30	<code>c.start</code> = aujourd'hui - 30	Membres du club	inbox, résumé	Appels J+30 / {n} adhérent(s) arrivent à un mois. Proposez un RDV coach.	Résumé 7 h 45
obj_late	<code>statusOf(pct, exp).key</code> = <code>verylate</code> pour un KPI à objectif, à partir du 7 du mois	Le commercial	push, inbox	{KPI} en retard / {réel} sur {objectif}. {x} par jour pour finir à l'objectif.	Mardi et jeudi 13 h 30
pal_near	Reste à faire du prochain palier inférieur ou égal à $\max(2, 5\% \text{ du seuil})$	Toute l'équipe du club	push, inbox	Palier {n} tout proche / Plus que {x} {KPI} pour la prime d'équipe.	Immédiat, 1 fois par palier et par mois
pal_hit	<code>palierState().reached</code> augmente	Toute l'équipe	push, inbox	Palier {n} atteint / {KPI} : {réel} pour l'équipe. {récompense}	Immédiat
record	Nouveau record personnel du mois ou du jour sur un KPI à points	Le commercial ; l'équipe si record club	push, inbox	Record battu / Votre meilleur mois en {KPI} : {valeur}.	Immédiat, 1 par KPI et par jour

| flash_new | Création de /pulse/challenges/{id} | Équipe du club sauf l'auteur | push (urgent si durée 6 h), inbox | Défi flash lancé / {titre}, {durée} h sur {KPI}. Classement en direct. | Immédiat |

| flash_end | challenge.end passé, une fois | Équipe du club | inbox, push au gagnant | Défi terminé / {Prénom} gagne {titre}. Voir le classement. | À la fin du défi |

| rsm_week | Lundi : rsm.routine[club] incomplet pour la semaine (ROUTINE_WEEK) | Manager du club | push, inbox, e-mail à 16 h | Imports du lundi / {k} sur {n} exports faits. Relances et impayés à jour après import. | Lundi 11 h 00 et 16 h 00 |

| rsm_month | Le 2 du mois, ROUTINE_MONTH incomplet | Manager | push, e-mail | Imports du mois / Exports de {mois} à déposer pour le récapitulatif. | 2 du mois 10 h 00 |

| am_digest | Planifié, jours ouvrés | Chaque utilisateur actif | push (1), inbox | Votre journée / {r} relance(s), {a} appel(s), objectif du jour : {x} {KPI}. | 7 h 45 |

| pm_digest | Planifié, jours ouvrés | Commercial : push ; manager : e-mail | push, e-mail | Bilan du jour / {n} saisie(s), rang #{rang}. Équipe : {contrats} contrat(s). | 19 h 30 |

| mgr_silent | Commercial actif sans saisie manuelle depuis 2 jours ouvrés | Manager | push, inbox | Commercial sans saisie / {Prénom} n'a rien saisi depuis {n} jours. | 16 h 00 |

| mgr_anomaly | Saisie supérieure à 5 fois la médiane de l'auteur, ou import qui fait chuter un KPI de plus de 30 %, ou impayés supérieurs à 3 fois le CA Pack | Manager | push, inbox | Chiffre à vérifier / Une saisie de {KPI} sort de l'ordinaire. Contrôlez-la. | Immédiat (groupé 30 min) |

| weekly | Lundi, si pref digest | Chaque utilisateur | e-mail | Semaine écoulée / Vos chiffres et ceux du club, comparés à la semaine d'avant. | Lundi 8 h 15 |

PROMPT CLAUDE

Contexte : fonctions/src/engine.ts existe (pipeline dispatch). Données sous /pulse identiques à fitpulse.html.

Crée les règles événementielles dans fonctions/src/rules/events.ts, avec des déclencheurs onValueCreated / onValueWritten (firebase-functions/v2/database, instance par défaut, région europe-west1) :

1. res_new : onValueCreated('/pulse/resiliations/{id}').
Cible : r.ownerId si présent, sinon clubMembers(r.clubId) + managers du club.
n = daysTo(r.effective) ; si null, corps "Date effective inconnue. Prenez le dossier."
Texte : titre "Résiliation reçue", corps "Un client veut partir. Effective dans {n} jours. Prenez le dossier."
url "/#/resiliations". group key `res\_new|{userId}`, fenêtre 10 min, texte groupé
"{count} résiliations reçues. Prenez les dossiers." Imports Resamania massifs : un seul message.
2. dun_new : onValueWritten('/pulse/clients/{id}/balance'), avant 0 ou absent, après > 0.
Cible : dunning.ownerId sinon managers du club. Groupé 30 min par club :
titre "Nouvel impayé", corps "{count} nouveau(x) dossier(s), {total} € au total. À attribuer." url
"/#/impayes".
3. flash_new : onValueCreated('/pulse/challenges/{id}'). Cible : clubMembers(clubId) sauf ch.by.
Priorité urgent si durée <= 12 h. Corps "{titre}, {durée} h sur {KPI}. Classement en direct." url
"/#/challenges".
4. pal_hit et pal_near : onValueCreated('/pulse/entries/{id}') pour les entrées du mois en cours.
Recalcule palierState(clubId, mois, kpiId). Persiste le dernier état vu dans /notif/state/{clubId}/{mois}/{kpiId}
= { reached, nearSent: [index] } pour ne pas renvoyer.
pal_near si reste <= max(2, ceil(0,05 x seuil)) et pas déjà envoyé pour ce palier.
5. record : même déclencheur. Record personnel = total du mois pour (userId, kpiId) > meilleur total des mois passés.

Stocke /notif/state/records/{userId}/{kpiId} = { best, mk }. Un envoi maximum par KPI et par jour.

6. mgr_anomaly : même déclencheur, source 'manual'. Médiane des 30 dernières saisies de l'auteur pour ce KPI ;

si value > 5 x médiane et value > seuil plancher (contrats 3, avis 10, euros 300) => manager.

Corps "Une saisie de {KPI} sort de l'ordinaire. Contrôlez-la." url "/#/members" (onglet historique).

Textes : reprends mot pour mot la colonne Texte du tableau du point 6 du dossier d'audit (chaînes en français, sans emoji, sans tiret cadratin ni demi-cadratin, aucun nom de client).

Critères d'acceptation (émulateur Firebase + tests) :

- Import de 12 résiliations en un batch => chaque membre reçoit 1 push "12 résiliations reçues. Prenez les dossiers."

- Une saisie qui fait passer l'équipe de 98 à 100 contrats (Pl = 100) => 1 push pal_hit à chaque membre, 0 doublon si on annule puis ressaisit.

- 99 contrats sur un seuil de 100 => pal_near une seule fois dans le mois.

- Saisie nutrition de 2 000 € par un commercial à médiane 40 € => 1 alerte manager.

- Aucun texte de payload ne contient la chaîne r.client ni un numéro de téléphone (test sur les payloads simulés).

7. LES ALERTES PLANIFIÉES : RELANCES, ANNIVERSAIRES, CONTRATS, OBJECTIFS, IMPORTS

Les tâches de rétention sont recalculées à la volée par `loyaltyTasks()` (l. 1454), avec une fenêtre large (anniversaire à 7 jours, fin de contrat à 30 jours, suivi de J+13 à J+45). Pour notifier, il faut des dates pivots précises : J pour l'anniversaire, J-30 et J-7 pour la fin de contrat, J+15 et J+30 pour l'arrivée. Ces tâches n'ont pas de responsable : on cible les membres du club et on regroupe tout dans un seul push « Appels du jour » à 10 h.

Objectif en retard : `statusOf()` (l. 1291) donne déjà `verylate` sous 75 % du rythme. On n'alerte que deux fois par semaine, avec le chiffre par jour à tenir (même calcul que `mission` dans `PAGES.home`, l. 4221), pour pousser sans culpabiliser. Import Resamania : `rsm.routine[club][exportId]` contient la date du dernier import (l. 3698), comparée au lundi de la semaine.

PROMPT CLAUDE

Contexte : functions/ Fit Pulse, engine.ts et core.ts en place. Fuseau Europe/Paris partout.

Crée functions/src/rules/schedule.ts avec `onSchedule({ schedule, timeZone: 'Europe/Paris' })` :

A. 07:45 lundi à samedi : `am_digest` pour chaque utilisateur actif, par club.

Calcul : `r` = nb de dossiers à son nom dus aujourd'hui (résiliations ouvertes + `dunDue`) ;

`a` = nb de tâches `loyaltyTasks` du club en state 'todo' dues aujourd'hui ;

objectif du jour = premier KPI en retard (même formule que `mission` dans `PAGES.home`, ligne 4221).

Titre "Votre journée". Corps "{r} relance(s), {a} appel(s), objectif du jour : {x} {KPI}."

Si tout vaut 0 : "Rien en retard. Objectif du jour : {x} {KPI}." url "/#/home".

Inclut dans l'inbox (pas en push) `loy_end30` et `loy_j30`.

B. 09:30 : `res_j7` pour `daysTo(effective)` dans {7, 3, 1}. Priorité urgent à 1.

Corps "Dernier délai pour appeler. Notez l'appel dans Fit Pulse." Titre "Sauvetage à J-{n}".

C. 09:45 : `dun_promise` pour `dunning.status === 'promesse'` et `dunning.next < aujourd'hui` et `balance > 0`.

D. 10:00 : groupe "calls" par utilisateur, fusionne `dun_due`, `loy_bday`, `loy_end7`, `loy_j15` en un seul push :

titre "Appels du jour", corps construit sur les compteurs non nuls, ex.

"3 relances, 2 anniversaires, 1 fin de contrat à J-7. Ouvrez Mes relances." url "/#/relances".

Une seule catégorie : utilise le texte individuel du tableau.

E. 10:15 : res_noowner (manager) : résiliations status 'nouvelle', sans ownerId, at < now - 24 h.

F. 13:30 mardi et jeudi, du 7 au 26 du mois : obj_late pour chaque KPI à objectif en verylate.
Corps "{réel} sur {objectif}. {x} par jour pour finir à l'objectif." Un seul KPI par push (le plus en retard).

G. 11:00 et 16:00 le lundi : rsm_week. ok = nb d'exports de ROUTINE_WEEK (ligne 3553) avec routine[id] >= lundi 00:00.
Si ok < total : manager. 16:00 ajoute l'e-mail.

H. 10:00 le 2 du mois : rsm_month sur ROUTINE_MONTH.

I. 16:00 lundi à vendredi : mgr_silent, commercial actif sans entrée manuelle depuis 2 jours ouverts (reprend la logique silent de managerCockpit, ligne 4262).

J. 19:30 lundi à samedi : pm_digest. Commercial : push "{n} saisie(s), rang #{rang}. Équipe : {contrats} contrat(s)."
Manager : e-mail (point 10).

K. Toutes les 5 minutes : vidage de /notif/queue (heures calmes et regroupements).

Jours fériés : liste française 2026 et 2027 en dur, traités comme un dimanche (silence sauf urgent).

Critères d'acceptation (tests avec horloge simulée) :

- Le 05/10/2026 (lundi) à 10:00, un commercial avec 2 relances et 1 anniversaire reçoit exactement 1 push "Appels du jour".
- Un adhérent né le 05/10 déclenche loy_bday le 05/10 seulement, pas le 29/09 (alors que loyaltyTasks le liste dès J-7).
- Aucun obj_late envoyé le 3 ni le 28 du mois.
- rsm_week ne part pas si les 7 exports sont faits depuis lundi 00:00.
- Le 25/12 aucun push normal.

8. ALERTES MANAGER : CE QUI DEMANDE UNE DÉCISION, PAS UN ÉCRAN DE PLUS

Le manager a déjà les calculs dans `managerCockpit()` (l. 4258) : résiliations J-7, dossiers sans responsable, impayés dus, équipe sans saisie. Il manque le déclenchement et une notion de seuil réglable par club. Le contrôle « impayés supérieurs à 3 fois le CA Pack » de `impManual()` (l. 2969) et le badge « sans code » de `memberRow()` (l. 2369) sont d'autres signaux utiles.

Règle d'or : une alerte manager doit proposer une action en un clic (attribuer, relancer, vérifier). Le manager règle les seuils et peut verrouiller une règle pour toute l'équipe (exemple : `res_new` non désactivable).

PROMPT CLAUDE

Contexte : `fitpulse.html` + `functions/. /notif/settings/{clubId}` existe.

Côté client : nouvel onglet "Alertes" dans `PAGES.members` (tableau T, ligne 2356), visible manager.
Formulaire qui écrit `/notif/settings/{clubId}` :

- `silentDays` (défaut 2), `silentHour` (défaut 16), `anomalyFactor` (défaut 5), `anomalyFloor` par KPI,
 - `importDay` (défaut lundi), `importHours` (défaut [11, 16]),
 - `locked`: { `res_new`: true, `res_j7`: true, `dun_promise`: true } (règles non désactivables par les membres),
 - `digestHourAm` (défaut 07:45), `digestHourPm` (défaut 19:30), `quiet` par défaut du club.
- Chaque ligne affiche le texte exact qui sera envoyé (aperçu), en lecture seule.

Côté serveur, règles manager supplémentaires :

1. `mgr_res_risk` (09:00) : nb de résiliations effectives dans les 7 jours > 3 => titre "Sauvetages en

```
retard",
  corps "{n} résiliations effectives sous 7 jours. Répartissez les appels." url "/#/resiliations".

2. mgr_dun_stock (lundi 09:00) : total dû en hausse de plus de 15 % sur 7 jours (snapshots
rsm.controls[club].du, ligne 4325) =>
  "Impayés en hausse" / "{total} € dus, +{x} % en une semaine. Attribuez les dossiers."

3. mgr_nocode (lundi 09:00) : membre actif sans codeHash en mode local, ou sans jeton push depuis 14
jours en mode Firebase =>
  "Équipe injoignable" / "{n} membre(s) ne reçoivent aucune alerte. Vérifiez leur accès."

4. mgr_anomaly import : après écriture de /pulse/rsm/controls/{club}, si un KPI du mois baisse de plus de
30 % par rapport au calcul précédent =>
  "Import à vérifier" / "Un import a fait baisser {KPI} de {x} %. Contrôlez le fichier."
Tous les messages manager portent sur des compteurs, jamais sur un nom de client.

Critères d'acceptation :

- Un membre ne voit pas l'onglet Alertes ; un manager le voit pour ses clubs seulement.

- Modifier silentDays à 3 décale la règle mgr_silent dès le lendemain.

- Une règle verrouillée apparaît grisée et cochée dans les préférences d'un membre.

- Le prénom du commercial apparaît dans mgr_silent (donnée interne équipe), aucune donnée adhérent dans
aucune alerte manager.
```

9. CENTRE DE NOTIFICATIONS IN-APP, BADGE ET PRÉFÉRENCES PAR RÔLE

Le push se perd ; l'inbox reste. Une cloche dans la `topbar` (l. 1656) avec compteur, un tiroir (le composant `openModal({ drawer: true })` existe, l. 1515) qui liste les messages, marque lu au clic et renvoie vers la route. Le badge de l'icône (`navigator.setAppBadge`) reprend le total non lu plus `myToDo()` (l. 4185), déjà utilisé par `tabBar()` (l. 4309).

Préférences : un socle par rôle (membre : ses dossiers, appels du jour, paliers, défis, résumé du matin ; manager : en plus les alertes manager, imports, bilan du soir par e-mail), puis un réglage individuel par règle et par canal, avec heures calmes personnelles. Elles remplacent la carte Notifications actuelle de `profAccount()` (l. 2628).

PROMPT CLAUDE

```
Contexte : fitpulse.html en mode Firebase. L'inbox est sous /notif/inbox/{userId} (hors /pulse).

1. Écoute séparée : après attach() (ligne 941), ouvre
  fb.database().ref('notif/inbox/' + ME.id).orderByChild('at').limitToLast(50).on('value', ...)
  et stocke dans une variable INBOX (pas dans S). Ne déclenche pas render() global : met à jour
  seulement la cloche.

2. Cloche dans la topbar (shell, ligne 1656), avant le bouton Saisir : icône + pastille du nombre non lu
(99+ au-delà).
  Clic : openModal({ title: 'Notifications', drawer: true }) avec la liste groupée Aujourd'hui / Hier /
  Plus ancien,
  chaque ligne : titre en gras, corps, heure relative (ago()), point si non lu. Clic sur une ligne :
  écrit readAt = Date.now(), ferme le tiroir, location.hash = url. Bouton "Tout marquer comme lu".

3. Badge appli : navigator.setAppBadge(nonLus + myToDo().res.length + myToDo().dun.filter(dunDue).length)
à chaque mise à jour, clearAppBadge si 0.

4. Préférences : remplace la carte "Notifications" de profAccount (ligne 2628) par :
  - Canaux : Push sur cet appareil (bouton d'activation du point 4), E-mail, SMS (si numéro renseigné).
  - Heures calmes : de [20:30] à [08:00], case "Silence le dimanche".
```

- Tableau des règles visibles pour le rôle (ROLES, ligne 1135), une ligne par règle avec cases Push / E-mail, libellé court et aperçu du texte. Règles verrouillées par le manager : cochées et grisées, mention "Imposé par le club".
- Plafond : "Au plus [6] alertes par jour".
- Conserver liveBanner (bandeau en direct) et brancher digest sur la règle weekly.

Écrit `/notif/prefs/{ME.id} = { channels, quiet, rules: { [ruleId]: { push, email } }, maxPush }`.

5. Valeurs par défaut par rôle dans un objet NOTIF_DEFAULTS = { membre: [...], manager: [...], createur: [...] }
partagé avec le serveur (même liste d'ids que le tableau du point 6).

Critères d'acceptation :

- Un push reçu apparaît dans la cloche en moins de 2 s si l'appli est ouverte, sans re-rendu de la page en cours.
- Cliquer une notification système ouvre la bonne route et la marque lue dans la cloche.
- Désactiver "Record battu" coupe push et e-mail de cette règle dès le message suivant.
- L'icône de l'appli installée affiche le badge sur Android Chrome et iOS 17 installé.
- Mode local : la cloche affiche les notify() locales du point 1, sans erreur.

10. E-MAIL ET SMS : LE RÉSUMÉ QUI ARRIVE MÊME SI LE TÉLÉPHONE EST MUET

L'e-mail porte ce qui se lit à froid : bilan du soir pour le manager, alerte imports du lundi à 16 h, bilan hebdomadaire (enfin branché sur la pref `digest`, l. 2630). Deux options : extension Firebase « Trigger Email » (écrit dans une collection, part par SMTP Brevo ou Gmail Workspace), ou Brevo en API directe depuis la fonction. Google Apps Script `MailApp` est un dépannage gratuit pour le seul e-mail du soir, appelé par un déclencheur horaire qui lit la base en REST.

SMS : réservé aux urgences opt-in (résiliation à J-1 sans appel noté, défi flash de 6 h), numéro du salarié stocké dans `/notif/private/{userId}`, jamais dans `/pulse` qui est lisible par toute l'équipe. Coût Brevo ou OVH d'environ 5 à 7 centimes par SMS : plafond mensuel par club.

PROMPT CLAUDE

Contexte : `functions/ Fit Pulse. engine.ts` appelle `sendEmail(userId, msg)` et `sendSms(userId, msg)`.

1. `functions/src/email.ts` :

- Fournisseur Brevo (API transactionnelle), clé dans Secret Manager `BREVO_KEY` (`defineSecret`).
- Expéditeur "Fit Pulse <alertes@{domaine}>", Reply-To = e-mail du manager du club.
- Gabarit HTML sobre, mobile d'abord, couleurs `#141413` et `#FFD200`, un bouton "Ouvrir Fit Pulse" vers l'URL de la route.
- Pied : "Vous recevez cet e-mail car les alertes Fit Pulse sont activées. Gérer mes alertes : `{url}/#/profile`".
- Lien de désinscription en un clic (`List-Unsubscribe` et `List-Unsubscribe-Post`) qui coupe le canal email dans `/notif/prefs`.

2. Gabarits :

- `pm_digest` manager, objet "Bilan du jour, {club}, {date}". Corps : contrats, avis, CA boutique, impayés récupérés, résiliations reçues et sauvées, membres sans saisie (prénoms), relances en retard (compteurs). Pas de nom d'adhérent.
- `rsm_week` 16:00, objet "Imports Resamania : {k} sur {n} faits". Liste des exports manquants (libellés de `ROUTINE_WEEK`).
- `weekly` lundi 08:15, objet "Votre semaine Fit Pulse". Chiffres de l'utilisateur et du club, flèches

comme `delta()` (ligne 4363), en texte.

3. `functions/src/sms.ts` (désactivé par défaut, `settings.smsEnabled` par club) :

- Envoi Brevo SMS, expéditeur "FITPULSE", 160 caractères max, sans accent risqué.
- Règles autorisées seulement : `res_j7` à J-1, `flash_new` de 6 h. Plafond `settings.smsMonthlyCap` (défaut 100).
- Texte `res_j7` : "Fit Pulse : sauvetage a J-1, dossier sans appel note. Ouvrez l'appli." (sans accents pour le GSM 7 bits).

4. Option sans Blaze : `apps-script/Code.gs`, déclencheur 19:30, lit `/pulse` en REST avec un jeton de service,

calcule le bilan du soir et envoie `MailApp.sendEmail` au manager. Documente les étapes dans un commentaire en tête du fichier.

Critères d'acceptation :

- Un manager reçoit un seul e-mail du soir par club, à 19:30 heure de Paris, lisible sur iPhone en mode sombre.
- Le lien de désinscription coupe l'e-mail sans connexion et sans toucher au push.
- Avec `smsEnabled` false, aucun appel à l'API SMS (vérifié par un mock).
- Aucun e-mail ne contient de téléphone, d'adresse ou de nom d'adhérent (test regex sur le HTML rendu à partir de `demoState`).

11. RGPD ET SÉCURITÉ : LE PUSH NE TRANSPORTE QUE DES COMPTEURS

Les payloads Web Push transitent par les serveurs de Google et d'Apple et s'affichent sur l'écran verrouillé. Règle stricte : ni nom d'adhérent, ni téléphone, ni montant rattaché à une personne, ni motif de résiliation (`RES_REASONS`, l. 3862, peut inclure « Santé », donnée sensible). Le détail n'apparaît qu'après ouverture, derrière l'authentification. Les toasts actuels de `resSaveIt` (`celebrate('CLIENT SAUVÉ', '{r.client} reste au club')`, l. 3938) restent acceptables à l'écran, mais ne doivent pas être recopiés tels quels dans un push.

Autre point : la configuration actuelle donne à toute adresse listée dans `/pulse/team` la lecture de tout l'arbre, téléphones des adhérents compris (`clients`). Les nouveaux chemins (`/notif/tokens`, `/notif/private`, `/notif/inbox`) ne doivent pas reproduire ce modèle. Base légale : intérêt légitime pour les alertes de travail des salariés, consentement explicite pour le push (permission navigateur) et pour le SMS. À inscrire au registre des traitements du club, avec une durée de conservation.

PROMPT CLAUDE

Contexte : `fitpulse.html` + `functions/` + `database.rules.json` du projet Fit Pulse.

1. Garde-fou de contenu : `functions/src/privacy.ts` exporte `assertSafe(msg, state)` appelé par engine avant tout envoi.

Rejette (et journalise sans le contenu) si title ou body contient :

- un nom présent dans `/pulse/clients/*/name` ou `/pulse/resiliations/*/client` (comparaison normalisée, `norm()` ligne du client),
- un motif `RES_REASONS`,
- un motif téléphone `/(\+33|0)[1-9](\s?\d{2}){4}/` ou un e-mail,
- un emoji, `U+2014` ou `U+2013`.

En cas de rejet : envoie la version générique de la règle ("Nouvelle alerte Fit Pulse. Ouvrez l'appli.").

2. Payload FCM "data only", avec `webpush.notification` absent, pour que le SW maîtrise l'affichage.

Option `prefs.lockscreenGeneric` (défaut false) : le SW affiche seulement "Fit Pulse" et "Nouvelle alerte" sur l'écran verrouillé.

3. Règles database.rules.json :

- /notif/tokens/\$uid : lecture et écriture par auth.uid === \$uid uniquement ; validation des champs (token string <= 4096, userId string).
 - /notif/private/\$userId : aucun accès client en lecture ; écriture par l'intéressé seulement (champ phone, format E.164).
 - /notif/inbox/\$userId : lecture par l'intéressé ; écriture client limitée à readAt (newData.child('readAt').isNumber()).
 - /notif/log et /notif/queue : aucun accès client.
- Ajoute des tests @firebase/rules-unit-testing pour chaque cas.

4. Conservation : purge planifiée quotidienne. inbox 30 jours, log 45 jours, queue 2 jours, tokens inactifs 60 jours.

5. Écran "Mes données d'alerte" dans Mon profil : liste des appareils abonnés (ua abrégé, lastSeen) avec bouton "Retirer",
numéro SMS avec bouton "Supprimer", bouton "Tout couper".

6. Texte d'information affiché avant la demande de permission, exact :

"Fit Pulse vous envoie des alertes de travail : relances, résiliations, objectifs. Aucune information sur les adhérents
n'apparaît dans la notification. Vous pouvez tout couper dans Mon profil."

Critères d'acceptation :

- Test : un message dont le body contient le nom d'un client de demoState est remplacé par la version générique.
- Test de règles : un membre ne lit pas les jetons d'un autre, ne lit aucun /notif/private, ne peut pas écrire title dans son inbox.
- Après 31 jours simulés, l'inbox ne contient plus les messages anciens.
- "Retirer" un appareil supprime son jeton ; le push suivant n'y arrive plus.

12. MESURER QUE LES ALERTES RAPPORTENT, ET POUVOIR TOUT COUPER

Une alerte se juge à l'action qui suit : appel noté sur la résiliation (resLog , l. 3918), relance passée en « Relancé » ou « Récupéré » (dunPatch , l. 4047), tâche de rétention traitée (ACTIONS.loyAct , l. 3016), saisie après un rappel d'objectif. On relie chaque envoi à l'action dans les heures suivantes. Sans cette boucle, on augmente le volume jusqu'à ce que l'équipe coupe tout.

Un interrupteur général par club et une limite globale protègent d'un bug de règle qui enverrait 200 push en une minute.

PROMPT CLAUDE

Contexte : fitpulse.html + functions/ Fit Pulse, moteur de règles en place.

1. Traçage :

- Chaque envoi écrit /notif/stats/{clubId}/{YYYY-MM}/{ruleId} : { sent, opened, acted } (incréments en transaction).
- opened : le SW (notificationclick) et la cloche (clic) appellent une callable markOpened(notifId).
- acted : fonction déclenchée sur /pulse/resiliations/{id}/actions, /pulse/clients/{id}/dunning, /pulse/loyalty/{id}, /pulse/entries/{id} ; si une notification de la règle liée a été envoyée à cet utilisateur dans les 4 h précédentes (log), incrémente acted. Correspondances : res_new, res_j7 => actions ; dun_* => dunning ; loy_* => loyalty ;
obj_late, pal_near, flash_new => entries.

2. Page "Alertes" côté manager (onglet du point 8) : tableau par règle du mois : envoyées, ouvertes, suivies d'une action, taux d'action, et une ligne "Désactivations par les membres". Tri par taux d'action croissant.
3. Interrupteur : `/notif/settings/{clubId}/paused` (booléen). Si vrai, engine ne fait plus que l'inbox. Coupe-circuit global : plus de 50 push en 5 minutes pour un club => `paused = true` automatiquement et e-mail au créateur "Alertes suspendues : volume anormal. Vérifiez les règles avant de réactiver."
4. Mode test : `settings.dryRun = true` => tout est calculé et écrit dans `/notif/dryrun/{date}` sans rien envoyer.
Bouton "M'envoyer un exemple" pour chaque règle dans l'onglet Alertes.

Critères d'acceptation :

- Une résiliation notifiée puis prise en charge 20 minutes après compte 1 sent, 1 acted pour `res_new`.
- Un import qui génère 60 candidats push en 2 minutes déclenche le coupe-circuit et l'e-mail au créateur.
- En `dryRun`, aucun appel FCM, Brevo ou SMS (mocks), et `/notif/dryrun` contient les messages attendus.
- Le tableau du manager affiche des taux cohérents sur l'état de démonstration rejoué sur 7 jours simulés.

ORDRE DE MISE EN OEUVRE CONSEILLÉ

1. Point 1 (corrections client, 1 jour), puis points 2, 3, 4 (socle Firebase, PWA, jetons).
2. Point 5 (moteur) et point 11 (garde-fous) avant le premier envoi réel.
3. Points 6 et 7 : commencer par `res_new`, `dun_promise`, appels du jour à 10 h, résumé du matin. Ce sont les plus proches du revenu.
4. Points 8, 9, 10 ensuite, point 12 dès la deuxième semaine pour couper ce qui ne déclenche aucune action.

5. FIABILITÉ DES CHIFFRES

Audit du fichier `fitpulse.html` (4 475 lignes). Les bugs marqués CONFIRMÉ ont été rejoués avec node en extrayant les fonctions du fichier (scripts `t1.js` à `t4.js` du dossier `scratchpad`). Pour la lecture XLSX, le test utilise SheetJS 0.18.5, la version chargée par l'application. Les points PROBABLE dépendent d'un contenu d'export ou d'une règle Firebase absente du dépôt.

1. CONFIRMÉ : LES MONTANTS « 1.234,56 » ET « 1,234 » SONT DIVISÉS PAR 1 000

`toNum` (l. 2732) et `rsmNum` (l. 3197) lisent « 1.234,56 » comme 1,234, « 1.234.567,89 » comme 1,234 et « 1,234 » comme 1,234. « (12,00) » et « EUR 12,50 » donnent 0. « 12,00- » donne +12 au lieu de -12. Dans l'import libre (lecture windows-1252), « ¤ 12,50 » donne 0 et la ligne est écartée sans prévenir. Conséquence : un solde ou une facture B2B de 1 234,56 € compte pour 1,23 €, ou disparaît.

PROMPT CLAUDE (CORRECTIF) :

Dans `fitpulse.html`, réécris `toNum` (vers la ligne 2732) et `rsmNum` (vers la ligne 3197) en une seule fonction `parseMontant(s)`, utilisée partout (import libre, Resamania, quickPad, manCell).

Règles :

1. Retirer espaces, espaces insécables (U+00A0, U+202F), « € », « ¤ », « EUR », « TTC ».
 2. Négatif si le texte commence par « - », finit par « - » ou est entre parenthèses.
 3. Si « , » et « . » sont présents tous les deux, le DERNIER des deux est le séparateur décimal, l'autre est un séparateur de milliers.
 4. Si seul « . » est présent et qu'il est suivi d'exactly 3 chiffres, avec au moins un groupe avant (ex. « 1.234 », « 12.345.678 »), c'est un séparateur de milliers. Pareil pour « , » seul suivi de 3 chiffres dans un fichier XLSX (format anglais). Sinon c'est la décimale.
 5. Texte illisible : renvoyer NaN, pas 0. L'appelant compte la ligne dans « écartées : montant illisible » et l'affiche dans le récapitulatif de l'import.
- Ajoute des tests (fichier `tests/parse.test.js`, `node --test`) :
- ```
"1 234,56" -> 1234.56 ; "1\u202f234,56" -> 1234.56 ; "1.234,56" -> 1234.56 ; "1.234.567,89" -> 1234567.89 ; "-12,00 €" -> -12 ; "12,00-" -> -12 ; "(12,00)" -> -12 ; "¤ 12,50" -> 12.5 ; "12,50 ¤" -> 12.5 ; "EUR 12,50" -> 12.5 ; "0,5" -> 0.5 ; "2 000" -> 2000 ; "abc" -> NaN ; "" -> NaN.
```
- Contexte XLSX : "1,234" -> 1234 ; contexte CSV français : "1,234" -> 1.234.

### 2. CONFIRMÉ : LES DATES XLSX SONT INVERSÉES JOUR/MOIS, ET LES DATES IMPOSSIBLES SONT ACCEPTÉES

`readXlsx` (l. 3151) lit les cellules avec `raw: false`. SheetJS rend alors le format date standard (n° 14) en « m/d/yy ». Le 05/10/2026 devient « 10/5/26 », que `rsmDate` lit comme le 10 mai 2026. Le 21/10/2026 devient « 2026-21-10 ». `toDate` et `rsmDate` ne valident rien : « 31/02/2026 » donne « 2026-02-31 », qui ne tombe dans aucun mois (comparaison de chaînes), donc la vente disparaît de tous les totaux. « 12/03/85 » donne une naissance en 2085. Une date Excel brute (45935) est rejetée par l'import libre.

#### PROMPT CLAUDE (CORRECTIF) :

Dans fitpulse.html :

```
1. readXlsx (vers la ligne 3151) : lire avec XLSX.read(buf, { type: 'array', cellDates: true }) et
sheet_to_json(..., { header: 1, raw: true, defval: '' }). Convertir chaque cellule Date en 'AAAA-MM-JJ'
avec les getters LOCAUX (getFullYear, getMonth, getDate), jamais toISOString. Convertir chaque nombre en
chaîne sans séparateur de milliers.

2. Fusionner toDate (l. 2733) et rsmDate (l. 3189) en parseDate(s, { pivotSiecle }) :
 - accepte AAAA-MM-JJ, JJ/MM/AAAA, JJ-MM-AAAA, JJ.MM.AAAA, JJ/MM/AA, numéro de série Excel (5 chiffres)
 ;
 - année sur 2 chiffres : 20AA si AA <= (année courante % 100) + 1, sinon 19AA (pour les naissances) ;
 - VALIDER : mois 1 à 12, jour 1 à daysIn(mois). Sinon renvoyer null ;
 - une chaîne ISO avec heure et fuseau (« 2026-10-05T23:30:00Z ») est convertie en date locale
 Europe/Paris.

3. Une date null compte dans « écartées : date illisible » avec l'exemple de la première valeur fautive.
Tests :
parseDate('05/10/26') -> '2026-10-05' ; parseDate('31/02/2026') -> null ; parseDate('10/25/26') -> null ;
parseDate('12/03/85') -> '1985-03-12' ; parseDate('45935') -> '2025-10-05' ; parseDate('2026-10-
05T23:30:00Z') -> '2026-10-06' (TZ=Europe/Paris).
Test XLSX : créer avec SheetJS une feuille avec la cellule {t:'n', v:46300, z:'m/d/yy'} puis la relire
avec readXlsx : la date doit valoir '2026-10-05', pas '2026-05-10'.
```

### 3. CONFIRMÉ : DOUBLE COMPTAGE ENTRE SAISIES MANUELLES ET IMPORTS

Aucun code ne rapproche les saisies manuelles des imports. Un conseiller clique « +1 Contrats » le 12/09, puis le manager importe « Vente d'abonnements » de septembre qui contient ce contrat : 2 contrats au lieu de 1. Même chose pour les impayés : « Réglé » dans Rétention (l. 3020) ou « Récupéré » dans Impayés (l. 4070) crée une saisie, puis l'import Incidents (canal équipe) en crée une seconde pour le même paiement. Le classement, les paliers et les points sont gonflés.

#### PROMPT CLAUDE (CORRECTIF) :

Dans fitpulse.html, ajoute une règle de priorité des sources par KPI et par mois : `S.kpis[k].source = 'manual' | 'import' | 'mixte'.`

1. Dans idx() (vers la ligne 1190) : pour un club, un KPI et un mois donnés, si au moins une saisie d'un import actif existe ET que la source du KPI est 'import', ignorer les saisies manuelles de ce KPI sur ce mois (sauf celles marquées `adjust: true`, qui sont des corrections de manager). Les garder visibles dans l'historique avec le libellé « remplacée par l'import ».

2. Pour les impayés : la saisie créée par Rétention (« Réglé ») et par Impayés (« Récupéré ») doit porter `clientNum`. À l'import Incidents, si une saisie manuelle impayés existe avec le même `clientNum` et un montant égal à 0,01 € près dans les 7 jours autour de la date de régularisation, ne pas créer l'entrée importée ; rattacher la régularisation à la saisie existante (champ `matchedKey`).

3. Afficher dans la revue d'import Resamania : « X saisies manuelles seront remplacées par l'import ».

Tests :

- 1 saisie manuelle contrats le 2026-09-12 (u2) + import ventes avec la même vente :  
`sumRange('niort','u2','contrats','2026-09-01','2026-09-30') === 1.`

- Rétention « Réglé » 59,90 € client 1234 le 2026-09-10 + import incidents régularisé 59,90 € client 1234 le 2026-09-09 canal équipe : total impayés septembre === 59.9.

- Saisie manuelle sur un KPI sans import ce mois-là : toujours comptée.

## 4. CONFIRMÉ : IMPORT LIBRE, L'ANNULATION PUIS LE RÉTABLISSEMENT DOUBLE LES CHIFFRES

La détection de doublons de `wizNext` (l. 2853) ne compare qu'aux imports ACTIFS. Scénario : import A « ventes septembre » (45 contrats) annulé par erreur, le manager réimporte le même fichier (import B, 45 contrats, aucun doublon détecté), puis clique « Rétablir » sur A (l. 2951). Résultat : 90 contrats. Autre cas : la clé de doublon contient `norm(r.join('|')).slice(0, 80)`. Un export identique avec une colonne en plus ou dans un autre ordre n'est pas reconnu. Le même mois importé par l'import libre puis par la liaison Resamania est compté deux fois, car les clés sont différentes.

### PROMPT CLAUDE (CORRECTIF) :

Dans `fitpulse.html` :

1. `ACTIONS.impRestore` (l. 2951) : avant de rétablir, recalculer les doublons entre les saisies de l'import à rétablir et celles des imports actifs (même clé métier). S'il y en a, afficher « N lignes sont déjà comptées par l'import X. Rétablir uniquement les M lignes nouvelles ? » et ne réactiver que celles-là (champ `e.suppressed = true` pour les autres).
  2. `wizNext` (l. 2853) : remplacer `rowKey` par une clé métier indépendante de l'ordre des colonnes : `userId|kpiId|date|valeur|valeurs normalisées des colonnes mappées (date, commercial, montant, catégorie)` et, si une colonne « numéro de facture / contrat / client » existe, ce numéro.
  3. Utiliser la même fonction de clé métier dans l'import libre et dans `rsmCommit` (l. 3681), pour que les deux chemins se détectent l'un l'autre : un numéro de facture + code produit + montant identiques = la même vente.
- Tests :
- importer A (3 lignes), annuler A, importer B (mêmes 3 lignes), rétablir A : total = 3, pas 6.
  - importer un CSV puis le même CSV avec une colonne « Statut » ajoutée en tête : 0 ligne nouvelle.
  - importer factures-avoirs via l'import libre puis via Resamania : total nutrition inchangé au second import.

## 5. CONFIRMÉ : RESAMANIA, ANNULER LE DERNIER IMPORT EFFACE LES VENTES D'UN IMPORT PRÉCÉDENT

Dans `rsmCommit` (l. 3681), l'identifiant d'une saisie dépend seulement de la clé de ligne. Réimporter une ligne déjà connue la RATTACHE au nouvel import. Simulation (`t4.js`) : import A (26 contrats du 05/09 au 30/09), puis import B (export J-30 suivant, 21 lignes dont 19 communes). Total 28, correct. On annule B : il reste 7 contrats au lieu de 26, alors que A est affiché « Actif ». À l'inverse, une vente passée en « annulée » dans Resamania est écartée au réimport, mais l'ancienne saisie n'est jamais retirée.

### PROMPT CLAUDE (CORRECTIF) :

Dans `fitpulse.html`, `ACTIONS.rsmCommit` (vers la ligne 3681) et `ACTIONS.impCancel / impRestore` :

1. Une saisie Resamania garde la liste des imports qui la contiennent : `e.importIds = { [impId]: true }`. Au réimport, ajouter le nouvel `impId` sans retirer les anciens.
2. Dans `idx()` et `challengeRanking` : une saisie compte si AU MOINS UN de ses imports est actif (et plus seulement `e.importId`).
3. Réconciliation des suppressions : pour un export de gestion qui couvre une période `[from, to]` complète (ventes, factures), toute saisie Resamania du même `defId`, du même club et dont la date est dans `[from, to]` mais dont la clé n'est plus présente dans le fichier passe à `e.removedBy = impId` (non comptée).

L'afficher dans la revue : « 3 ventes disparues de Resamania depuis le dernier import (annulées ?) ».

4. Pour les lignes écartées pour « vente annulée », si une saisie existe déjà avec la même clé, la marquer removedBy.

Tests :

- A = 26 ventes, B = 21 ventes dont 19 communes ; annuler B -> 26 ; annuler A puis B -> 0 ; rétablir A -> 26.

- Import 1 avec la vente sub:123:2026-09-10:pack, import 2 sur la même période avec Etat = « Annulé » : la vente ne compte plus.

## 6. CONFIRMÉ : ARRONDI FLOTTANT, OBJECTIF ATTEINT POUR LES POINTS MAIS PAS POUR LE STATUT

Trois ventes à 29,90 € plus une à 10,30 € donnent 99,9999999999999 pour un objectif de 100 €. tierOf (l. 1267) applique une tolérance de 1e-9 et donne 100 % des points. statusOf (l. 1291), reached, accomplishments (trophée « premier 100 % ») et paceMessage testent pct >= 1 sans tolérance. L'écran affiche donc « À l'heure, plus que 0 € avant l'étape des 100 % » et le trophée n'est pas attribué.

### PROMPT CLAUDE (CORRECTIF) :

Dans fitpulse.html, stocke et additionne les euros en CENTIMES entiers :

1. idx() (l. 1190) : pour un KPI en euros, ajouter Math.round(Number(e.value) \* 100) ; sumRange renvoie centimes / 100. Pour un KPI en quantité, entiers.

2. Créer const reached = p => p != null && p >= 1 - 1e-9 et l'utiliser dans statusOf, statsFor (reached), paceMessage, accomplishments, palierState, ranking des trophées KPI (score >= 0.5 - 1e-9).

3. histCell, wizNext, rsmCommit, quickAdd : arrondir les valeurs en euros à 2 décimales à l'écriture (déjà fait en partie), et ajouter un test qui le vérifie.

Tests :

- saisies nutrition 29.9, 29.9, 29.9, 10.3, objectif 100 : real === 100, pct === 1, status.key === 'done', tierOf === 1, accomplishments.first100 === true.

- 0.1 + 0.2 + 0.3 € pour un objectif de 0.6 : status 'done'.

- 1000 saisies de 0.01 € : sumRange === 10 exactement.

## 7. CONFIRMÉ : LE TOTAL CLUB NE CORRESPOND NI AU CLASSEMENT NI À L'OBJECTIF CLUB

La vue club additionne la clé club[\*] (l. 1197) : toutes les saisies du club, y compris celles des membres archivés et celles d'un utilisateur d'un autre club rattaché par un import. L'objectif club (clubMonthTarget, l. 1256) ne compte que les membres actifs et le classement aussi. Scénario : Julie (objectif 20 contrats) signe 8 contrats puis est archivée le 15/10. L'objectif club perd 20, le réalisé garde 8 : le pourcentage club monte. La somme des lignes du classement ne fait plus le total affiché. Le récapitulatif (l. 4398) inclut encore les archivés dans « par commercial ».

### PROMPT CLAUDE (CORRECTIF) :

Dans fitpulse.html, définis UNE règle de périmètre et applique-la partout :

« Total club du mois M = somme des saisies des membres qui étaient actifs dans ce club pendant M » ; l'objectif club suit le même périmètre.

1. Ajouter `u.activeFrom` / `u.archivedAt` (déjà présent) et une fonction `memberInClubDuring(u, clubId, from, to)`.
  2. `clubMonthTarget` (l. 1256) et `statsFor(clubId, null, r)` : utiliser ce périmètre ; le réalisé club devient la somme des `sumRange` par membre du périmètre, plus une ligne « Non attribué » affichée à part (saisies de `userId` hors périmètre).
  3. `ranking`, `monthFigures` (l. 4331), `recap` (l. 4387) et `palierState` utilisent la même fonction.
  4. Ajouter un contrôle : `somme(ranking.real) + nonAttribue === total club`, sinon bandeau d'alerte.
- Tests :
- u1 actif 4 contrats, u2 archivé le 2026-10-15 avec 8 contrats, objectifs 20/20 : octobre -> réalisé 12, objectif 40 ; novembre -> u2 hors périmètre.
  - saisie importée attribuée à un membre d'un autre club : comptée dans « Non attribué », pas dans un membre.
  - somme des lignes du classement + non attribué === total vue club pour chaque KPI.

## 8. CONFIRMÉ : LA CORRECTION D'HISTORIQUE ÉCRASE DES SAISIES ET TRICHE LES DÉFIS FLASH

`histCell` (l. 2487) supprime TOUTES les saisies manuelles du jour (y compris les « +1 » des membres, avec leur heure) et écrit une seule saisie d'ajustement datée de maintenant (`at: Date.now()`). Or `challengeRanking` (l. 1417) date les saisies manuelles par `at`. Scénario : défi flash « contrats » de 14 h à 18 h ; à 15 h le manager corrige le lundi précédent de 2 à 3 contrats ; le membre gagne 3 contrats dans le défi. Deux managers qui corrigent la même case en même temps créent chacun leur ajustement : la valeur est doublée.

### PROMPT CLAUDE (CORRECTIF) :

Dans `fitpulse.html`, `ACTIONS.histCell` (l. 2487) :

1. Ne plus supprimer les saisies existantes. Créer UNE saisie { `adjust: true`, `value: écart`, `date: d`, `at: midi du jour d (dateOf(d).getTime() + 12 h)`, `effectiveAt: Date.now()`, `by: ME.id`, `reason` } ; l'id est déterministe : `'adj_' + club + '_' + user + '_' + kpi + '_' + date`, pour qu'une seconde correction remplace la première au lieu de s'ajouter.
  2. Calcul de l'écart : `want - (somme des saisies du jour hors ajustement)`. Utiliser une transaction Firebase (`ref.transaction`) sur le chemin de l'ajustement pour éviter deux écritures concurrentes.
  3. `challengeRanking` (l. 1417) : ignorer les saisies `adjust: true` et toute saisie dont la date est antérieure au début du défi.
  4. Journaliser chaque correction dans `S.audit` (voir point 15).
- Tests :
- jour à 2 contrats (2 saisies manuelles) ; correction à 3 -> 3 saisies, total 3, les 2 saisies d'origine existent toujours.
  - correction à 3 puis à 4 -> une seule saisie `adj_...`, total 4.
  - défi flash aujourd'hui 14 h à 18 h, correction à 15 h d'un jour passé : valeur du défi inchangée.
  - deux corrections simultanées (simuler deux états S divergents) : total final = dernière valeur voulue, jamais la somme.

## 9. CONFIRMÉ : UN MEMBRE PEUT ANTIDATER SES SAISIES DANS UN MOIS CLOS (DROITS CÔTÉ NAVIGATEUR SEULEMENT)

Le formulaire « Saisir » (l. 2121) a `max = aujourd'hui` mais aucun minimum. Le 2 novembre, un membre peut saisir 3 contrats au 31 octobre : le classement d'octobre, les trophées et les points cumulés (`allTime`, recalculés à chaque affichage) changent après coup. La restriction « un membre ne saisit que pour lui » n'existe que dans l'interface (liste déroulante). PROBABLE : les règles Firebase de `/pulse` ne sont pas dans le dépôt (le `database.rules.json` racine ne contient pas `pulse`). Tout compte de l'équipe peut donc a priori écrire les saisies et les objectifs d'un autre membre depuis la console du navigateur.

### PROMPT CLAUDE (CORRECTIF) :

```
1. Dans fitpulse.html, ACTIONS.openSaisies (l. 2121) et quickAdd : un rôle 'membre' ne peut saisir que pour lui-même, et seulement pour aujourd'hui et la veille (min = addDays(today(), -1)). Un mois est « clos » le 3 du mois suivant (paramètre S.settings.closeDay) ; après, seule une saisie adjust d'un manager est possible, avec motif obligatoire.

2. Figer les résultats : à la clôture, écrire S.closed[club][mk] = { ranking, points, trophies, figé le, par } ; allTime, allTrophies et le récapitulatif lisent ce gel pour les mois clos.

3. Créer database.rules.pulse.json (à déployer) :
 - entries/$id : écriture si auth.token.email est dans team ET (newData.child('userId').val() === uid du membre lié à cet email OU rôle manager du club de newData.child('clubId')) ; pour un membre, newData.child('date').val() >= aujourd'hui moins 1 jour ;
 - targets, imports, rsm, users/$uid/role : écriture manager uniquement ;
 - .validate : value est un nombre, date au format AAAA-MM-JJ, kpiId existe.
Tests (émulateur Firebase, @firebase/rules-unit-testing) :

- membre u3 écrit une saisie userId u2 -> refusé ; userId u3 date du jour -> accepté ; date du mois dernier -> refusé ;

- membre modifie targets -> refusé ; manager -> accepté.
Test navigateur : après clôture d'octobre, une saisie antidatée au 31/10 ne change pas allTime ni les trophées d'octobre.
```

## 10. CONFIRMÉ : « IMPAYÉS RÉCUPÉRÉS » A TROIS VALEURS DIFFÉRENTES SELON L'ÉCRAN, ET LE « CA » INCLUT LES IMPAYÉS

Le tableau de bord et le classement lisent le KPI `impayes` (saisies manuelles et lignes Incidents du canal équipe). Le récapitulatif (`monthFigures`, l. 4331) affiche `recupere` à partir de `S.recov`, tous canaux confondus (prélèvement automatique compris). Il ignore les saisies manuelles. Exemple : 300 € « Régulé » saisis en Rétention et 1 200 € de représentations automatiques importées : dashboard 300 €, récap 1 200 €. Par ailleurs `caMonth` (l. 1490) additionne tous les KPI en euros, donc la courbe « CA » ajoute les impayés récupérés à la nutrition et aux accessoires.

### PROMPT CLAUDE (CORRECTIF) :

```
Dans fitpulse.html :

1. Définir dans un objet METRICS une définition unique par indicateur : { id, libellé, source, formule, périmètre }. Exemple : impayesEquipe = KPI impayes (saisies + import canal équipe, après dédoublonnage du point 3) ; impayesTousCanaux = S.recov hors 'annule'.

2. monthFigures (l. 4331) : le tuile « Impayés récupérés » affiche impayesEquipe en grand et « dont tous canaux : X » en sous-titre, avec la même valeur que le tableau de bord pour le même mois.

3. caMonth (l. 1490) : ne garder que les KPI marqués k.revenue === true (nutrition, accessoires) ;
```

renommer la courbe « CA boutique ».

4. Ajouter une info-bulle « comment ce chiffre est calculé » sur chaque tuile, générée depuis METRICS.  
Tests :

- septembre : saisie manuelle impayés 300 €, recov canal 'auto' 1 200 € : dashboard club impayés = 300 ; recap impayésEquipe = 300 ; recap tous canaux = 1 200.
- caMonth septembre avec nutrition 500, accessoires 200, impayés 300 -> 700.

## 11. PROBABLE : RATTACHEMENT VENDEUR, HOMONYMES, AUTRES CLUBS ET VENDEUR QUI CHANGE SELON LE FICHER

`resolveSeller` (l. 3226) cherche dans TOUS les utilisateurs (tous clubs, archivés compris) et prend le premier nom identique. Deux « Thomas Martin » (Niort et La Rochelle) : les ventes de Niort peuvent aller au membre de La Rochelle. Elles comptent alors dans le total de Niort mais n'apparaissent dans aucune ligne du classement. Les alias `S.rsm.aliases` sont globaux, pas par club. Enfin, « Factures & avoirs » attribue la ligne à l'« Auteur » et « Lignes de factures » au « Vendeur », avec la même clé de saisie : le dernier fichier importé décide du vendeur et les points passent d'un membre à l'autre.

### PROMPT CLAUDE (CORRECTIF) :

Dans `fitpulse.html`, `resolveSeller` (l. 3226) prend un paramètre `clubId` :

1. Recherche automatique limitée aux membres du club importé (`clubMembers(clubId, { all: true })`), archivés seulement si la date de la ligne est antérieure à `u.archivedAt`.

2. Si plusieurs membres correspondent (homonymes), renvoyer status 'ambiguous' avec la liste : la revue d'import demande de choisir.

3. Alias stockés par club : `S.rsm.aliases[clubId][cle]`.

4. Priorité de source pour le vendeur d'une même clé `fl:... : 'Vendeur' (lignes de factures) > 'commercial initial' > 'Auteur'`. Au réimport, ne jamais remplacer un vendeur de priorité supérieure par un vendeur de priorité inférieure ; signaler le conflit dans la revue.

Tests :

- deux utilisateurs « Thomas Martin » dans niort et larochelle, import niort : rattachement au membre niort ; deux homonymes dans niort : status 'ambiguous'.

- membre archivé le 2026-09-15 : une vente du 2026-09-10 lui est attribuée, une vente du 2026-09-20 ne l'est pas.

- importer factures-avoirs (Auteur = accueil) puis lignes-factures (Vendeur = u2) puis de nouveau factures-avoirs : la saisie reste attribuée à u2.

## 12. PROBABLE : MEMBRE RATTACHÉ À DEUX CLUBS, OBJECTIF COMPTÉ DEUX FOIS

Les objectifs sont stockés par mois et par membre (`S.targets[mk][uid]`), sans club. Un manager présent à Niort et à La Rochelle avec 20 contrats d'objectif pèse 20 dans l'objectif de chaque club, soit 40 au total réseau. `allTime` (l. 1344) additionne ses points club par club avec ce même objectif, donc son niveau peut doubler. Les badges « tous les objectifs atteints » sont aussi évalués deux fois.

### PROMPT CLAUDE (CORRECTIF) :

Dans fitpulse.html :

1. Passer les objectifs en `S.targets[mk][clubId][uid][kpi]` avec une migration : pour un membre à un seul club, recopier tel quel ; pour un membre à plusieurs clubs, mettre l'objectif sur son club principal (`u.clubs[0]`) et 0 ailleurs, puis afficher une alerte au manager.
  2. `monthTarget(mk, userId, kpiId, clubId)` ; mettre à jour `targetRange`, `clubMonthTarget`, `challengeRanking`, `memTargets`.
  3. `allTime (l. 1344)` : points = somme sur les clubs des points gagnés avec l'objectif de CE club ; plafonner au total des points disponibles du mois.
- Tests :
- ul dans niort et larochelle, objectif contrats 20 à niort, 0 à larochelle, 25 contrats à niort : objectif niort 20, objectif larochelle 0, points contrats = 1000 une seule fois.
  - migration : un état ancien { targets: { '2026-09': { ul: { contrats: 20 } } } } devient { '2026-09': { niort: { ul: { contrats: 20 } } } }.

## 13. PROBABLE : RYTHME ET PROJECTION, DIMANCHES, JOUR ENTAMÉ ET CHANGEMENT D'HEURE

`elapsed (l. 1239)` compte les jours calendaires, dimanches compris, et compte aujourd'hui comme un jour entier dès minuit. Le 1er du mois à 8 h, un conseiller sans vente est « Très en retard » s'il a un objectif. Si le club est fermé ou peu actif le dimanche, l'objectif attendu au lundi est trop haut. Le calcul par différence de dates locales donne 31,04 jours en octobre (passage à l'heure d'hiver le 25/10) au lieu de 31. L'écart est faible, mais le rythme est faux de quelques dixièmes de point.

### PROMPT CLAUDE (CORRECTIF) :

Dans fitpulse.html, réécris `elapsed(r)` (l. 1239) :

1. Compter les jours par itération de dates 'AAAA-MM-JJ' (`addDays`), jamais par différence de millisecondes.
  2. Poids par jour de la semaine paramétrable par club : `S.clubs[id].dayWeights = [dim, lun, ..., sam]`, défaut `[0.5, 1, 1, 1, 1, 1, 0.8]`. Jours fériés en option (liste `S.clubs[id].closedDays`).
  3. Aujourd'hui compte au prorata de l'heure d'ouverture :  $(\text{heure} - \text{ouverture}) / (\text{fermeture} - \text{ouverture})$ , borné à `[0, 1]`.
  4. Afficher « attendu à date : X » sous chaque carte KPI, et une projection fin de mois = réalisé / `elapsed`, masquée tant que `elapsed < 0.15`.
- Tests (TZ=Europe/Paris) :
- octobre 2026, poids tous à 1, le 31/10 à 23 h -> 1 ; le 15/10 à 0 h -> 14/31.
  - octobre 2026, total de jours = 31 exactement malgré le changement d'heure.
  - poids `[0,1,1,1,1,1,1]`, mois de septembre 2026, le lundi 7 à l'ouverture : 5 jours ouverts sur 26 passés (dimanche 6 exclu).

## 14. PROBABLE : INCIDENTS, CLÉS QUI SE CHEVAUCHENT, SOLDES REMIS À ZÉRO ET B2B GONFLÉ

Clé Incidents : `inc:numéroPaiement:dateIncident:numClient`. Si le numéro de paiement est vide et qu'un client a deux rejets le même jour (abonnement et pack), la seconde ligne écrase la première : un montant est perdu. Si un fichier Incidents contient quelques lignes « en cours », `rsmCommit` (l. 3726) considère la photo complète : tous les autres clients débiteurs passent à 0 et en « Récupéré » automatique. Enfin, toute facture hors boutique avec une « Société du client » compte un Contrat B2B (l. 3314) : la facture mensuelle d'un abonnement d'entreprise recompte un contrat chaque mois.

### PROMPT CLAUDE (CORRECTIF) :

Dans `fitpulse.html`, définition RSM 'incidents' (vers la ligne 3336) et `rsmCommit` (l. 3681) :

1. Clé incidents : ajouter le montant, le type d'incident et un compteur d'occurrence dans le fichier (comme `seen[base]` des factures). Si numéro de paiement vide, warning « clé faible ».
  2. Remise à zéro des soldes : uniquement si `r.balances.src === 'clients-incident'` (photo complète). Pour 'incidents', ne jamais remettre à zéro les clients absents. Ne jamais marquer 'recupere' automatiquement sans une régularisation (`recov`) correspondante : sinon statut '`a_verifier`'.
  3. B2B : ne compter qu'une fois par client et par contrat (clé `b2b:numClient:produit`), et uniquement si la facture est la première de ce contrat (pas de facture antérieure du même client et du même produit dans S). Afficher la liste des factures B2B retenues dans la revue.
- Tests :
- deux incidents même client, même date, numéro de paiement vide, 39,90 et 9,90 : deux `recov`, total 49,80.
  - import 'incidents' avec 2 lignes en cours alors que 10 clients ont un solde : les 8 autres soldes ne bougent pas.
  - deux factures mensuelles (septembre, octobre) de la même société pour le même abonnement : 1 contrat B2B au total.

## 15. PROPOSITION : PAGE « CONTRÔLE QUALITÉ DES DONNÉES » ET SUITE DE TESTS AUTOMATISÉS

Aujourd'hui, un seul contrôle croisé existe : contrats Fit Pulse contre « Performances commerciales » (l. 3765), affiché une fois après l'import puis perdu. Aucun chiffre ne remonte à sa ligne source et aucun journal ne dit qui a modifié quoi. La page proposée rend chaque écart visible et traçable. La suite de tests verrouille les 14 points ci-dessus.

### PROMPT CLAUDE (CORRECTIF) :

Dans `fitpulse.html`, crée une page manager `PAGES.quality` (« Contrôle qualité », route `#/quality`, onglet dans la barre manager) :

1. Rapprochement mensuel par club, un tableau par indicateur : valeur Fit Pulse | valeur Resamania de contrôle | écart | écart % | statut (vert  $\leq 1\%$  ou 1 unité, orange  $\leq 3\%$ , rouge au-delà). Sources : `contrats vs S.rsm.controls[club].perf[mk]` (par commercial et total) ; prospects transformés vs `tti` ; impayés en ligne vs web ; encaissements vs paiements ; entrants vs `evo.gained` ; total dû vs du.
2. Alertes automatiques : imports qui se recouvrent (mêmes clés dans deux imports actifs), saisies manuelles et importées du même jour sur le même KPI (point 3), listes à 2 000 lignes, dates hors mois, montants illisibles, vendeurs non rattachés, membres archivés avec des saisies, total club différent de la somme du classement (point 7).

3. Traçabilité : chaque montant du tableau de bord, du classement et du récap est cliquable et ouvre un tiroir « d'où vient ce chiffre » : liste des saisies (date, membre, source manuelle / import X / ajustement, fichier, clé de ligne rowKey, auteur, heure). Stocker à l'import la ligne source brute (r.join(';'), tronquée à 500 caractères) dans S.sources[rowKey].

4. Journal d'audit : toute écriture sur entries, targets, imports, rsm.aliases passe par db.batch qui ajoute [['audit', newId()], { at, by, chemin, avant, après, motif }]. Page filtrable par membre, KPI, mois. Lecture seule, écriture append-only dans les règles Firebase.

5. Export CSV du rapprochement et du journal.

Suite de tests : extraire la logique pure (parseMontant, parseDate, hkey, resolveSeller, idx/sumRange, statsFor, ranking, elapsed, RSM\_DEFS.parse, wizNext, rsmCommit sans le rendu) dans un bloc <script id="core"> que tests/run.mjs charge avec node:vm. Lancer avec node --test, TZ=Europe/Paris, et dans un second passage TZ=America/New\_York (les dates ne doivent pas changer).

Jeux de tests obligatoires :

- parsing : tous les cas des points 1 et 2 ;

- fixtures : les 17 modèles de fit-pulse-imports/1-exports-resamania remplis de 20 lignes connues ; vérifier le total par KPI et par vendeur attendu ;

- idempotence : importer deux fois le même lot -> mêmes totaux ; importer deux exports qui se recouvrent -> pas de doublon ; annuler / rétablir dans tous les ordres (points 4 et 5) ;

- cohérence : pour 3 mois de données de démo (demoState avec graine fixe), somme du classement + non attribué === total club === récap, pour chaque KPI ;

- flottants : 1 000 saisies aléatoires en centimes, sumRange === somme entière / 100 ;

- droits : règles Firebase testées à l'émulateur (point 9) ;

- concurrence : deux clients simulés qui corrigent la même case (point 8).

Ajouter un script npm « test » et un hook git pre-commit qui lance la suite.

## 6. PILOTAGE MANAGÉRIAL

Constat général. Fit Pulse est aujourd'hui un très bon outil de motivation du commercial (accueil, saisie rapide, paliers, classement, bilan façon stories). Il n'est pas encore un outil de pilotage pour le manager. Le manager voit des totaux de club (récap mensuel, paliers), des files de dossiers (résiliations, impayés) et un compteur "sans saisie aujourd'hui" (managerCockpit, l.4258). Pour voir un commercial, il doit passer par `PAGES.dashboard` en "Vue perso" et choisir les noms un par un dans une liste (l.1954). Rien ne garde la trace d'un entretien, d'un objectif de progrès ou d'un plan d'action. Le calcul du rythme a aussi un biais qui fausse les statuts affichés. Les points ci-dessous vont du plus impactant au moins impactant.

### 1. UNE VUE ÉQUIPE EN UN ÉCRAN : QUI EST EN RETARD, SUR QUEL KPI

Aucun écran ne croise les membres et les KPI avec le rythme. `memHistory` (l.2452) donne des valeurs brutes sans objectif ni rythme. `memRecaps` (l.2524) ne regarde que le mois terminé. `PAGES.recap` (l.4387) ne compare qu'au mois précédent, et surtout sur les contrats. Pourtant tout existe déjà dans `statsFor` (l.1269) : `pct`, `expected` et `statusOf`. Il manque une matrice membres x KPI, colorée par le ratio `pct / rythme`, avec le "reste à faire par jour" et une projection de fin de mois.

#### PROMPT CLAUDE

Contexte : application Fit Pulse, un seul fichier HTML (fitpulse.html). Les calculs sont dans `statsFor(clubId, userId, r)` (vers la ligne 1269), qui renvoie `rows[{k, real, target, pct, earned, status}], score` et `expected`. `healthOf(ratio)` (vers 1766) donne la couleur. `clubMembers(clubId)` donne les membres actifs. Les pages sont déclarées par `PAGES.xxx = { title, manager, render() }` et la navigation par le tableau NAV (vers 1606).

Objectif : créer une page manager "Pilotage équipe" (route `#/team`, `manager: true`), ajoutée dans NAV juste après "Récap du mois", avec l'icône 'users'.

Écran :

1. En-tête : nom du club, mois (`monthNav` sur `UI.teamMonth`, mois courant par défaut), jour X sur N, rythme attendu en %.
2. Bandeau club : pour chaque KPI obligatoire, réalisé / objectif club, % et projection de fin de mois.
3. Matrice : une ligne par membre actif, une colonne par KPI actif ayant un objectif, plus les colonnes "Score", "KPI en retard" et "Dernière saisie".
  - Cellule : `pct` en gros, réalisé / objectif en petit, fond coloré par `healthOf(pct / expected)`.Classes `h-good`, `h-watch`, `h-alert` existantes.
  - Infobulle (attribut `title`) : "Reste à faire : X par jour sur Y jours restants".
  - Colonne "KPI en retard" : nombre de KPI avec `pct / expected < 0,75`, suivi de leurs libellés.
  - Colonne "Dernière saisie" : date de la dernière entrée manuelle du membre dans ce club, en rouge si elle a plus de 2 jours.
4. Tri par menu : "Plus en retard d'abord" (par défaut, ratio `score / expected` croissant), "Score", "Nom".
5. Un clic sur le nom d'un membre ouvre sa fiche de coaching si la route `#/coach/<uid>` existe. Sinon il ouvre le tableau de bord en vue perso (`UI.dashView = 'perso'`, `UI.dashUser = uid`).

Formules exactes :

- `exp = statsFor(...).expected` (part de la période écoulée).
- `ratio = pct / exp` quand `exp > 0`, sinon `null`.

- `joursRestants = max(1, daysIn(mk) - jourDuMois + 1)`. Si une fonction `workdaysLeft(uid, mk)` existe, l'utiliser.
- `resteParJour = max(0, target - real) / joursRestants`. Arrondi supérieur pour unit 'qty', 2 décimales pour 'eur'.
- `projection = exp >= 0.1 ? real / exp : null`. Afficher "n.d." quand elle est nulle.
- Projection club d'un KPI = somme des réalisés / exp, comparée à `clubMonthTarget(mk, clubId, kpiId)`.

Contraintes : aucun nouveau stockage. Tout est calculé, et mémorisé via `memo()` avec la clé ``team|${clubId}|${mk}``. La page doit rester utilisable sur téléphone : tableau dans `.table-wrap` avec défilement horizontal, première colonne collante (`position: sticky; left: 0`). Aucun emoji décoratif, aucun tiret cadratin dans les textes d'interface.

Critères d'acceptation :

- Avec la démo (bouton "Découvrir avec des données de démonstration") et le profil Hugo Lefèvre (manager), `#/team` affiche les 4 membres actifs de Niort, avec une cellule par KPI ayant un objectif.
- Pour une cellule donnée, la couleur est identique à celle de la carte KPI du même membre dans Mes objectifs, vue perso.
- Le tri "Plus en retard d'abord" place en tête le membre dont `score / expected` est le plus faible.
- Un membre (rôle membre) qui tape `#/team` est renvoyé à l'accueil.
- Changer de mois avec `monthNav` recalcule tout. Un mois terminé donne `exp = 1` et la projection est égale au réalisé.

## 2. UN RYTHME JUSTE : POINTS EN MARCHES, JOURS OUVRÉS, ABSENCES

Le statut du tableau de bord compare des points en marches à un rythme linéaire. Dans `dashObjectives` (l.1977 à 1979), `pctPts = earned / max` utilise `tierOf` (marches de 25 %), puis `statusOf(pctPts, exp)`. Prenons un commercial à 49 % de tous ses KPI au jour 10 sur 31. Il est en avance (49 % contre 32 %), mais il affiche 25 % des points, donc "En léger retard" et des "pts de retard". La bague de l'accueil (l.4227) a le même défaut. Autre biais : `elapsed` (l.1239) compte des jours calendaires, avec le dimanche et la journée entière dès le matin. Enfin, l'écran Mes objectifs (tous les KPI, l.1950) ne montre pas le même score que le classement (KPI obligatoires seulement, l.1318).

### PROMPT CLAUDE

Contexte : Fit Pulse, fichier unique `fitpulse.html`. Trois biais faussent le rythme affiché.

- `dashObjectives` (vers 1976) calcule `pctPts = st.earned / st.max` avec des points en marches de 25 % (`tierOf`, `TIERS`), puis le compare au rythme linéaire avec `statusOf(pctPts, exp)`. Même chose pour `moodOf` (vers 1965) et la bague de `PAGES.home` (`myPct`, vers 4227).
- `elapsed(r)` (vers 1239) compte les jours calendaires, dimanche compris, et la journée en cours entière dès minuit.
- Le score de Mes objectifs inclut les KPI facultatifs, alors que `ranking()` (vers 1315) utilise `requiredOnly`.

Travail demandé :

- Ajouter dans `statsFor` un champ `progress = somme(min(pct, 1) x points) / somme(points)`, sur les KPI avec objectif et `points > 0`. Utiliser `progress` (continu) partout où un % est comparé au rythme : statut du hero, "pts de retard", `moodOf`, bague et `healthChip` de l'accueil, `dayRecap`. Les points en marches (`earned`) restent affichés comme "points gagnés", sans jamais être comparés au rythme.  
`lag (pts de retard) = round(max x exp - progress x max)`.
- Calendrier de travail :
  - Champ `club.openDays` : tableau des jours ouverts 0..6, [1,2,3,4,5,6] par défaut. Éditable par le

créateur dans le formulaire du club (clubForm).

- Absences : `S.absences[userId][date] = 'conge' | 'maladie' | 'formation'`. Ajouter absences: {} à `emptyState()` (vers 878) pour que `normalizeState` le crée.
- `workdays(uid, clubId, from, to)` = nombre de jours de [from, to] ouverts pour le club et sans absence du membre.
- `elapsedFor(uid, clubId, r) = (jours ouverts terminés avant aujourd'hui + fraction du jour) / workdays(uid, clubId, r.from, r.to)`. Fraction du jour = `clamp((heure - 9) / 12, 0, 1)` si aujourd'hui est ouvert, 0 sinon. Si le total vaut 0, renvoyer 1.
- Sans `userId` (vue club), utiliser les jours ouverts du club, sans absence.
- `statsFor` utilise `elapsedFor` à la place de `elapsed`. La clé memo doit dépendre de l'heure arrondie, pour que le rythme avance dans la journée.
- Exposer `workdaysLeft(uid, mk) = jours ouverts restants, aujourd'hui compris s'il est ouvert`. `paceMessage`, l'accueil (mission du jour) et `palierBlock` l'utilisent à la place de `daysIn - jour + 1`.

3. Écran de saisie des absences : dans Équipe & paliers, nouvel onglet "Présences". Grille membres x jours du mois. Un clic fait tourner la case : vide, Congé, Maladie, Formation. Le total de jours ouverts de chaque membre s'affiche en fin de ligne.

4. Mettre une seule définition du score : le score affiché dans Mes objectifs est celui des KPI obligatoires, comme le classement. Libellé : "Score classement (KPI obligatoires)". Ajouter en dessous, en petit, "Score tous KPI : X %".

Règles UI : aucun emoji décoratif, aucun tiret cadratin dans les textes d'interface.

Critères d'acceptation :

- Cas test : mois de 30 jours, aucune absence, jour 10 à 9 h, tous les KPI à 49 %. Le statut affiché est "En avance" et aucun "pts de retard" n'apparaît.
- Un membre avec 5 jours de congé sur le mois a un rythme attendu plus faible qu'un collègue sans absence, le même jour.
- Un dimanche (club fermé le dimanche), le rythme attendu est le même que le samedi soir.
- Mes objectifs et le Classement affichent le même score pour le même membre et le même mois.
- Les trophées et le classement ne changent pas de méthode : seul le rythme change.

### 3. LE BARÈME DE POINTS : PERTINENCE ET ÉQUITÉ

Le champ `points` de `DEFAULT_KPIS` (l.848) sert de poids. Sur les 5 KPI obligatoires (2 850 pts), cela donne : contrats 35 %, impayés 26 %, nutrition 17,5 %, accessoires 17,5 %, avis 3,5 %. Avec les objectifs de la démo (l.1033), un shake à 30 € vaut autant qu'un contrat (50 pts), et un impayé récupéré de 30 € vaut près de deux contrats. Or une bonne partie des impayés se régularise sans le commercial (canaux auto, client, tiers, l.1115). Le "Régulé" de la rétention crédite celui qui clique (`loyAct`, l.3020). Autres défauts : les petits objectifs (B2B 2, sauvetage 2, invités 3) font des sauts de 0 à 50 puis 100 %, et les marches de 25 % poussent à garder une vente pour le mois suivant. Les prospects valent 0 pt alors que c'est l'indicateur d'activité clé. Enfin, les poids ne sont affichés nulle part en %.

#### PROMPT CLAUDE

Contexte : Fit Pulse, `fitpulse.html`. `DEFAULT_KPIS` (vers 848) donne à chaque KPI un champ `points`, qui sert de poids dans le score (`statsFor`, vers 1269) et de valeur en marches de 25 % (`tierOf`). Le réglage se fait dans `clubSettings` (vers 3095). Les poids actuels surpondèrent les euros de boutique et d'impayés face aux contrats.

Travail demandé :

1. Afficher le poids réel : dans `clubSettings`, ajouter une colonne "Poids dans le score" = `points / somme des points des KPI actifs obligatoires`, en %. Ajouter une colonne "Valeur d'une unité" = `points / objectif moyen du mois courant (moyenne des S.targets[mk][uid][kpi] > 0)`, par exemple "1 contrat = 50 pts, 10 € de nutrition = 17 pts". Si la valeur d'une unité d'un KPI en euros dépasse 50 % de celle d'un

contrat pour moins de 50 €, afficher l'alerte "Ce KPI pèse plus qu'une vente pour un petit montant".

2. Proposer un barème recommandé, appliqué par un bouton "Appliquer le barème recommandé" avec confirmation (confirmDlg) : contrats 1000, impayés 400, nutrition 300, accessoires 200, avis 250, sauvetage 400, invites 300, b2b 300, prospects 150. Rendre sauvetage obligatoire (required: true). Ne pas modifier DEFAULT\_KPIS : écrire dans S.kpis.

3. Petits objectifs : ajouter un réglage KPI "Calcul continu" (k.linear, booléen). Valeur par défaut : vrai si l'objectif moyen du KPI est inférieur à 5. Dans statsFor, quand k.linear est vrai : earned = min(pct, 1) x points, au lieu de tierOf(pct) x points.

4. Impayés : ne compter dans le KPI impayés que les montants dont le canal est 'equipe' quand la donnée existe. Pour les entrées créées par loyAct avec o === 'paid', ajouter le champ needsCheck: true. Une case "Validé" apparaît dans l'historique des saisies (memHistory) pour le manager. Tant que la case n'est pas cochée, la saisie compte dans le réalisé mais elle est signalée par un point orange.

5. Texte d'aide (bloc de règles en haut de clubSettings) : expliquer en 4 phrases courtes la différence entre score (moyenne des % pondérée, plafond 150 %) et points (marches de 25 %, ou continu si "Calcul continu").

Règles UI : aucun emoji décoratif ajouté, aucun tiret cadratin dans les textes d'interface.

Critères d'acceptation :

- Avec les KPI par défaut, la colonne Poids affiche 35 % pour Contrats signés et 3,5 % pour Avis Google (arrondi à une décimale).
- Après "Appliquer le barème recommandé", Contrats pèse plus que nutrition + accessoires réunis, et la somme des poids obligatoires fait 100 %.
- Avec un objectif B2B de 2 et 1 contrat B2B réalisé, le KPI rapporte 50 % des points en continu comme en marches. Avec un objectif de 3 et 2 réalisés, il rapporte 67 % en continu, contre 50 % en marches.
- Un "Réglé" saisi depuis Action Rétention apparaît avec le marqueur "à valider" dans l'historique des saisies du manager.

## 4. LA FICHE DE COACHING INDIVIDUELLE

Le manager n'a aucune fiche commercial. `PAGES.profile` (I.2594) ne montre que l'utilisateur connecté. `PAGES.wrap` (I.2640) est un bilan "stories" orienté célébration, sans tendance ni comparaison à soi-même. Il n'existe aucun endroit pour les notes d'entretien, les objectifs SMART ou un plan d'action suivi. C'est le manque le plus visible pour "un gros retour managérial".

### PROMPT CLAUDE (1/2 : ÉCRAN ET CALCULS)

Contexte : Fit Pulse, fitpulse.html. Données de saisie via `sumRange(clubId, userId, kpiId, from, to)` et `statsFor(clubId, userId, rangeOf('month', mk))`. Membres : `clubMembers(clubId)`. Les pages manager déclarent `manager: true`. Le routeur passe les segments d'URL en args (`render(args)`).

Objectif : nouvelle page "Fiche coaching", route `#/coach/<uid>`, réservée aux managers. Un membre peut ouvrir sa propre fiche en lecture, sauf l'onglet Notes. Accès depuis : Équipe & paliers > Organigramme (bouton "Fiche" sur chaque `memberRow`), la page Pilotage équipe si elle existe, et le menu de sélection de Mes objectifs.

Contenu, en onglets :

A. "Tendances" :

- Courbe du score des 6 derniers mois (`lineChart` existant) et, en pointillé, la moyenne du club (moyenne des scores de `ranking(club, mois)`).
- Tableau KPI x 6 mois : `pct` de chaque mois, coloré par `healthOf(pct)`.
- Pour le mois courant, colonne "vs ma moyenne" : `écart = pct courant projeté - moyenne des pct des 3 mois terminés précédents`, en points de %. Le `pct` courant projeté vaut `pct / expected` quand `expected >= 0,2`, sinon on n'affiche rien.

B. "Forces et axes de progrès" :

- Indice relatif d'un KPI = pct moyen du membre sur les 3 derniers mois terminés / pct moyen de l'équipe active sur la même période. Le KPI est ignoré s'il n'a pas d'objectif sur au moins 2 des 3 mois.
- Forces = les 2 indices les plus hauts et  $\geq 1,1$ . Axes de progrès = les 2 plus bas et  $\leq 0,9$ . Si rien ne passe le seuil, afficher "Profil homogène".

- Pour chaque axe, une phrase générée : "<KPI> : X % de l'objectif en moyenne, contre Y % pour l'équipe."

- Taux de transformation : contrats / prospects du membre sur 3 mois. Si `S.rsm.controls[club].tti[mk][uid]` existe, l'utiliser à la place.

C. "Activité" : tâches cochées sur 4 semaines, actions de rétention (`S.loyalty` avec `userId`), actions sur résiliations (`actions[]`.by), relances d'impayés. Réutiliser `activityFor()` s'il existe.

D. "Objectifs et plan d'action" : voir le prompt 2/2.

E. "Notes d'entretien" : voir le prompt 2/2.

Bouton "Imprimer la fiche" : réutiliser la classe `body.printing` et `window.print()`, comme `recapPrint` (vers 4444). La version imprimée contient A, B, D et la dernière note partagée.

Règles UI : aucun emoji décoratif, aucun tiret cadratin dans les textes d'interface. Mise en page lisible à 375 px.

Critères d'acceptation :

- Démo, profil Hugo Lefèvre : `#/coach/u5` (Sarah Garnier) montre 6 mois de score. B2B n'apparaît ni en force ni en axe, car son objectif est 0.

- Lucas Petit (compétence 0,7 dans la démo) a au moins un axe de progrès affiché, avec la phrase comparative.

- Un membre qui ouvre `#/coach/<son uid>` voit Tendances, Forces, Activité et Objectifs, mais pas Notes. Un membre qui ouvre `#/coach/<autre uid>` est renvoyé à l'accueil.

## PROMPT CLAUDE (2/2 : NOTES, OBJECTIFS SMART, PLAN D'ACTION)

Contexte : Fit Pulse, `fitpulse.html`. Écriture par `db.set(path, value)` et `db.batch(ops)`. Les collections de premier niveau doivent être déclarées dans `emptyState()` (vers 878) pour que `normalizeState` les recrée (Firebase ne garde pas les objets vides).

Modèle de données à ajouter dans `emptyState` : `coaching: {}`.

```
S.coaching[uid] = {
 notes: { [id]: { id, at, by, type: 'entretien' | 'point' | 'recadrage' | 'felicitation', text,
shared: false, mood: 1..5 | null } },
 goals: { [id]: { id, createdAt, by, label, kpiId | null, metric: 'pct' | 'valeur' | 'activite',
target, from, due, status: 'en_cours' | 'atteint' | 'non_atteint' | 'abandonne', closedAt | null } },
 actions: { [id]: { id, createdAt, by, label, due, goalId | null, done: false, doneAt | null, owner:
'membre' | 'manager' } }
}
```

Écrans, dans la fiche `#/coach/<uid>` :

1. Onglet "Objectifs et plan d'action" :

- Formulaire SMART guidé, avec 5 champs obligatoires : Spécifique (label), Mesurable (`kpiId` + `metric` + `target`, ou "activité" + `target` en nombre d'actions), Atteignable (rappel automatique de la moyenne des 3 derniers mois du membre sur ce KPI, affichée à côté du champ `target`), Réaliste (case "Validé avec le commercial"), Temporel (`due`, au plus 90 jours après `from`).

- Avancement auto quand `kpiId` est renseigné : `metric 'valeur' = sumRange(club, uid, kpiId, from, min(today, due)) / target` ; `metric 'pct' = pct du KPI sur le mois de due` ; `metric 'activite' = nombre d'actions done entre from et due / target`.

- Statut auto à l'échéance : 'atteint' si `avancement >= 1`, sinon 'non\_atteint'. Le manager peut forcer le statut 'abandonne'.

- Plan d'action : liste d'actions avec échéance et propriétaire. Case à cocher (le membre peut cocher les siennes). Une action en retard (`due < today` et non faite) est en rouge.

- En haut, compteur : "X objectifs en cours, Y actions en retard".

2. Onglet "Notes" (managers seulement) :

- Saisie : `type`, humeur perçue 1 à 5, texte libre, case "Partager avec le commercial".

- Liste antichronologique. Les notes partagées sont aussi visibles par le membre dans sa fiche, sous le titre "Retours de mon manager".

- Modèle d'entretien prérempli au clic sur "Préparer un entretien" : il insère dans le texte les 2 forces, les 2 axes, l'état des objectifs et les actions en retard, calculés par la fiche.

3. Accueil du membre (PAGES.home) : carte "Mon plan d'action" avec ses actions non faites, triées par échéance. Elle s'affiche seulement s'il en a.

Confidentialité : en mode Firebase, documenter dans un commentaire que les règles doivent interdire la lecture de `/pulse/coaching/<uid>/notes` aux non-managers. En mode local, masquer côté interface.

Règles UI : aucun emoji décoratif, aucun tiret cadratin dans les textes d'interface.

Critères d'acceptation :

- Créer un objectif "Contrats : 18 d'ici le <fin du mois>" pour un membre. L'avancement affiché vaut ses contrats du mois / 18 et se met à jour après une saisie rapide.

- On ne peut pas enregistrer un objectif sans échéance ou avec une échéance à plus de 90 jours. Message clair.

- Une note non partagée n'apparaît jamais dans l'interface du membre concerné. Vérifier en se connectant avec son profil de démo.

- Cocher une action depuis l'accueil du membre la marque faite dans la fiche du manager (même chemin `S.coaching[uid].actions[id]`).

- La sauvegarde JSON (exportAll) contient la collection coaching, et une restauration la retrouve.

## 5. MESURER L'ACTIVITÉ, PAS SEULEMENT LES VENTES

Les ventes sont un résultat. Le manager pilote d'abord des comportements. Les données d'activité existent mais ne sont jamais consolidées. Les tâches cochées sont stockées dans `S.tasks.done[date][uid][planId]` (l.2146), mais seul "Validations du jour" les montre (l.2557). Elles sont rangées par `planId`, donc tout l'historique est perdu si une tâche est retirée du planning (`unplan`, l.2574). Les relances sont dans `S.loyalty` avec `userId`. Les gestes sur résiliations sont dans `actions[]`.by et les impayés dans `dunning`. Les prospects valent 0 pt. Il manque un indice d'activité par personne, et un croisement activité / résultat pour distinguer "travaille mais ne convertit pas" de "ne travaille pas".

### PROMPT CLAUDE

Contexte : Fit Pulse, `fitpulse.html`. Sources d'activité existantes :

- tâches cochées : `S.tasks.done[date][userId][planId] = timestamp` (cochage dans `ACTIONS.openSaisies`, vers 2146). Le planning du club est dans `S.tasks.plan[clubId][planId] = { taskId, hour }`.

- relances de rétention : `S.loyalty[id] = { clientId, type, outcome, userId, at }`.

- résiliations : `S.resiliations[id].actions[] = { at, by, label }`.

- impayés : `clients[]`.dunning et les fonctions `dunPatch` (vers 4047).

- prospects : entrées `kpiId 'prospects'`.

Travail demandé :

1. Fiabiliser l'historique des tâches : au moment du cochage, écrire `{ at, taskId, hour }` au lieu d'un simple timestamp. La lecture accepte les deux formats (nombre ou objet). Retirer une tâche du planning ne doit plus faire disparaître l'historique.

2. Fonction `activityFor(uid, clubId, from, to)`, mémorisée avec `memo()` :

- `tachesPrevues` = nombre de jours ouvrés de la période x nombre de tâches du planning du club.

Utiliser `workdays()` s'il existe, sinon les jours hors dimanche.

- `tachesFaites` = cochages du membre sur la période.
- `tauxTaches` = `tachesFaites` / `tachesPrevues`.
- `relances` = actions `S.loyalty` du membre sur la période. `relancesAbouties` = celles dont `OUTCOMES[outcome].done && !lost`.
- `gestesResil` = actions de résiliation avec `by = uid`, hors "Demande enregistrée".
- `gestesImpayes` = actions d'impayés horodatées par ce membre (ajouter `by` et `at` dans `dunPatch` si ce n'est pas déjà enregistré).
- `prospects` = `sumRange(club, uid, 'prospects', from, to)`.
- `indice` =  $0,4 \times \min(\text{tauxTaches}, 1) + 0,3 \times \min(\text{relances} / \text{cibleRelances}, 1) + 0,3 \times \min(\text{prospects} / \text{cibleProspects}, 1)$ . Les cibles valent l'objectif du mois proratisé ; `cibleRelances` par défaut = 40 par mois, réglable dans les réglages du club (`club.activityTargets = { relances, prospects }`).

3. Écran : dans Équipe & paliers, nouvel onglet "Activité". Tableau membres x [Tâches %, Relances (abouties), Gestes résiliation, Gestes impayés, Prospects, Indice], sur la semaine, le mois ou 4 semaines glissantes (seg).

4. Matrice activité / résultat : un nuage de points SVG simple, axe X = indice d'activité, axe Y = score du mois (statsFor). Quatre quadrants légendés : "Moteurs" (haut droite), "Talents à cadrer" (haut gauche), "Travaille sans convertir : coaching technique" (bas droite), "À relancer" (bas gauche). Seuils : 0,7 en X, et le rythme attendu en Y.

5. Afficher l'indice d'activité dans la fiche coaching et dans la page Pilotage équipe si elles existent.

Règles UI : aucun emoji décoratif, aucun tiret cadratin dans les textes d'interface.

Critères d'acceptation :

- Cocher 3 tâches pour un membre aujourd'hui fait monter son "Tâches %" du jour à 3 / (nombre de tâches du planning).
- Retirer une de ces tâches du planning ne change pas le nombre de tâches faites de la journée.
- Dans la démo, les 3 relances déjà faites (l0, l1, l2) apparaissent chez u3, u4 et u2, et seule celle de u4 (noanswer) est comptée "non aboutie".
- Chaque membre apparaît dans un seul quadrant, avec son prénom en étiquette.

## 6. DÉTECTER TÔT : BAISSÉ D'ACTIVITÉ ET SIGNAUX DE DÉCROCHAGE

La seule alerte individuelle est "Sans saisie aujourd'hui" (`managerCockpit`, l.4262). Elle est bruyante le matin et aveugle sur les tendances. Personne n'est prévenu quand un bon commercial perd 40 % d'activité sur deux semaines, quand un KPI décroche, ou quand quelqu'un ne se connecte plus. Les données suffisent pour une détection par règles simples et explicables.

### PROMPT CLAUDE

Contexte : Fit Pulse, `fitpulse.html`. Saisies via `sumRange`, `statsFor` et `S.entries` (`e.source === 'manual'`, `e.at`). Activité via `activityFor()` si elle existe (sinon la créer à minima : tâches cochées dans `S.tasks.done` et relances dans `S.loyalty`). Cockpit manager : `managerCockpit()` (vers 4258) sur l'accueil.

Travail demandé :

1. Suivi de connexion : à chaque `renderNow`, écrire `S.prefs[ME.id].lastSeen = Date.now()` au plus une fois par heure (comparer avec la valeur existante).
2. Fonction `alertsFor(clubId)`, mémorisée. Elle renvoie une liste { `key`, `uid`, `level`: 'alerte' | 'vigilance', `rule`, `text`, `since` }. Règles exactes, évaluées sur les membres actifs :
  - R1 Silence : aucune saisie manuelle depuis au moins 2 jours ouvrés. Niveau alerte à partir de 4 jours.
  - R2 Baisse d'activité :  $A7$  = indice d'activité sur les 7 derniers jours,  $A28$  = moyenne hebdomadaire des 28 jours précédents (J-35 à J-8). Vigilance si  $A7 < 0,7 \times A28$ , alerte si  $A7 < 0,5 \times A28$ . Règle ignorée si

A28 = 0.

R3 Baisse de résultats :  $P7 = \text{somme}(\min(\text{real7\_k} / \text{cible7\_k}, 1,5) \times \text{points\_k}) / \text{somme}(\text{points\_k})$ , avec  $\text{cible7\_k} = \text{objectif mensuel} \times 7 / \text{jours du mois}$ . P28 = même calcul sur les 28 jours précédents, ramené à 7 jours. Mêmes seuils que R2.

R4 KPI décroché : pour un KPI obligatoire,  $\text{expected} \geq 0,4$  et  $\text{pct} / \text{expected} < 0,6$ .

R5 Tâches : taux de tâches sur 7 jours  $< 0,5$  alors qu'il était  $\geq 0,8$  sur les 28 jours précédents.

R6 Trois semaines en baisse : score hebdomadaire ( $\text{statsFor}$  sur  $\text{rangeOf('week')}$ ) strictement décroissant sur les 3 dernières semaines terminées.

R7 Déconnexion :  $\text{lastSeen}$  plus vieux que 5 jours ouvrés.

La clé `key = `${rule}|${uid}|${semaine ISO}`` évite de répéter une alerte déjà traitée dans la semaine.

3. Prise en compte : `S.alertAcks[clubId][key] = { by, at, note }`. Ajouter `alertAcks: {}` à `emptyState()`. Bouton "Pris en compte" avec note facultative ; la note peut être copiée en note d'entretien si `S.coaching` existe.

4. Affichage :

- Nouvelle tuile dans `managerCockpit` : "Signaux faibles", nombre d'alertes non traitées. Couleur h-alert s'il y a au moins une alerte de niveau alerte, h-watch s'il n'y a que des vigilances.
- La tuile "Sans saisie aujourd'hui" ne passe à l'orange qu'après 14 h.
- Page ou tiroir "Signaux faibles" : liste groupée par membre. Chaque ligne affiche la règle en clair, les chiffres utilisés ("7 derniers jours : 3 relances, contre 9 en moyenne") et un lien vers la fiche.

5. Les textes doivent rester factuels et bienveillants, sans jugement : "Activité en baisse" plutôt que "Démotivé".

Règles UI : aucun emoji décoratif, aucun tiret cadratin dans les textes d'interface.

Critères d'acceptation :

- Supprimer toutes les saisies d'un membre de démo sur les 5 derniers jours déclenche R1 (alerte) et R3 pour lui seul.
- Marquer l'alerte "Pris en compte" la retire de la tuile jusqu'à la semaine suivante.
- Chaque alerte affiche les deux nombres comparés et la période.
- Aucune alerte R2 ou R3 pour un membre créé il y a moins de 35 jours (`createdAt`) : afficher "historique insuffisant".

## 7. LES RITUELS : BRIEF DU MATIN, POINT HEBDO, REVUE MENSUELLE

Il n'existe pas de support de réunion. Seul `PAGES.recap` (l.4387) est prêt les 5 premiers jours du mois (bannière l.4240). La préférence "Bilan hebdomadaire du club (lundi matin)" (`profAccount`, l.2630) est une case à cocher qu'aucun code n'utilise : c'est une promesse non tenue. Un brief du matin généré en un clic, lisible en 5 minutes et copiable dans le groupe WhatsApp, ferait de Fit Pulse l'outil du quotidien du manager.

### PROMPT CLAUDE

Contexte : Fit Pulse, `fitpulse.html`. Briques disponibles : `statsFor`, `ranking`, `palierState(clubId, mk, kpiId)`, `paliersFor`, `myToDo()` (relances par membre), `resToHandle`, `resUrgent`, `dunRows`, `dunDue`, `loyaltyTasks`, `S.challenges`, `sendChat(o)` (vers 2290). La préférence 'digest' (`profAccount`, vers 2630) n'est utilisée nulle part.

Travail demandé : un menu "Rituels" (bouton sur l'accueil du manager, au-dessus du cockpit) avec trois supports générés.

1. Brief du matin (modale, lecture en 5 minutes). Sections fixes, dans cet ordre :

- "Hier" : pour chaque membre, ce qu'il a saisi hier, en une ligne ("Inès : 2 contrats, 45 € nutrition, 1 avis"). Les membres sans saisie sont listés à part.
- "Le mois" : jour X / N, rythme attendu, et pour chaque KPI obligatoire du club : réalisé / objectif, pct et statut (`statusOf`).
- "Paliers" : pour chaque palier collectif non atteint, "Encore X contrats pour P2, soit Y par jour"

(même calcul que palierBlock).

d. "Bravo" : le membre avec le meilleur Plj d'hier,  $Plj = \text{somme}(\min(\text{real\_k} / (\text{objectif\_k} / \text{jours du mois}), 2) \times \text{points\_k}) / \text{somme}(\text{points\_k})$ . En cas d'égalité, le plus de contrats.

e. "Focus du jour" : le KPI obligatoire du club au plus faible pct / expected, avec la quantité par personne à faire aujourd'hui =  $\text{reste} / \text{jours restants} / \text{membres actifs}$ .

f. "Aujourd'hui" : résiliations à J-7 ou moins, impayés à relancer aujourd'hui, anniversaires et appels de suivi du jour, défi flash en cours.

Boutons : "Copier le texte" (texte brut sans HTML, lignes courtes, compatible messagerie), "Publier dans le chat du club" (sendChat avec le texte), "Imprimer".

2. Point hebdo (le lundi, ou à la demande) : semaine précédente comparée à la semaine d'avant, par membre. Colonnes : score de la semaine (statsFor sur rangeOf('week')), évolution en points, indice d'activité si disponible, objectifs SMART en cours et actions en retard si S.coaching existe. Puis 3 questions d'animation générées à partir des données ("Quelle action a permis à X de doubler ses avis ?").

3. Revue mensuelle : réutiliser PAGES.recap et ajouter en tête une synthèse en 5 phrases générées : meilleur KPI, pire KPI, meilleure progression individuelle, membre à accompagner (plus bas score / expected), paliers atteints. Puis une page par membre (sa ligne de matrice, ses forces et axes, son plan d'action), imprimable.

4. Digest : implémenter la préférence 'digest'. Le lundi, à la première ouverture, si pref digest est vrai, afficher en haut de l'accueil une carte "Bilan de la semaine" (contenu du point hebdo résumé en 4 lignes), refermable. Stocker la date de fermeture dans prefs.digestSeen. Si la fonction n'est pas faite, retirer la case de profAccount.

Règles UI : aucun emoji dans les textes générés ni dans l'interface, aucun tiret cadratin. Phrases de 15 mots au plus dans les textes générés.

Critères d'acceptation :

- Démo, profil Hugo Lefèvre, un mardi : le brief liste les membres de Niort avec leurs saisies de lundi, et le total du bloc "Hier" est égal à la somme des saisies du jour précédent dans l'historique des saisies.
- "Copier le texte" place dans le presse-papiers un texte sans balise HTML et sans emoji.
- "Publier dans le chat" crée un message unique dans le canal du club, visible par un membre.
- Le lundi, la carte Bilan de la semaine apparaît une fois. Après fermeture, elle ne revient pas avant le lundi suivant.

## 8. FIXER LES OBJECTIFS : RÉPARTITION, SAISONNALITÉ, N-1

La fixation est entièrement manuelle. memTargets (l.2499) est une grille de cases, avec seulement "Reprendre les objectifs du mois précédent" (copyTargets, l.2513). Le reste est déjà calculé mais jamais branché : "Contrats à signer" = objectif fin de mois - base nette (clubBase, l.3091), l'historique N-1 du club (S.monthly, utilisé dans dashAnalyses, l.2050) et l'évolution à période égale. La démo donne d'ailleurs le même objectif à tout le monde (l.1040), quels que soient l'ancienneté, le temps de travail ou l'historique.

### PROMPT CLAUDE

Contexte : Fit Pulse, fitpulse.html. Objectifs individuels : S.targets[mk][uid][kpiId], édités dans memTargets (vers 2499). Historique club : S.monthly[clubId][mk] = { contrats, complements, goodies, impayes, ... } ; correspondance kpi -> champ : contrats -> contrats, nutrition -> complements, accessoires -> goodies, impayes -> impayes. Base adhérents : S.base[clubId][mk] = { actifs, sortants, objectif }. Absences : S.absences si elles existent.

Travail demandé : un assistant "Répartir l'objectif du club", accessible par un bouton dans l'onglet Objectifs.

Étape 1, objectif club du mois mk, pour chaque KPI :

- Suggestion saisonnière :  $G\_sugg = \text{valeur N-1 du même mois} \times (1 + \text{croissance})$ . Croissance = (somme N des mois terminés de l'année - somme N-1 des mêmes mois) / somme N-1 des mêmes mois, bornée entre -20 % et +30 %. Sans N-1 dans S.monthly, utiliser la somme des saisies du club pour ce mois de N-1. Sinon, la moyenne des 3 derniers mois.

- Pour contrats, afficher aussi "Besoin base adhérents" =  $S.\text{base}[\text{club}][\text{mk}].\text{objectif} - (\text{actifs} - \text{sortants})$  quand il existe.

- Le manager choisit G pour chaque KPI (champ prérempli avec G\_sugg, modifiable).

Étape 2, poids de chaque membre i actif :

-  $\text{presence\_i} = \text{jours ouvrés prévus de i dans le mois} / \text{jours ouvrés du club}$ . Utiliser workdays() et les absences si disponibles, sinon 1.

-  $\text{historique\_i} = \text{part de i dans le réalisé de l'équipe sur ce KPI, sur les 3 derniers mois terminés}$ . Sans historique, part égale.

-  $\text{rampe\_i}$  selon l'ancienneté (createdAt) : 50 % le 1er mois, 75 % le 2e, 100 % ensuite.

-  $w\_i = \text{presence\_i} \times \text{rampe\_i} \times (\alpha \times (1 / n) + (1 - \alpha) \times \text{historique\_i})$ , avec alpha réglable par un curseur de 0 à 1 (0,5 par défaut). alpha = 1 donne une répartition égale, alpha = 0 suit l'historique.

-  $\text{objectif\_i} = G \times w\_i / \text{somme}(w)$ . Arrondi à l'entier pour qty, à 10 € pour eur, par la méthode du plus fort reste, pour que la somme des objectifs vaille exactement G.

Étape 3, aperçu : tableau membres x KPI avec l'objectif proposé, l'objectif actuel, l'écart, et la moyenne réalisée sur 3 mois en gris. Chaque case reste modifiable. Si un objectif proposé dépasse 130 % de la moyenne réalisée du membre, afficher "Ambitieux". En dessous de 80 %, afficher "Facile".

Étape 4, "Appliquer" : un seul db.batch qui écrit S.targets[mk][uid][kpi]. Stocker S.targetPlans[clubId][mk] = { G, alpha, at, by } pour la traçabilité (ajouter targetPlans: {} à emptyState).

Règles UI : aucun emoji décoratif, aucun tiret cadratin dans les textes d'interface.

Critères d'acceptation :

- Pour G = 100 contrats et 4 membres, la somme des objectifs proposés vaut exactement 100, quel que soit alpha.

- alpha = 1 et présences égales : chaque membre reçoit 25.

- Un membre avec 10 jours de congé sur 25 jours ouvrés reçoit 60 % de l'objectif d'un collègue au profil identique.

- Un membre créé ce mois-ci reçoit la moitié de l'objectif qu'il aurait sans la rampe.

- L'assistant n'écrit rien tant que "Appliquer" n'est pas cliqué. L'annulation laisse S.targets intact.

## 9. PRIMES : SIMULATION TRANSPARENTE DE LA RÉMUNÉRATION VARIABLE

Les paliers collectifs existent (S.paliers, l.4097 ; memPaliers, l.4285), mais la récompense est un texte libre ("Prime 50 € chacun", l.1123). Rien n'est chiffré, ni par personne, ni pour le coût du club. Il n'y a aucune prime individuelle. Le commercial ne peut pas répondre à la question qui motive le plus : "si je signe encore 3 contrats, je gagne combien ?". Une simulation transparente, clairement marquée "indicative", ajoute de la clarté et de la confiance.

### PROMPT CLAUDE

Contexte : Fit Pulse, fitpulse.html. Paliers collectifs : S.paliers[clubId][mk][kpiId] = [{ target, reward }] (texte libre), édités dans memPaliers (vers 4285). Calcul : palierState (vers 4106). Objectifs individuels : S.targets. Réalisé : sumRange.

Travail demandé :

1. Modèle : `S.comp[clubId][mk] = {`  
     `individual: { [kpiId]: { mode: 'unite' | 'paliers', perUnit: number, minPct: number, tiers: [{ pct,`  
     `amount }] } },`  
     `cap: number | null,`  
     `prorataPresence: true`  
   `}`  
 Ajouter comp: {} à emptyState. Ajouter un champ numérique amount (en € par personne) à chaque palier collectif, à côté du texte reward, qui reste.

2. Calcul pour un membre i, un mois :

- Mode 'unite' : `prime_k = pct_k >= minPct ? real_k x perUnit : 0.`
- Mode 'paliers' : `prime_k = montant du plus haut palier { pct } atteint par pct_k = real_k / objectif_k.`
- Collectif : somme des amount des paliers collectifs atteints par le club, multipliée par `presence_i` (jours présents / jours ouverts) si `prorataPresence`.
- Total = `min(cap, somme des prime_k) + collectif`. Sans cap, pas de plafond sur la part individuelle.

3. Écrans :

- Manager, onglet "Primes" dans Équipe & paliers : réglage des règles du mois (bouton "Reprendre le mois précédent"), puis tableau membres x [part individuelle, part collective, total], avec le coût total du club et sa projection de fin de mois (calcul avec `real / expected`).
- Membre, carte "Ma prime estimée" sur l'accueil et dans Mes objectifs : montant acquis à date, montant projeté, et un simulateur "Si je fais encore [ - ] N [ + ] <KPI>" qui recalcule le total en direct. Détail ligne par ligne : "Contrats : 14 x 5 € = 70 €".
- Mention obligatoire sous chaque montant : "Simulation indicative, non contractuelle. Seule la fiche de paie fait foi."

4. Visibilité : un membre ne voit que sa propre prime. Seul le manager voit le tableau de l'équipe.

Règles UI : aucun emoji décoratif, aucun tiret cadratin dans les textes d'interface. Montants au format `fmtE`.

Critères d'acceptation :

- Règle contrats en mode 'unite', 5 € par contrat dès 80 %, objectif 20 : 15 contrats donnent 0 € (75 %), 16 contrats donnent 80 €.
- Le simulateur passe de 0 € à 80 € quand on ajoute 1 contrat à 15.
- Palier collectif P1 à 50 € atteint : un membre présent 10 jours sur 20 reçoit 25 € de part collective.
- Un membre connecté ne peut afficher la prime d'aucun autre membre, ni par l'interface ni par l'URL.

## 10. MULTI-CLUBS : COMPARER ET PARTAGER LES BONNES PRATIQUES

Un manager sur deux clubs (Camille dans la démo, l.1018) doit changer de club dans le sélecteur pour tout voir. La comparaison se limite à `clubRanking` (l.1329), un score moyen par club, et à `clubList` (l.3063), un % de points du mois. Aucun benchmark par KPI, rien de ramené à l'effectif ou à la base adhérents, aucune comparaison N-1 entre clubs.

### PROMPT CLAUDE

Contexte : Fit Pulse, `fitpulse.html`. `myClubs()` renvoie les clubs du manager connecté. Données par club : `statsFor(clubId, null, r)`, `clubMembers(clubId)`, `S.base[clubId][mk].actifs`, `S.monthly[clubId]`, `monthFigures(clubId, mk)` (vers 4331 : contrats, résiliations, taux, impayés, avis, boutique), `S.rsm.controls[clubId].tti[mk]`.

Travail demandé : dans Mes clubs, nouvel onglet "Comparatif", visible quand `myClubs().length > 1`. Il ne dépend pas du club sélectionné.

1. Tableau clubs x indicateurs, mois choisi par `monthNav` :

- Score moyen (clubRanking).
- Par KPI obligatoire : % de l'objectif club, puis valeur par membre actif = réalisé / nombre de membres actifs.
- Contrats pour 1 000 adhérents = contrats / actifs x 1000.
- Taux de résiliation et taux de sauvetage (monthFigures).
- Part équipe des impayés récupérés = recEquipe / recupere.
- Taux de transformation (tti) quand il est importé.
- Évolution N / N-1 à période égale des contrats (même méthode que dashAnalyses : mois terminés seulement).

2. Pour chaque indicateur, le meilleur club est en gras. L'écart du club le plus faible au meilleur s'affiche en points.

3. Bloc "À dupliquer" : pour chaque indicateur où l'écart dépasse 20 %, une phrase générée : "La Rochelle convertit 34 % des prospects contre 22 % à Niort." Plus un bouton "Partager dans le canal commun" (sendChat sur le canal 'all').

4. Vue consolidée : une ligne "Total nos clubs" (sommes et moyennes pondérées par les actifs).

5. Export CSV du tableau, avec un séparateur point-virgule et un BOM UTF-8 pour Excel (downloadFile).

Règles UI : aucun emoji décoratif, aucun tiret cadratin dans les textes d'interface.

Critères d'acceptation :

- Démo, profil Camille Roux : l'onglet Comparatif affiche Niort et La Rochelle côte à côte sans changer de club.

- Contrats pour 1 000 adhérents de Niort = contrats du mois / S.base.niort[mk].actifs x 1000, arrondi à une décimale.

- Avec un seul club, l'onglet n'apparaît pas.

- Le CSV s'ouvre dans Excel avec les accents corrects et une colonne par indicateur.

## 11. EXPORT ET PARTAGE : RÉCAP PDF, ENVOI AU DIRECTEUR, FICHIERS EXPLOITABLES

Les sorties actuelles sont limitées. Il y a l'impression navigateur du récap (recapPrint, l.4444), l'image PNG du bilan mensuel (wrapDownload, l.2700) et la sauvegarde JSON complète (exportAll, l.3109). Le manager ne peut ni exporter un tableau en CSV, ni préparer un envoi au directeur, ni produire un bilan individuel sobre pour l'entretien (le wrap est au format stories).

### PROMPT CLAUDE

Contexte : Fit Pulse, fitpulse.html. downloadFile(name, content, type) existe (vers 1746). L'impression passe par body.printing et window.print() (recapPrint, vers 4444), avec un bloc @media print vers la ligne 681. PAGES.recap (vers 4387) construit le récap mensuel.

Travail demandé :

1. Réglage club : champ club.directorEmail (et un nom), éditable dans le formulaire du club (clubForm).

2. Dans PAGES.recap, bouton "Envoyer au directeur" : ouvre un lien mailto:<directorEmail> avec :

- sujet : "Récapitulatif <club> <mois>" ;

- corps texte brut (moins de 1 800 caractères, encodé avec encodeURIComponent) : les 8 tuiles du récap avec leur variation ("Contrats : 112 (+8, +8 %)", les 3 faits marquants générés (meilleure hausse, plus forte baisse, palier atteint), puis la phrase "PDF joint : Imprimer / PDF depuis Fit Pulse".

- Sans directorEmail, ouvrir le formulaire du club avec le message "Renseignez l'e-mail du directeur".

3. Exports CSV (séparateur point-virgule, BOM UTF-8, nombres avec virgule décimale) :

- Historique des saisies du mois (memHistory, mode mois) : membres x KPI, objectif et %.
  - Matrice Pilotage équipe, si la page existe.
  - Tableau "par commercial" du récap.
- Nom de fichier : fitpulse-<type>-<club>-<AAAA-MM>.csv.

4. Bilan individuel imprimable (format A4, sobre, noir sur blanc) depuis memRecaps : bouton "Bilan PDF" à côté de "Voir le bilan". Contenu : score, rang, tableau KPI (réel, objectif, %, écart), comparaison au mois précédent et à la moyenne des 3 mois, et un encadré vide "Commentaire du manager" à remplir à la main, ou prérempli avec la dernière note partagée si S.coaching existe.

5. Feuille de style d'impression : masquer la navigation, les boutons et la barre du téléphone. Forcer le thème clair, éviter les coupures dans les cartes (break-inside: avoid) et mettre le nom du club et le mois en en-tête de page.

Règles UI : aucun emoji dans les exports ni dans les textes, aucun tiret cadratin.

Critères d'acceptation :

- "Envoyer au directeur" ouvre le client mail avec le bon destinataire, le bon sujet et un corps lisible sans HTML.
- Le CSV de l'historique, ouvert dans Excel, affiche "Contrats signés" avec son accent et des montants comme 1 234,50.
- L'aperçu avant impression du bilan individuel tient sur une page A4 pour 9 KPI.
- Aucun emoji n'apparaît dans le PDF ni dans le CSV (les champs emoji des KPI sont exclus).

## 12. FIABILITÉ DES CHIFFRES : CONTRÔLE PAR VENDEUR ET TRAÇABILITÉ DES CORRECTIONS

Piloter suppose des chiffres sûrs. Aujourd'hui, un membre peut saisir à n'importe quelle date passée : le champ date n'a qu'un `max` (l.2129). Un manager peut saisir pour un autre (`by`, l.2156) ou corriger un total par écriture d'ajustement (`histCell`, l.2487), sans aucun journal visible. Le contrôle Resamania compare uniquement le total du club (`rsmDoneCard`, l.3767), alors que l'export "performances commerciales" est déjà stocké par vendeur (`S.rsm.controls[club].perf[mk][uid]`, l.3752).

### PROMPT CLAUDE

Contexte : Fit Pulse, fitpulse.html. Saisies : `S.entries[id] = { userId, clubId, kpiId, date, value, source, at, by, adjust?, importId? }`. Contrôle Resamania par vendeur déjà importé : `S.rsm.controls[clubId].perf[mk][uid]` = nombre de contrats (écrit vers 3752). `rsmDoneCard` (vers 3764) ne compare que le total.

Travail demandé :

1. Contrôle par vendeur : dans Équipe & paliers > Historique des saisies (mode mois), si `perf[mk]` existe, ajouter une colonne "Contrats Resamania" à côté de "Contrats signés", avec l'écart. Écart toléré = max(1, 5 % de la valeur Resamania). Au-delà, la case est orange, avec l'infobulle "Écart avec Resamania : vérifier les saisies". Les vendeurs Resamania non rattachés (`uid` commençant par "x:") sont listés en bas avec un lien vers Correspondances Resamania.
2. Verrouillage des mois clos : réglage `club.lockDay` (5 par défaut). À partir du jour `lockDay` du mois M, un membre ne peut plus saisir, ni supprimer, une entrée datée du mois M-1 ou avant. Le champ date du formulaire reçoit `min` = premier jour autorisé. Le manager garde la main, mais chaque saisie de sa part sur un mois clos demande une confirmation et un motif, stocké dans `entry.reason`.
3. Journal des corrections : nouvel onglet "Journal" dans Équipe & paliers. Il liste les entrées dont `by !== userId`, celles avec `adjust: true`, et les suppressions. Pour les suppressions, ajouter une collection `S.audit[id] = { at, by, action: 'delete' | 'adjust' | 'proxy', entry, reason }` alimentée par la suppression de saisie (`data-delentry`), `histCell` et la saisie pour autrui. Ajouter `audit: {}` à `emptyState`.

Filtres : membre, mois, type.

4. Dans la fiche coaching et la page Pilotage équipe, si elles existent, une pastille "X corrections ce mois" par membre.

Règles UI : aucun emoji décoratif, aucun tiret cadratin dans les textes d'interface.

Critères d'acceptation :

- Démo : après un import "performances commerciales" du mois précédent, la colonne Resamania apparaît pour les vendeurs rattachés, et l'écart est calculé sur les saisies de ce mois.
- Le 6 du mois, un membre ne peut pas choisir une date du mois précédent. Un manager le peut, après avoir saisi un motif.
- Supprimer une saisie crée une ligne dans le Journal avec l'auteur, la date et la valeur supprimée.
- Une correction par histCell apparaît dans le Journal avec l'ancien et le nouveau total du jour.

## 13. RECONNAISSANCE MANAGÉRIALE ET TON DES MESSAGES

La reconnaissance est entièrement automatique : trophées (`allTrophies`, l.1355), réactions du feed et phrase du bilan choisie selon le score (l.2651). Le manager ne peut pas ajouter un mot personnel au bilan. L'effort n'est jamais valorisé (relances, régularité, progression contre soi-même). La phrase "Mois difficile. On repart plus fort." tombe, sans contexte, sur quelqu'un qui a pu être absent la moitié du mois. Le classement all-time (l.2204) met aussi côte à côte des anciens et des nouveaux, ce qui décourage les arrivants.

### PROMPT CLAUDE

Contexte : Fit Pulse, `fitpulse.html`. Bilan mensuel : `PAGES.wrap` (vers 2640) avec `slides[]` et une phrase selon `st.score` (vers 2651). Trophées : `allTrophies` (vers 1355). Classement all-time : `PAGES.leaderboard` (vers 2204).

Travail demandé :

- Mot du manager : modèle `S.wrapNotes[mk][uid] = { text, by, at }` (ajouter `wrapNotes: {}` à `emptyState`). Dans `memRecaps`, bouton "Ajouter un mot" sur chaque ligne (`textarea`, 280 caractères au plus). Le bilan affiche ce mot sur une slide dédiée, juste avant "Ton résumé", signée du prénom du manager. Sans mot, la slide est absente.
- Phrase de fin contextuelle : la phrase du bilan tient compte de la progression contre soi-même. Si `score >= moyenne des 3 mois précédents + 10 points`, écrire "Ta meilleure progression depuis 3 mois.". Si le membre a plus de 5 jours d'absence (`S.absences` si disponible), écrire "Mois court : ton rythme sur tes jours présents était de X %." ( $X = \text{score} / \text{part des jours présents}$ ). Sinon, garder les phrases actuelles.
- Trophées d'effort, ajoutés dans `allTrophies` pour les mois terminés :
  - "Régularité" : au moins une saisie chaque jour ouvré du mois.
  - "Meilleure progression" : la plus forte hausse de score par rapport au mois précédent dans le club (au moins +10 points).
  - "Pilier rétention" : le plus de relances abouties du mois (`S.loyalty`), avec un minimum de 15.
- Classement all-time : ajouter un filtre "12 derniers mois", sélectionné par défaut, à côté de "Score" et "Badges". Le cumul ne prend que les 12 derniers mois.

Règles UI : aucun emoji ajouté dans les nouveaux textes, aucun tiret cadratin dans les textes d'interface.

Critères d'acceptation :

- Un mot ajouté par le manager pour un membre apparaît dans le bilan de ce membre, et dans aucun autre bilan.

- Un membre qui passe de 55 % à 70 % de score reçoit le trophée "Meilleure progression" s'il a la plus forte hausse du club.
- Le filtre "12 derniers mois" exclut les points des mois plus anciens (vérifiable avec un membre archivé de la démo).

# 7. REVENUS : MAXIMISER LE CHIFFRE D'AFFAIRES DU CLUB

Hypothèses pour un club type : 1 600 adhérents, prix moyen 29,99 € TTC par mois (Basic, Premium, Ultimate mélangés), soit environ 48 000 € de revenu récurrent mensuel (MRR). Durée de vie moyenne 14 mois, donc une valeur adhérent d'environ 420 €. 110 contrats et 3 % de résiliations par mois (environ 48 demandes). Les montants plus bas sont des ordres de grandeur annuels, à recalculer sur les vraies données du club.

Constat transversal : l'appli compte bien les actes (contrats, sauvetages, impayés) mais presque jamais les euros qu'ils représentent. L'import "Vente d'abonnements" lit déjà l'offre, le prix et le canal de chaque adhérent (`c.client(num, { offer, canal, price })`, ligne ~3290), et aucun écran ne s'en sert : `grep "\.price"` ne renvoie rien. C'est la matière première de presque tous les points ci-dessous.

## 0. PRÉREQUIS : UN SOCLE "VALEUR" FIABLE (MRR, VALEUR ADHÉRENT, CHURN EN EUROS)

`caMonth()` (ligne 1490) additionne les KPI en euros : nutrition, accessoires et impayés récupérés par l'équipe. Il manque les abonnements, qui font plus de 90 % du revenu, et un impayé récupéré n'est pas un nouveau chiffre d'affaires. La courbe "CA" du tableau de bord (`dashAnalyses`, ~2066) est donc trompeuse. Les exports "Paielements" (encaissé réel, ~3452) et "Évolution clients" (gains et pertes avec le nom de l'abonnement, ~3466) sont importés mais réduits à un total ou à un compteur. Sans ce socle, aucun levier ne peut être classé en euros.

### PROMPT CLAUDE

Contexte : application Fit Pulse, fichier HTML unique (`fitpulse.html`). Ajoute un socle de calcul "valeur" sans casser l'existant.

1. Barème des offres. Dans Mes clubs > Réglages, ajoute un tableau "Offres et prix" stocké dans `S.offers[clubId]` : { libelle, gamme ('basic'|'premium'|'ultimate'|'autre'), prixMensuelTTC, fraisInscription }. Pré-remplis la liste avec les valeurs distinctes de `S.clients[*].offer` du club, et propose un prix par défaut égal à la médiane des `c.price` importés pour ce libellé (import "Vente d'abonnements", colonne "Prix toutes taxes"). La gamme est devinée par mots clés (basic, premium, ultimate, sinon autre) et reste modifiable.

2. Fonctions pures, à côté de `caMonth()` :

- `mensualite(client)` = `prixMensuelTTC` de son offre dans `S.offers`, sinon `client.price`, sinon le prix moyen du club.
- `mrrClub(clubId, mk)` = somme des mensualités des clients actifs du club (statut non ancien, perdu, prospect, exclu, temporaire, même filtre que `loyaltyTasks`).
- `dureeVieMois(clubId)` = `1 / tauxResiliationMoyen`, où `tauxResiliationMoyen` = moyenne sur les 6 derniers mois terminés de (résiliées du mois / actifs du mois), avec `monthFigures()`. Bornes : entre 6 et 36 mois. Si aucune donnée : 14.
- `valeurAdherent(client)` = `mensualite(client) x dureeVieMois(clubId)`.
- `churnEuros(clubId, mk)` = somme des mensualités des résiliations effectives du mois (rapprochement résiliation / client par `tokensKey` du nom).
- `encaisseMois(clubId, mk)` = somme de `S.rsm.controls[clubId].payments` sur le mois (export Paiements, statut validé).

3. Renomme la courbe "CA" du tableau de bord en "Ventes additionnelles (boutique)" et n'y garde que nutrition + accessoires. Les impayés récupérés restent sur leur propre carte.

4. Dans le Récap du mois (`PAGES.recap`), ajoute une rangée de 4 tuiles : "Revenu récurrent mensuel", "Encaissé réel" (avec la mention "source : export Paiements" ou "non importé"), "Churn en euros" (mensualités perdues ce mois), "Valeur d'un adhérent". Chaque tuile affiche la variation par rapport au mois précédent avec `delta()`.

Règles d'interface : aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin dans les textes affichés.

Critères d'acceptation :

- Avec les données de démonstration, mrrClub renvoie un nombre > 0 et valeurAdherent est entre 6 et 36 fois la mensualité.
- Un client dont l'offre n'a pas de prix dans S.offers prend c.price ; sans c.price, il prend le prix moyen.
- La courbe du tableau de bord n'additionne plus les impayés (vérifier : saisir 100 € d'impayés ne la change pas).
- Si l'export Paiements n'a jamais été importé, la tuile "Encaissé réel" affiche "non importé" et pas 0 €.
- grep sur le diff : aucun caractère U+2014 ni U+2013, aucun emoji ajouté.

## 1. ENTONNOIR PROSPECT, VISITE, ESSAI, CONTRAT : VOIR OÙ ÇA FUT

Plus gros gisement. Le taux de conversion du tableau de bord (~1980) divise les contrats du mois par les prospects créés du mois : deux cohortes différentes. L'import Prospects (~3412) ne garde qu'un compteur et jette les colonnes "Statut de prospection", "Valeur du prospect" et la provenance. Les invités ne sont qu'une saisie manuelle ( QUICK\_QTY , ligne 4158) sans liste nominative, et le canal du contrat (web, accueil) lu à l'import n'est jamais affiché. Passer de 35 % à 40 % de transformation sur 250 prospects par mois, c'est 12 contrats de plus, soit environ 5 000 € de valeur vie par mois, 50 000 à 60 000 € par an.

### PROMPT CLAUDE

Contexte : Fit Pulse, fichier HTML unique. Objectif : un entonnoir commercial par cohorte, par commercial et par canal.

Données sources (Resamania) :

- Prospects : Clients > Prospects > Exporter. Lire en plus du nom, prénom, date de création et commercial : "Statut de prospection", "Valeur du prospect", "Provenance" ou "Origine" ou "Source" (colonne optionnelle, chercher par mot clé), téléphone, e-mail. Stocker chaque prospect dans S.prospects[id] = { clubId, nom, prenom, tel, email, creeLe, commercialId, statut, provenance, valeur } avec un id stable 'p' + hkey(club + '|' + tokensKey(nom prenom) + '|' + date). Garder l'entrée KPI 'prospects' existante.
- Vente d'abonnements : déjà importé. Garder c.canal et c.offer.
- Invités : ajouter un formulaire rapide "Invité" (nom, téléphone, date de passage, parrain adhérent facultatif) stocké dans S.guests.

Rapprochement : un prospect est "converti" si un client du même club a le même tokensKey(nom prénom) ou le même téléphone normalisé, avec une date de début entre creeLe et creeLe + 60 jours. Délai de conversion = start - creeLe en jours.

Calculs :

- Cohorte = prospects créés dans le mois M. Taux de transformation cohorte = convertis à J+60 / créés. Afficher "en cours" tant que M + 60 jours n'est pas passé.
- Étapes : Créés, Contactés (statut contient contact, rdv, visite, essai), Visite ou essai (statut contient visite, essai, rdv honoré, ou invité enregistré au même nom), Contrat.
- Par commercial et par provenance : créés, convertis, taux, délai médian, et euros signés = somme des mensualités (fonction mensualite si elle existe, sinon c.price) x 12.

- Fuite principale = étape avec la plus forte perte relative ; l'afficher en phrase : "Le passage visite vers contrat perd 58 % des prospects de Julie, contre 41 % pour le club."

Écran : nouvel onglet "Entonnoir" dans Mes objectifs (manager voit le club, commercial voit le sien). Barres horizontales par étape, tableau par commercial, tableau par provenance, liste "Prospects chauds non convertis" : créés il y a 2 à 21 jours, non convertis, triés par valeur du prospect puis date, avec bouton "Appeler" (lien tel:) et "Noter un appel".

Remplace le calcul du taux de conversion du tableau de bord (~ligne 1980) par le taux cohorte du mois M-1 quand S.prospects est rempli, et garde l'ancien calcul en repli.

Règles d'interface : aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- Importer deux fois le même fichier Prospects ne crée pas de doublon dans S.prospects.
- Un prospect créé le 3 et dont le client démarre le 20 du même mois compte converti avec un délai de 17 jours.
- Un prospect sans correspondance après 60 jours compte "non converti".
- Le total "Créés" de l'entonnoir égale la somme du KPI prospects sur la même période.
- Si la colonne de provenance est absente, le tableau par provenance affiche "Provenance non renseignée dans l'export" au lieu d'être vide.

## 2. SAUVETAGE DES RÉSILIATIONS EN EUROS, ET FINS D'ENGAGEMENT ANTICIPÉES

Les résiliations sont un compteur ( `resFunnel` , 1820 ; page 3873). Une demande Ultimate à 39,99 € engagée encore 8 mois pèse beaucoup plus qu'un Basic sans engagement, mais elles sont triées pareil (urgence puis date). L'offre proposée ( `RES_OFFERS` , 3863) est notée en texte libre dans le journal et jamais mesurée. Les fins d'engagement à 30 jours ( `loyaltyTasks` , 1465) n'ont pas de valeur non plus. +10 points de taux de sauvetage sur 48 demandes, c'est environ 5 adhérents gardés par mois, soit 2 000 € de valeur vie par mois, 20 000 à 25 000 € par an.

### PROMPT CLAUDE

Contexte : Fit Pulse, fichier HTML unique. Page Résiliations ( `PAGES.resiliations` , `resCard` , `resFunnel` ) et Action Rétention ( `loyaltyTasks` ).

1. Valeur en jeu par demande :  $\text{valeurEnJeu}(r) = \text{mensualite}(\text{client}) \times \text{moisRestants}$ , où `client` est rapproché par `tokensKey(r.client)` dans `S.clients` du club, et `moisRestants` =  $\max(1, \text{mois entre aujourd'hui et client.end si client.end existe, sinon } \text{dureeVieMois}(\text{clubId}) / 2)$ . Si `mensualite` n'existe pas encore, utiliser `client.price`, sinon 29,99. Arrondir à l'euro.

2. Page Résiliations :

- Tri de l'onglet "À traiter" : urgentes (échéance  $\leq 7$  jours) d'abord, puis `valeurEnJeu` décroissante.
- `resCard` affiche "En jeu : 320 €" et l'offre de l'adhérent.
- Bandeau de stats : ajouter "Valeur sauvée ce mois" (somme des `valeurEnJeu` des dossiers passés à `sauvee` dans le mois) et "Valeur perdue ce mois".
- Le formulaire d'appel enregistre l'offre choisie dans un champ structuré `action.offer` (pas seulement dans le libellé).
- Onglet "Historique du mois" : tableau "Offre proposée" avec, par offre de `RES_OFFERS` : proposée N fois, acceptée M fois (dossier sauvé après cette offre), taux, valeur sauvée. Et tableau par motif : demandes, sauvées, taux.

3. Action Rétention, type renouvellement : ajouter la valeur en jeu ( $\text{mensualite} \times \text{dureeVieMois}$ ) sur chaque tâche, trier par valeur décroissante, et étendre la fenêtre à 45 jours avant `client.end`.

4. KPI "Sauvetage" : garder le compteur pour le classement, mais afficher sous la carte "soit X € de

valeur gardée".

Règles d'interface : aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- Deux demandes ouvertes non urgentes de 30 € x 2 mois et 40 € x 8 mois : celle à 320 € s'affiche en premier.
- Une demande sans client rapproché affiche "Valeur estimée" et utilise le prix moyen.
- Marquer un dossier "Sauvée" augmente immédiatement "Valeur sauvée ce mois" de sa valeurEnJeu.
- Le tableau des offres somme exactement le nombre d'appels où une offre autre que "Aucune" a été notée.

### 3. PAGE "OPPORTUNITÉS" : QUOI FAIRE CE MATIN POUR GAGNER LE PLUS

Aujourd'hui, la liste du jour ( `todoList` , 4191) mélange résiliations, impayés et appels sans ordre de valeur, ne montre que les dossiers déjà pris par l'utilisateur, et ignore prospects, montée en gamme et réactivation. Le cockpit manager (4258) donne des compteurs, pas des euros. Une page unique, classée par euros attendus, transforme les points 1, 2, 4, 5, 6 et 7 en gestes concrets. Gain : la somme des leviers exécutés, avec un effet d'exécution de l'ordre de 15 000 à 20 000 € par an à lui seul.

#### PROMPT CLAUDE

Contexte : Fit Pulse, fichier HTML unique. Crée une page `PAGES.opportunités` (route `#/opportunités`), en tête du menu NAV, visible par tous (un commercial voit les siennes et les non attribuées, le manager voit tout le club et peut attribuer).

Moteur : fonction `opportunités(clubId, userId)` qui renvoie une liste d'objets { `type`, `titre`, `sousTitre`, `client`, `eurosPotentiels`, `probabilite`, `eurosAttendus`, `echance`, `action`, `ownerId` }. `eurosAttendus` = `eurosPotentiels` x `probabilite`. Probabilités par défaut, modifiables dans Réglages (`S.oppConfig`) et recalibrées chaque mois sur les résultats réels quand il y a au moins 20 cas :

- Résiliation ouverte : `potentiel` = `valeurEnJeu(r)` ; `probabilité` = `taux de sauvetage` des 3 derniers mois (défaut 0,25) ; x 1,5 si `échéance` <= 7 jours.
- Impayé : `potentiel` = `solde` ; `probabilité` selon l'âge (aujourd'hui - `balanceAt`) : 0 à 7 j 0,70 ; 8 à 30 j 0,50 ; 31 à 60 j 0,30 ; plus de 60 j 0,10.
- Prospect chaud (`S.prospects`, non converti, créé il y a 2 à 21 jours) : `potentiel` = `prix moyen` x `dureeVieMois` ; `probabilité` = `taux cohorte` du commercial (défaut 0,30), divisé par 2 au-delà de 10 jours.
- Invité non converti (`S.guests`, passé il y a 0 à 10 jours) : `potentiel` = `prix moyen` x `dureeVieMois` ; `probabilité` 0,35.
- Fin d'engagement dans 45 jours : `potentiel` = `mensualite` x `dureeVieMois` ; `probabilité` 0,15.
- Montée en gamme (client Basic actif depuis 60 à 400 jours) : `potentiel` = `écart de prix vers Premium` x 12 ; `probabilité` 0,05.
- Nutrition J+15 (client démarré il y a 12 à 20 jours, aucun achat nutrition) : `potentiel` = `panier nutrition moyen` du club ; `probabilité` 0,20.
- Ancien membre réactivable (voir point dédié) : `potentiel` = `prix moyen` x 6 ; `probabilité` 0,04.
- Sans mandat : `potentiel` = `mensualite` x 2 ; `probabilité` 0,5.

Écran :

- En haut, une phrase : "Aujourd'hui, 1 840 € attendus si vous traitez ces 12 actions." et la ligne "Il manque X pour le prochain palier" reprise du point Prévision si elle existe.

- Liste triée par eurosAttendus décroissants, regroupable par type, avec pour chaque ligne : titre, client, téléphone en lien tel:, montant attendu, échéance, boutons "Je m'en occupe", "Fait", "Pas maintenant" (reporte de 3 jours). Les actions écrivent dans les structures existantes (resiliations.actions, clients.dunning, S.loyalty) pour ne pas dupliquer le suivi.

- Un compteur en bas : "Réalisé aujourd'hui : 3 actions, 420 € sécurisés".

- Sur l'accueil (PAGES.home), remplacer le bloc todoList par les 5 premières opportunités de l'utilisateur.

Règles d'interface : aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin. Montants avec fmtE.

Critères d'acceptation :

- La liste est strictement triée par eurosAttendus décroissants.

- Un impayé de 200 € âgé de 5 jours (attendu 140 €) passe devant un impayé de 400 € âgé de 90 jours (attendu 40 €).

- "Pas maintenant" masque la ligne 3 jours puis elle revient.

- Une opportunité "Fait" disparaît et son montant s'ajoute au compteur du jour.

- Sans aucun import, la page affiche un état vide qui liste les exports Resamania à déposer pour l'alimenter.

- La page se calcule en moins de 300 ms sur la démo (utiliser memo()).

## 4. IMPAYÉS : LA VITESSE, C'EST DE L'ARGENT

La table des impayés (dunTable, 4002) trie par "à relancer" puis montant. Elle n'a ni tranches d'âge, ni probabilité, ni temps passé. La date de l'incident est lue à l'import (iDI, ~3341) mais seulement mise dans la clé, pas stockée : impossible de mesurer le délai incident vers régularisation. Un encours type de 12 000 € récupéré 10 points plus vite rapporte environ 1 200 € par mois, soit 12 000 à 15 000 € par an.

### PROMPT CLAUDE

Contexte : Fit Pulse, fichier HTML unique. Import Incidents (RSM\_DEFS id 'incidents'), page Impayés (dunTable, impayesAnalyse).

1. Import : stocker dans chaque S.recov[id] le champ incidentDate = rsmDate(colonne "Date de l'incident"). Pour les incidents en cours, stocker sur le client oldestIncident = la plus ancienne date d'incident ouverte. Réimporter ne doit pas créer de doublon (même id).

2. Indicateurs, onglet "Récupéré par canal" :

- Délai médian de régularisation (regularisation - incidentDate) par canal, sur le mois.

- Courbe de récupération : part des euros régularisés à J+7, J+15, J+30, J+60 après l'incident, sur les 6 derniers mois.

- Euros récupérés par heure d'appel : euros régularisés canal équipe du mois / heures d'appel. Heures d'appel = nombre d'actions de relance notées (dunning.history) x minutes par appel (réglage, défaut 4 minutes) / 60. Afficher "1 relance = 23 € récupérés en moyenne".

3. Table de suivi :

- Colonne "Âge" avec 4 tranches colorées : 0 à 7 j, 8 à 30 j, 31 à 60 j, plus de 60 j (âge = aujourd'hui - oldestIncident, sinon balanceAt).

- Colonne "Attendu" = solde x probabilité de la tranche (0,70 ; 0,50 ; 0,30 ; 0,10).

- Tri par défaut : Attendu décroissant.

- Bandeau "Encours par tranche d'âge" en 4 tuiles avec euros et nombre de dossiers.

- Alerte quand un dossier franchit J+30 sans responsable.

4. Bouton "Lien de paiement" sur chaque ligne qui copie un message SMS prêt à envoyer (texte modifiable dans Réglages, avec le montant et le prénom) et note l'action dans dunning.history.

Règles d'interface : aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- Un incident du 1er régularisé le 9 compte 8 jours de délai.
- La somme des 4 tuiles d'âge égale le "Total dû" affiché.
- Changer le réglage minutes par appel de 4 à 8 divise par 2 les euros par heure.
- Le tri par Attendu place un dossier de 150 € à 3 jours (105 €) avant un dossier de 300 € à 70 jours (30 €).

## 5. MONTÉE EN GAMME : BASIC VERS PREMIUM VERS ULTIMATE

Aucune trace de gamme dans l'appli (grep -i premium : rien). Pourtant l'offre de chaque adhérent est importée (c.offer, ~3290), et l'import exclut les "changement d'offre" (PRODUCT\_EXCLUDE, 3266) au lieu de les compter comme upsell. Si 50 % des adhérents sont en Basic et que 2 % d'entre eux passent en Premium chaque mois pour +10 €, le club gagne 160 € de MRR de plus chaque mois, soit environ 10 000 à 12 000 € la première année, cumulatif ensuite.

### PROMPT CLAUDE

Contexte : Fit Pulse, fichier HTML unique. S'appuie sur S.offers (barème offres et gammes) ; s'il n'existe pas, le créer comme décrit : { libelle, gamme, prixMensuelTTC } édité dans Mes clubs > Réglages.

1. Import Vente d'abonnements (RSM\_DEFS id 'ventes') : ne plus jeter les lignes "changement d'offre". Les compter dans une nouvelle entrée KPI 'upsell' (unit 'eur', valeur = nouveau prix - ancien prix si positif, sinon 0) attribuée au commercial initial, et ne pas les compter en contrats. L'ancien prix vient de S.clients[num].price avant mise à jour. Ajouter le KPI upsell dans DEFAULT\_KPIS (label "Montée en gamme", unit 'eur', points 300, required false) sans emoji.

2. Mix de gamme : dans le Récap du mois, un bloc "Mix des abonnements" : part Basic, Premium, Ultimate, autre, en nombre et en MRR, et variation sur 3 mois. Et la part de Premium et plus dans les contrats signés du mois, par commercial.

3. Cibles de montée en gamme (alimente la page Opportunités si elle existe, sinon un onglet dans Action Rétention) : clients actifs en gamme basic, démarrés il y a 60 à 400 jours, sans impayé, triés par ancienneté. Afficher l'écart mensuel vers la gamme supérieure et le gain sur 12 mois. Bouton "Proposé" et "Accepté" qui écrivent dans S.loyalty avec le type 'upsell'.

Règles d'interface : aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- Une ligne "Changement d'offre" de 24,99 vers 34,99 crée une entrée upsell de 10 € et aucune entrée contrats.
- Une descente de gamme crée une entrée upsell à 0 et apparaît dans un compteur "Descentes de gamme".
- La somme des parts du mix fait 100 %.
- Un client avec un solde impayé n'apparaît pas dans les cibles.

## 6. RÉACTIVATION DES ANCIENS MEMBRES

loyaltyTasks (1462) écarte explicitement les statuts "ancien" et "perdu" : ils disparaissent de l'appli. L'export Évolution clients (detail-perte, ~3466) liste chaque sortant avec son abonnement, mais seul le nombre de lignes est gardé. Les anciens sont les prospects les moins chers : ils connaissent le club. Sur 1 500 sortants des 24 derniers mois, réactiver 8 personnes par mois avec une offre de retour vaut environ 3 000 € de valeur vie par mois, 10 000 € par an en hypothèse prudente.

### PROMPT CLAUDE

Contexte : Fit Pulse, fichier HTML unique.

1. Import Évolution clients detail-perte (RSM\_DEFS id 'evolution') : en plus du compteur, stocker chaque ligne dans S.clients avec status 'ancien', endDate (date de sortie lue dans l'export, colonne contenant "date"), offer (nom de l'abonnement), num si la colonne existe. Import Clients club : accepter aussi Statut = Ancien client.
  2. Onglet "Anciens membres" dans Action Rétention :
    - Segments : sortis il y a 3 à 6 mois, 6 à 12 mois, 12 à 24 mois. Exclure ceux qui avaient un impayé non soldé et ceux qui ont déjà un nouveau contrat (rapprochement par numéro ou tokenKey du nom).
    - Priorité de contact : ancienne mensualité x (1 si 3 à 6 mois, 0,6 si 6 à 12, 0,3 si 12 à 24), décroissante.
    - Saisonnalité : afficher un bandeau "Fenêtre forte : campagne de retour conseillée" de fin décembre à fin janvier et de fin août à fin septembre.
    - Actions "Appelé", "Offre de retour envoyée", "Revenu", "Pas intéressé" dans S.loyalty, type 'reactivation'.
    - Un adhérent est compté "revenu" automatiquement si un contrat est importé à son nom dans les 90 jours suivant la première action.
  3. Indicateur dans le Récap : "Anciens revenus ce mois" et valeur vie correspondante.
  4. Export CSV du segment (nom, téléphone, e-mail, ancienne offre, date de sortie) pour une campagne SMS.
- Règles d'interface : aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin.
- Critères d'acceptation :
- Un sortant avec un impayé ouvert n'est pas listé.
  - Un sortant qui a re-signé n'est pas listé et compte "revenu" s'il avait une action dans les 90 jours.
  - Le bandeau saisonnier n'apparaît qu'aux dates prévues (tester avec une date simulée au 15 janvier et au 15 avril).

## 7. PANIER BOUTIQUE : NUTRITION À LA SIGNATURE ET À J+15

La nutrition est un total mensuel par vendeur (import factures, ~3305). Rien ne relie un achat à un nouvel adhérent : on ne sait pas quel pourcentage des signatures repart avec un produit. L'appel J+15 existe (LOYALTY\_TYPES.suivi, 1438) mais sans proposition boutique. Passer de 15 % à 30 % de signatures avec un achat de 35 € sur 110 contrats, plus un rebond J+15, rapporte environ 700 € par mois, soit 8 000 à 10 000 € par an.

### PROMPT CLAUDE

Contexte : Fit Pulse, fichier HTML unique. Imports Factures & avoirs et Lignes de factures (RSM\_DEFS ids 'factures', 'lignes-factures').

1. À l'import, stocker pour chaque ligne nutrition ou accessoire le numéro client si la colonne existe ("numero du client", "num client" ou "client"), dans un index S.shop[clubId][num] = liste { date, kpiId, montant }.

2. Indicateurs par mois et par commercial (commercial = vendeur de la signature) :
  - Taux d'attachement à la signature = nouveaux adhérents du mois ayant un achat nutrition ou accessoire entre start et start + 1 jour / nouveaux adhérents du mois.
  - Taux J+15 = nouveaux adhérents ayant un achat entre start + 2 et start + 30 jours / nouveaux adhérents dont start + 30 est passé.
  - Panier moyen boutique par acheteur, et panier moyen par adhérent actif.
 Afficher ces 3 lignes dans le Récap du mois et dans Mes objectifs, avec la moyenne club en repère.
3. Tâche J+15 : dans Action Rétention, l'appel de suivi J+15 indique "Aucun achat boutique depuis l'inscription" quand c'est le cas, et propose un bouton "Vente boutique faite" qui ouvre la saisie rapide nutrition.
4. Objectif optionnel : dans Objectifs, un KPI "Attachement nutrition" en pourcentage par commercial (cible par défaut 25 %).

Règles d'interface : aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- Un adhérent qui signe le 10 et achète une whey le 10 compte dans le taux à la signature.
- Un achat le 22 pour un adhérent démarré le 10 compte dans le taux J+15, pas dans la signature.
- Sans colonne numéro client dans l'export, l'écran affiche "Export sans numéro client : utilisez Factures & avoirs (gestion)" au lieu d'un taux à 0 %.

## 8. SAISONNALITÉ : OBJECTIFS ET EFFORTS CALÉS SUR JANVIER ET LA RENTRÉE

Les objectifs sont copiés du mois précédent (copyTargets, ~2513) et les paliers par défaut sont fixes (PALIER\_DEFAULT, 4098 : 100, 115, 130 contrats). L'historique N-1 existe pourtant (S.monthly, saisi dans Imports > Historique, ~2963, et utilisé seulement en comparaison graphique dans dashAnalyses). Un objectif plat sous-exploite janvier et septembre, qui pèsent souvent 1,4 à 1,6 fois un mois moyen, et décourage en juillet et août. Mieux répartir l'effort vaut 5 à 10 contrats de plus sur les pics, soit environ 8 000 € par an.

### PROMPT CLAUDE

Contexte : Fit Pulse, fichier HTML unique. Objectifs (memTargets, copyTargets), paliers (paliersFor, memPaliers), historique mensuel S.monthly et entrées S.entries.

1. Coefficient saisonnier par mois et par KPI (contrats, nutrition, accessoires) : coef[m] = moyenne des 2 dernières années du mois m / moyenne mensuelle de ces années. Source : S.monthly si présent, sinon sumRange. Si moins de 12 mois d'historique, utiliser les coefficients par défaut : janv 1,5 ; févr 1,1 ; mars 1,0 ; avr 0,95 ; mai 0,9 ; juin 0,8 ; juil 0,6 ; août 0,7 ; sept 1,4 ; oct 1,1 ; nov 0,95 ; déc 0,8, puis normaliser pour que la moyenne fasse 1.
2. Dans Objectifs, un bouton "Proposer les objectifs de l'année" : saisir un objectif annuel club par KPI ; l'outil répartit objectifMois = annuel x coef[m] / 12, puis par commercial au prorata de sa part des 3 derniers mois (ou part égale s'il est nouveau). Aperçu avant validation, valeurs modifiables.
3. Paliers : proposer P1, P2, P3 = objectifMois x 0,9 ; 1,0 ; 1,15 arrondis, au lieu de recopier le mois précédent.
4. Bandeau sur l'accueil 3 semaines avant janvier et septembre : "Pic de saison dans 21 jours : préparez prospects et invités." avec un lien vers l'Entonnoir.

Règles d'interface : aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- La moyenne des 12 coefficients vaut 1 à 0,01 près.
- La somme des 12 objectifs mensuels proposés égale l'objectif annuel à l'unité près.
- Rien n'est écrit dans S.targets avant le clic sur "Valider".

## 9. PRÉVISION DE FIN DE MOIS ET "CE QU'IL MANQUE" TRADUIT EN ACTIONS

La course au palier (`palierRace`, 1770) projette en ligne droite (`acc / dayNow * n`) sans tenir compte des week-ends ni de la fin de mois, qui concentre les signatures. Le message du palier (`palierBlock`, 4126) dit "Encore 12 pour le Palier 2, 1,3 par jour", sans dire comment. Il n'existe aucune prévision en euros. Bien piloter les 10 derniers jours vaut 3 à 5 contrats de plus par mois, environ 5 000 € par an.

### PROMPT CLAUDE

Contexte : Fit Pulse, fichier HTML unique. `palierRace`, `palierBlock`, `paceMessage`.

1. Prévision : pour chaque jour de la semaine (lundi à dimanche), calculer le poids moyen des 3 derniers mois terminés (part du KPI réalisée ce jour-là / part attendue en ligne droite). Prévision fin de mois = réalisé + somme sur les jours restants de (moyenne journalière du mois en cours x poids du jour de semaine). Ajouter un bonus de fin de mois mesuré : ratio des 5 derniers jours ouverts par rapport à la moyenne, sur les 3 derniers mois. Afficher la fourchette basse et haute (plus ou moins un écart type des erreurs de prévision des mois précédents au même jour).

2. Traduction en actions : quand un palier n'est pas atteint, afficher sous le palier :

"Il manque 6 contrats pour le Palier 2. Concrètement : rappeler 18 prospects chauds (taux 33 %) ou convertir 4 invités de la semaine et 2 prospects." Calculs : prospects nécessaires =  $\text{ceil}(\text{manque} / \text{taux cohorte du club})$ , invités nécessaires =  $\text{ceil}(\text{manque} / \text{taux invités})$ . Si le palier est en euros (nutrition), l'équivalent : "450 € de nutrition, soit 13 paniers moyens de 35 €" avec le panier moyen réel du mois.

Un bouton "Voir les 18 prospects" ouvre la liste filtrée (page Opportunités ou Entonnoir).

3. Équivalence multi KPI pour le score : "Pour remonter au statut À l'heure : 3 contrats ou 450 € de nutrition ou 2 sauvetages", calculé avec les points réels des KPI (`S.kpis[k].points`) et l'écart à l'objectif.

Règles d'interface : aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- Sur un mois terminé, la prévision égale le réalisé.
- Si le taux de conversion est inconnu, la phrase n'affiche pas de nombre de prospects et propose d'importer l'export Prospects.
- Le nombre de prospects proposé est un entier arrondi au supérieur.
- Testé sur la démo : la ligne de projection du graphique utilise la nouvelle prévision, pas la ligne droite.

## 10. B2B : LES ENTREPRISES DU SECTEUR

Le B2B n'est qu'un compteur déduit d'une colonne "Société du client" sur les factures (~3313), avec un avertissement "à vérifier". Pas de liste d'entreprises, pas de pipeline, pas de valeur. La tâche "Prospection entreprises du secteur" existe (`DEFAULT_TASKS`, 862) sans suivi. 2 conventions par trimestre de 10 salariés à 25 €, c'est environ 500 € de MRR en plus à chaque trimestre, soit 6 000 € et plus la première année.

## PROMPT CLAUDE

Contexte : Fit Pulse, fichier HTML unique.

1. Base entreprises : `S.companies[id] = { clubId, nom, secteur, effectif, contact, tel, email, statut ('a_contacter'|'contacte'|'rdv'|'proposition'|'signe'|'perdu'), ownerId, prochaineAction, notes, offre }`. Création manuelle et import CSV simple (nom, effectif, adresse).

2. À l'import Factures & avoirs, regrouper les adhérents par "Société du client" : pour chaque société, nombre d'adhérents actifs et MRR (somme des mensualités). Créer ou mettre à jour l'entreprise avec statut 'signe'.

3. Page "B2B" (manager et commerciaux) : pipeline en colonnes par statut, avec pour chaque carte la valeur potentielle = effectif x taux d'adhésion attendu (réglage, défaut 8 %) x prix B2B moyen x 12. Total pipeline pondéré : rdv 0,2 ; proposition 0,4. Liste "Entreprises clientes" triée par MRR, avec alerte quand une entreprise perd plus de 20 % de ses adhérents sur 3 mois.

4. Le KPI 'b2b' compte désormais une entreprise passée en 'signe' dans le mois (plus une facture), et le Récap affiche "MRR B2B".

Règles d'interface : aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- Trois factures d'une même société le même mois ne comptent qu'une entreprise.
- Le pipeline pondéré change quand une carte passe de rdv à proposition.
- Une entreprise sans effectif affiche "Effectif inconnu" et n'entre pas dans le total pondéré.

## 11. PARRAINAGE : LE CANAL LE MOINS CHER N'EST PAS MESURÉ

Aucune notion de parrain dans le code (`grep -i parrain` : rien). Le KPI "Invités > Contrats" (855) est saisi à la main, sans lien avec l'adhérent qui a invité. Le canal et l'offre importés pourraient révéler les offres "parrainage" de Resamania. Un programme suivi qui produit 5 contrats de plus par mois vaut 2 000 € de valeur vie par mois ; même en prudence, 5 000 € par an.

## PROMPT CLAUDE

Contexte : Fit Pulse, fichier HTML unique. Formulaire Invité (S.guests) et import Vente d'abonnements.

1. Formulaire Invité : champ "Invité par" (recherche dans S.clients du club). À la conversion d'un invité (rapprochement nom ou téléphone avec un nouveau client dans les 30 jours), enregistrer `client.parrainId`.

2. Détection à l'import : si le nom de l'offre ou le canal contient "parrain", marquer le contrat source 'parrainage' même sans invité saisi.

3. Indicateurs dans le Récap et l'Entonnoir : contrats issus du parrainage, part dans les contrats du mois, valeur vie apportée, et "Top parrains" (adhérents ayant apporté 2 contrats ou plus sur 12 mois), avec rappel de la récompense à remettre si un réglage "Récompense parrain" est rempli.

4. Opportunité quotidienne : adhérents ayant un avis Google récent ou plus de 6 mois d'ancienneté sans impayé, à solliciter pour un parrainage, limités à 5 par jour.

Règles d'interface : aucun emoji décoratif, aucun tiret cadratin ni demi-cadratin.

Critères d'acceptation :

- Un invité saisi le 3 par l'adhérent A, qui signe le 12, porte `parrainId = A`.
- Un contrat dont l'offre contient "Parrainage" compte en source parrainage sans invité saisi.

- Le top parrains n'affiche que les adhérents à 2 contrats ou plus.

## ***ORDRE DE MISE EN OEUVRE CONSEILLÉ***

---

1. Point 0 (socle valeur), sinon aucun euro n'est fiable.
2. Points 1 et 2 (entonnoir et sauvetage en euros) : les deux plus gros gisements.
3. Point 3 (page Opportunités), qui branche les suivants au fur et à mesure.
4. Points 4, 5, 7, puis 6, 8, 9, 10, 11.

## 8. TECHNIQUE, SÉCURITÉ, PERFORMANCE ET RGPD

Fichier audité : `fitpulse.html` (4 475 lignes). Les numéros de ligne renvoient à ce fichier.

Verdict en une phrase : le code est propre et lisible, mais toute la sécurité vit dans le navigateur. En mode Firebase, un simple membre peut lire, modifier ou effacer toute la base, y compris les données des adhérents. Les points 1 à 5 sont à traiter avant de charger de vraies données Resamania sur une base partagée.

Ordre conseillé : 1, 4, 3, 2, 5, puis le reste. Le point 14 donne le plan de migration complet.

### 1. CRITIQUE : UN MEMBRE PEUT LIRE ET ÉCRIRE TOUT L'ARBRE /PULSE

Constat : `firebaseBackend.attach` (l. 942) écoute `ref('pulse')` en entier. `write`, `batch` et `replaceAll` (l. 956 à 958, 975) écrivent n'importe quel chemin. Les rôles ne sont vérifiés que dans l'interface (`isCreator`, `isManager`, l. 1503 et 1504). Le fichier de règles cité l. 712 (`club/database.rules.pulse.json`) n'est pas livré. Le commentaire l. 713 laisse entendre que tout membre de `/pulse/team` a les droits complets. Risque : depuis la console du navigateur, un membre peut écrire `users/<son id>/role = "createur"`, ajouter une adresse dans `/pulse/team`, modifier ses saisies pour gonfler ses primes ou faire `ref('pulse').set(null)`. Autre faille : l'inscription (l. 954) ne vérifie pas l'adresse e-mail. Quelqu'un peut donc s'inscrire le premier avec l'adresse d'un invité et prendre son compte.

#### PROMPT CLAUDE :

Contexte : application Fit Pulse, fichier unique `fitpulse.html`, mode Firebase Realtime Database sous `/pulse`.

Aujourd'hui, tout utilisateur connecté de `/pulse/team` lit et écrit tout l'arbre. Les rôles ne sont vérifiés que côté client.

Objectif : durcir les droits SANS changer la structure des données (étape 0 d'une migration).

1. Produis `database.rules.json` complet, avec ces règles :

- Aucun `".write"` au niveau `/pulse`. Dans RTDB, un droit accordé sur un parent ne peut pas être retiré plus bas. Les écritures se déclarent donc collection par collection.
- `".read"` sur `/pulse` : `auth != null && auth.token.email_verified === true && root.child('pulse/team/' + auth.token.email.toLowerCase().replace('.', ''))val() === true`
- Nouveaux nœuds `/pulse/admins/{emailKey}: true` et `/pulse/managers/{emailKey}: true`. Écriture : jamais depuis le client (`".write": false`). On les remplit à la main dans la console Firebase.
- `team`, `admins`, `managers`, `clubs`, `kpis`, `meta` : écriture réservée aux admins.
- `users/{id}` : écriture par un admin. Un manager peut écrire seulement si `newData.role === 'membre'` et que l'ancien rôle était absent ou 'membre'. Pour le compte lui-même (`data.child('email').val() === auth.token.email.toLowerCase()`) : écriture permise seulement sur `first`, `last`, `avatar`, et sur `status` de 'pending' vers 'active'. `role`, `clubs`, `salt`, `codeHash` : admin, ou manager pour un membre.
- `entries/{id}` : admin ou manager. Un membre peut écrire si `root.child('pulse/users/' + newData.child('userId').val() + '/email').val() === auth.token.email.toLowerCase()`. Même test sur `data` pour une suppression. Champs à valider : `value` nombre entre -100000 et 100000, `date` au format AAAA-MM-JJ (regex), `kpiId` chaîne qui existe dans `/pulse/kpis`.
- `targets`, `imports`, `monthly`, `base`, `rsm`, `paliers`, `challenges`, `tasks/library`, `tasks/plan` : admin ou manager.
- `clients`, `loyalty`, `resiliations`, `recov`, `chat`, `reactions`, `tasks/done`, `prefs` : tout membre de `team`, avec validation des tailles (texte de chat 2000 caractères au plus, `image` : chaîne qui commence par `'data:image/jpeg;base64,'` et fait moins de 400000 caractères). Suppression d'un client : admin ou manager seulement.
- `"$other": { ".validate": false }` sous `/pulse` pour refuser toute collection inconnue.

2. Produis `firebase.json` avec `"database": { "rules": "database.rules.json" }`.

3. Dans `fitpulse.html` :

- `signUp` (l. 954) : après la création, appelle `user.sendEmailVerification()`. Tant que `emailVerified` est `false`, affiche un écran « Vérifiez votre e-mail » avec un bouton « J'ai validé » qui appelle `user.reload()`.
- En mode `firebase`, masque les boutons qui appellent `db.replace` ou `backend.wipe` (`loadDemo`, `askDemo`, `resetAll`, `importAll`). Les règles les refuseront de toute façon.
- Dans `attach`, en cas d'erreur de lecture, n'affiche jamais `PAGES.onboarding` : affiche un écran « Accès refusé » avec un bouton de déconnexion.

4. Produis `tests/rules.test.js` avec `@firebase/rules-unit-testing` et l'émulateur.

Critères d'acceptation vérifiables (dans les tests) :

- Un membre ne peut pas écrire `/pulse/users/<son id>/role`, `/pulse/team/x`, `/pulse/kpis/avis/points`, ni `/pulse` (`set null`).
- Un membre peut créer une saisie à son nom et ne peut pas en créer une au nom d'un collègue.
- Un compte dont l'e-mail n'est pas vérifié ne lit rien.
- Un manager peut créer un membre mais pas un manager.
- La commande `firebase emulators:exec "npm vitest run tests/rules.test.js"` passe.

## 2. CRITIQUE : EN MODE LOCAL, LA CONNEXION SE CONTOURNE EN UNE LIGNE

Constat : au démarrage (l. 4459 à 4461), la session est rétablie avec le seul identifiant stocké dans `localStorage['parkpulse.session']`. Le code n'est jamais redemandé. Les identifiants des comptes créateurs sont écrits en clair l. 731 et 732 (`kg-createur`). Les profils sans code se connectent d'un clic (`loginAs`, l. 1917). Risque : sur le PC de l'accueil, n'importe qui tape `localStorage['parkpulse.session']='kg-createur'`, recharge la page et devient créateur. Il voit alors tous les adhérents, téléphones et impayés, stockés en clair dans le navigateur.

**PROMPT CLAUDE** :

Contexte : `fitpulse.html`, mode local (`window.PARKPULSE_FIREBASE = null`). La session se restaure depuis `localStorage['parkpulse.session']` sans aucune preuve (l. 4459 à 4461). Les données des adhérents sont en clair dans `localStorage['parkpulse.v1']`.

Changements :

1. Le mode local devient un mode DÉMO :
  - Bandeau permanent « Mode démo : données fictives, ne pas importer de fichiers réels ».
  - Imports `Resamania`, import « Résumé clients » et saisie manuelle de clients bloqués en mode local, avec un message qui renvoie vers le mode partagé. Ajoute une constante `LOCAL_REAL_DATA = false` pour pouvoir lever ce blocage en connaissance de cause.
2. Session : remplace `localStorage` par `sessionStorage` pour `SESSION_KEY`. Ajoute une expiration (8 h) et une déconnexion après 30 min d'inactivité. Stocke `{ id, exp }` et vérifie `exp` au démarrage.
3. Supprime `loginAs` pour tout utilisateur qui a un `codeHash`. Garde-le pour les profils démo seulement si `S.meta.demo === true`.
4. Retire les identifiants et les e-mails réels de `PARKPULSE_ACCOUNTS` dans le fichier publié. Ils passent dans un `config.local.js` non versionné. Fournis un `config.example.js`.
5. Ajoute un bouton « Effacer les données de ce navigateur » visible sur l'écran de connexion.

Critères d'acceptation :

- Écrire `localStorage['parkpulse.session']='kg-createur'` puis recharger affiche l'écran de connexion.
- Après fermeture de l'onglet, le code est redemandé.
- En mode local, le bouton d'import Resamania est désactivé et affiche le message.
- `grep "guellec" fitpulse.html` ne renvoie rien.

### 3. CRITIQUE : XSS STOCKÉE VIA EMOJI DE KPI, IMAGE ET RÉACTIONS DU CHAT

Constat : `esc()` (l. 761) est bien utilisé sur les noms et les textes du chat. Mais plusieurs champs passent bruts dans le HTML : `k.emoji` (l. 2021, 2112, 2131) et `t.icon` dans les trophées (l. 2200, 2618, 4199). `m.image` est injecté dans `src="..."` (l. 2260). La clé de réaction `em` est injectée dans `data-em` et dans le texte (l. 2261). Le fichier n'a aucune Content Security Policy. Risque : avec le point 1, un membre écrit `chat/x/image = 'x' onerror="..."` ou un emoji piégé. Le script s'exécute chez le créateur, qui ouvre le chat, et agit avec ses droits : rôle, export de la base, suppression.

#### PROMPT CLAUDE :

Contexte : `fitpulse.html` rend tout en chaînes HTML avec `esc()` (l. 761). Certains champs stockés en base ne sont pas échappés.

Changements :

1. Échappe systématiquement avec `esc()` :

- `k.emoji` partout (l. 2021, 2112, 2131 et toute autre occurrence) ;
- `t.icon` et `x.icon` (l. 2200, 2618, 4199) ;
- `em`, la clé de réaction, dans le texte ET dans `data-em` (l. 2261) ;
- tout `${...}` qui affiche un champ venu de S. Cherche-les avec la regex `\${[^\}]*\}`.

(`name|label|title|emoji|icon|client|note|reason|offer|reward|cat|text`)\b et liste chaque occurrence corrigée.

2. Image du chat : n'accepte `m.image` que si `/^data:image\/(jpeg|png|webp);base64,[A-Za-z0-9+\/=]+$/` le valide. Sinon, n'affiche rien.

3. Emoji de KPI : liste blanche d'une quarantaine d'emojis dans un sélecteur. Rejette toute autre valeur dans `kpiSet` et `kpiCreate`.

4. Réactions : liste blanche de 5 emojis, écrits en échappements Unicode (`\u{1F44D}` pouce, `\u{1F525}` flamme, `\u{1F4AA}` biceps, `\u{1F44F}` applaudissements, `\u{2764}\u{FE0F}` coeur), côté client et dans les règles (`.validate`).

5. Ajoute dans `<head>` :

```
<meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self'
https://www.gstatic.com https://cdnjs.cloudflare.com 'unsafe-inline'; connect-src 'self'
https://*.firebaseio.com wss://*.firebaseio.com https://*.firebase.database.app
wss://*.firebase.database.app https://identitytoolkit.googleapis.com https://securetoken.googleapis.com;
img-src 'self' data: blob:; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src
'self' https://fonts.gstatic.com; object-src 'none'; base-uri 'none'">
```

Note dans un commentaire que `'unsafe-inline'` sur `script-src` disparaîtra quand le JS sortira du HTML (voir le point découpage en modules).

6. Ajoute `tests/xss.test.js` (Vitest + jsdom). Il injecte dans S les charges `'<img src=x onerror=alert(1)>'`, `'<svg onload=alert(1)>'`, `'x' onerror="alert(1)'` dans `clients.name`, `chat.text`, `chat.image`, `kpis.emoji`, `challenges.title`, `resiliations.client`, puis rend chaque page.

Critères d'acceptation :

- Le test vérifie qu'aucun élément rendu n'a d'attribut `on*` et qu'aucune balise `<script>`, `<svg>` ou `<img>` non prévue n'apparaît.

- Le chat affiche les charges comme du texte.
- La console du navigateur ne signale aucune violation CSP sur un parcours normal (connexion, saisie, import, chat).

## 4. CRITIQUE : REMPLACEMENTS COMPLETS ET RÉÉCRITURES DE TABLEAUX QUI ÉCRASENT LE TRAVAIL DES COLLÈGUES

Constat : `db.replace` appelle `ref('pulse').set(5)` (l. 957, 978). Il est déclenché par la restauration (l. 3117), la démo (l. 1870, 3121) et l'onboarding (l. 1866). La restauration ne valide rien à part la présence de `users` et `clubs` (l. 3115). L'historique d'une résiliation est un tableau réécrit en entier (`resLog`, l. 3918). Il en va de même pour le planning (l. 2590), les paliers (l. 4302 à 4304) et les fiches clients à l'import (l. 3759). Risque : un créateur qui clique « Charger la démo » ou restaure un vieux JSON efface la production de 10 clubs. Une sauvegarde modifiée peut aussi injecter un compte créateur. Enfin, deux vendeurs qui notent une action sur le même dossier en même temps : le dernier écrase l'autre.

### PROMPT CLAUDE :

Contexte : `fitpulse.html`. `db.replace` (l. 978) réécrit tout `/pulse`. `resLog` (l. 3918) réécrit le tableau actions d'une résiliation. `dunPatch` (l. 4047) et l'import (l. 3759) réécrivent des fiches clients entières.

Changements :

1. Interdis `db.replace` et `backend.wipe` en mode firebase : lève une erreur explicite si on les appelle. Les règles du point 1 les bloquent aussi.
2. Restauration (`ACTIONS.importAll`, l. 3110) :
  - Valide le JSON avec une fonction `validateBackup(data)` écrite à la main, sans bibliothèque. Elle vérifie les collections connues, le type de chaque champ, et refuse tout utilisateur 'createur' absent de l'état actuel.
  - Mode « fusion par club » : l'utilisateur choisit un club. On écrit seulement les chemins de ce club, par lots de 500 via `db.batch`.
  - Avant toute restauration, exporte automatiquement l'état courant (`fit-pulse-avant-restauration-<date>.json`).
  - Pour confirmer, il faut taper le nom du club.
3. Historiques en ajout seul : remplace `resLog` par une écriture sous `resiliations/<id>/log/<newId()> = {at, by, label, ...}`. Adapte l'affichage (tri par at). Fais de même pour `dunning.history` si ce champ existe. Le code doit lire l'ancien tableau actions s'il est présent (compatibilité).
4. Fiches clients : à l'import, écris champ par champ (`['clients', id, 'balance']`, etc.), jamais l'objet entier. Ne touche jamais `dunning.ownerId`, `dunning.note` ni `dunning.next` s'ils existent déjà.
5. Planning et paliers : écris `tasks/plan/<club>/<id>` élément par élément. Pas de set du nœud parent.

Critères d'acceptation :

- `grep "ref('pulse').set" fitpulse.html` ne renvoie que du code protégé par un test de mode local.
- Test : deux appels concurrents à l'ajout d'une action sur la même résiliation produisent deux entrées dans log.
- Test : réimporter un fichier d'impayés ne change pas le responsable ni la note d'un dossier déjà suivi.
- Une sauvegarde contenant `users.hacker.role = 'createur'` est refusée avec un message clair.

## 5. CRITIQUE : RGPD, AUCUNE BASE DOCUMENTÉE, AUCUNE DURÉE DE CONSERVATION, AUCUN EFFACEMENT

Constat : l'application stocke nom, téléphone, e-mail, date de naissance complète, offre, impayés et motifs de résiliation (l. 2893, 3759, 3969, 4079). Aucun code ne supprime un client (aucun `['clients', id], null`), ni un vieux message de chat, ni un vieux import. Les exports CSV (l. 3975, 4085) et la sauvegarde JSON (l. 3109) sortent toute la base en clair. Le fichier ne contient ni mention d'information, ni registre, ni journal d'accès. Risque : en cas de contrôle de la CNIL ou de plainte d'un adhérent relancé pour un impayé, le club ne peut prouver ni la base légale, ni la durée de conservation, ni l'effacement. Les données d'impayés sont sensibles pour la réputation de la personne.

### PROMPT CLAUDE :

Contexte : fitpulse.html stocke des données d'adhérents (clients, loyalty, resiliations, recov, chat) importées depuis Resamania. Rien n'est jamais purgé.

Changements :

#### 1. Minimisation :

- clients.birth ne garde que MM-JJ (champ birthMD). Migration : convertis les valeurs existantes au chargement et supprime l'année.
- Ne stocke pas l'e-mail des adhérents si aucune fonction ne l'utilise (vérifie avec grep). Sinon, documente l'usage.
- Chat : interdis les photos de documents. Ajoute un avertissement sous le bouton trombone : « Pas de pièce d'identité, de RIB ni de données d'adhérent ».

2. Conservation : crée la constante RETENTION = { clientInactifMois: 36, impayeSoldeMois: 24, resiliationMois: 24, chatMois: 12, importsMois: 13 }. Ajoute une fonction purgeOld(), lancée par le créateur depuis Réglages, qui affiche le nombre d'éléments à supprimer avant confirmation. En cible, elle tournera dans une Cloud Function planifiée chaque nuit.

3. Droit d'effacement : dans la fiche client, bouton « Effacer cet adhérent » (manager). Il supprime clients/<id> et toutes les entrées liées (loyalty, resiliations, recov où clientNum ou clientId correspond). Il écrit une trace sans donnée personnelle dans audit/<id> = {at, by, action:'erase', hash: sha256(club+num)}. Un import ultérieur du même numéro affiche un avertissement.

4. Information : page « Données personnelles », accessible depuis le profil. Elle indique : responsable de traitement (la société exploitante du club), finalités (suivi commercial, relances de fidélisation, recouvrement amiable), base légale (intérêt légitime pour le suivi, exécution du contrat pour les impayés), destinataires (équipe du club), sous-traitant (Google Firebase, région europe-west1), durées, et contact pour exercer ses droits.

5. Exports : chaque export CSV ou JSON écrit audit/<id> = {at, by, type, club, lignes}. Le nom de fichier porte la mention CONFIDENTIEL.

6. Produis docs/rgpd/registre-traitement.md : fiche de registre article 30 pour le traitement « Pilotage commercial et fidélisation Fit Pulse » (catégories de personnes et de données, durées, mesures de sécurité, sous-traitants). Produis aussi docs/rgpd/analyse-risques.md (une page : risques, mesures, risque résiduel) pour décider si une AIPD complète est nécessaire.

Critères d'acceptation :

- Après effacement d'un adhérent, aucune occurrence de son nom ou de son numéro dans JSON.stringify(S), à part le hash dans audit.
- purgeOld() sur la démo supprime les messages de plus de 12 mois et laisse les autres.
- Aucune date de naissance au format AAAA-MM-JJ dans S après chargement.
- Les deux fichiers docs/rgpd existent, remplis avec les valeurs du code.

## 6. IMPORTANT : DÉPENDANCES TIERCES NON MAÎTRISÉES (SHEETJS VULNÉRABLE, CDN SANS SRI, GOOGLE FONTS)

Constat : `xlsx 0.18.5` est chargé depuis `cdnjs` (l. 3138). Cette version est touchée par CVE-2023-30533 (prototype pollution, corrigée en 0.19.3) et par CVE-2024-22363 (ReDoS, corrigée en 0.20.2). `cdnjs` ne publie plus les versions récentes. Les scripts sont injectés sans `integrity` (l. 932, 3142). Google Fonts est appelé à chaque ouverture (l. 3 à 5) et transmet l'adresse IP à Google. Risque : un fichier XLSX piégé, reçu par mail et importé par un manager, peut corrompre l'application. Un CDN compromis exécute son code avec accès à toute la base. Google Fonts chargé à distance a déjà été jugé contraire au RGPD en Allemagne (LG München, 2022).

### PROMPT CLAUDE :

Contexte : `fitpulse.html` charge `jszip 3.10.1` et `xlsx 0.18.5` depuis `cdnjs` (l. 3137 et 3138), le SDK Firebase 10.12.2 depuis `gstatic` (l. 932) et les polices depuis Google Fonts (l. 3 à 5).

Changements :

- Crée un dossier `vendor/` :
  - `xlsx 0.20.3` téléchargé depuis `https://cdn.sheetjs.com/xlsx-0.20.3/package/dist/xlsx.full.min.js` ;
  - `jszip 3.10.1` ;
  - les trois fichiers compat Firebase 10.12.2.Ajoute un script `tools/vendor.sh` qui les télécharge et affiche leur empreinte `sha384`.
- Dans `loadLib` et `loadSdk`, charge depuis `vendor/` avec `s.integrity = 'sha384-...'` et `s.crossOrigin = 'anonymous'`. Range les empreintes dans un objet `LIB_SRI`.
- Polices : télécharge Barlow Condensed (600, 700, 800, 900 italique) et Montserrat (400 à 800) en `woff2` dans `fonts/`. Déclare-les avec `@font-face` et `font-display: swap`. Supprime les trois balises Google Fonts.
- Import XLSX : appelle `XLSX.read` avec `{ dense: true, cellFormula: false, cellHTML: false }`. Refuse les fichiers de plus de 20 Mo et les ZIP qui se décompressent au-delà de 200 Mo (bombe ZIP).
- Mets à jour la CSP du point 3 : retire `cdnjs`, `gstatic` et `fonts.googleapis.com`.

Critères d'acceptation :

- L'onglet Réseau de Chrome ne montre aucune requête vers `fonts.googleapis.com`, `fonts.gstatic.com` ni `cdnjs.cloudflare.com`.
- Modifier un octet de `vendor/xlsx.full.min.js` fait échouer le chargement avec le message « Bibliothèque indisponible ».
- Les imports Resamania de démonstration (CSV, ZIP, XLSX) donnent les mêmes résultats qu'avant (comparer le résumé `summary`).

## 7. IMPORTANT : INJECTION DE FORMULES DANS LES EXPORTS CSV

Constat : les exports résiliations (l. 3975) et impayés (l. 4085) mettent les valeurs entre guillemets mais ne neutralisent pas les cellules qui commencent par `=`, `+`, `-`, `@`, tabulation ou retour chariot. Les noms des clients viennent des fichiers Resamania et de la saisie libre. Risque : un nom saisi `=HYPERLINK("http://..."; "Payer")` devient une formule active quand le manager ouvre le CSV dans Excel. Le fichier contient aussi les téléphones.

### PROMPT CLAUDE :

Contexte : `fitpulse.html` génère deux CSV (l. 3974 à 3976 et 4084 à 4086) avec `.map(x => `${String(x).replace(/"/g, '""')}`)`.

Changements :

1. Crée une fonction unique `csvCell(x)` : convertit en chaîne. Si la valeur commence par `= + - @ \t` ou `\r`, préfixe-la d'une apostrophe (`'`). Échappe les guillemets, puis entoure de guillemets.  
Exception : les montants numériques négatifs déjà formatés par le code (vérifie avec `/^-?\d+([,]\d+)?$/`) ne sont pas préfixés.

2. Crée une fonction `toCsv(headers, rows)` qui utilise `csvCell` et le BOM UTF-8. Remplace les deux exports par des appels à `toCsv`.

3. Test Vitest : `toCsv` sur `['=1+1', '+33 6', '-12,50', '@SUM(A1)', 'Dupont "Jo"']`.

Critères d'acceptation :

- Le test produit `"'=1+1";'+33 6';"-12,50";"@SUM(A1)";"Dupont ""Jo"""`.
- Le CSV d'importés ouvert dans Excel n'affiche aucune formule évaluée.

## 8. IMPORTANT : MONTÉE EN CHARGE, ARBRE UNIQUE TÉLÉCHARGÉ EN ENTIER, IMPORT DE 50 000 LIGNES EN UNE SEULE ÉCRITURE

Constat : `root.on('value')` (l. 945) synchronise tout `/pulse`. À chaque changement, `snap.val()` reconstruit l'objet entier, `normalizeState` le parcourt, `REV++` invalide l'index et `idx()` (l. 1190) reparcourt toutes les saisies. L'import Resamania pousse toutes les lignes dans un seul `db.batch(ops)` (l. 3760). Les photos du chat sont stockées en base64 dans l'état (l. 2297). Risque : à 10 clubs, avec 50 000 saisies et 30 000 clients, chaque saisie d'un collègue fait télécharger et recalculer plusieurs Mo sur chaque téléphone. RTDB refuse une écriture de plus de 16 Mo. En mode local, le quota de 5 Mo de `localStorage` est vite atteint : `flush` (l. 920) échoue et ne laisse qu'un toast.

### PROMPT CLAUDE :

Contexte : `fitpulse.html` écoute tout `/pulse` et recalcule tout à chaque événement. L'import Resamania écrit tout en un seul `update`.

Changements, sans changer encore la structure des données :

- Écritures par lots : `db.batch` découpe `ops` en paquets de 500 chemins. Envoie-les en séquence avec `await`, affiche une barre de progression « Import : 3 000 / 48 000 », et reprend au dernier lot réussi en cas d'erreur. Ajoute `importId` à chaque lot pour pouvoir annuler.
- Écoute ciblée : remplace `root.on('value')` par des écoutes `child_added`, `child_changed` et `child_removed` sur chaque collection. Applique les deltas dans `S` avec `setPath`. Les collections lourdes (`entries`, `clients`, `recov`, `loyalty`, `chat`) passent en requêtes bornées :
  - `entries` : `orderByChild('date').startAt(<1er jour du mois M-13>)` ;
  - `chat` : `limitToLast(200)`.
- Index incrémental : `idx()` ne se reconstruit plus en entier. `db.set` et les deltas Firebase mettent à jour `IDX.day` pour la seule saisie concernée.
- Photos du chat : upload dans Firebase Storage sous `chat/<club>/<id>.jpg`, et seule l'URL est stockée. En mode local, refuse les photos.
- Mode local : avant d'écrire, mesure `JSON.stringify(S).length`. Au-delà de 4 Mo, bloque l'action avec un message clair au lieu d'échouer en silence.
- Ajoute `tools/seed-large.js` : il génère un état de 10 clubs, 50 000 `entries`, 30 000 clients, 5 000 messages.

Critères d'acceptation, mesurés avec `tools/seed-large.js` et l'émulateur :

- Une saisie d'un collègue déclenche moins de 50 Ko de trafic sur les autres postes (onglet Réseau, trames WebSocket).

- Le temps entre l'événement et l'affichage reste sous 100 ms sur un téléphone milieu de gamme (profil CPU x4 dans Chrome).
- L'import d'un fichier de 50 000 lignes aboutit, sans erreur « write too large ».

## 9. IMPORTANT : RE-RENDU COMPLET DE LA PAGE À CHAQUE FRAPPE ET À CHAQUE SAISIE

Constat : les champs de recherche `histQ`, `loyQ` et `dunQ` (l. 2482, 3014, 4046) appellent `render()` à chaque touche. `renderNow` (l. 1696) remplace tout `#app` par `innerHTML`, puis restaure le focus avec un bricolage (l. 1701). En mode Firebase, chaque `db.set` provoque deux rendus : le local, puis l'écho du serveur (l. 949). Risque : sur la liste des impayés ou de la fidélisation, avec quelques milliers d'adhérents, la frappe devient saccadée sur téléphone. La position du curseur et l'état des menus ouverts se perdent.

### PROMPT CLAUDE :

Contexte : `fitpulse.html`. `renderNow` (l. 1680) réécrit tout `#app`. Les recherches `dunQ`, `loyQ` et `histQ` appellent `render()` à chaque frappe.

Changements :

1. Ajoute une fonction `debounce(fn, ms)` et utilise-la (200 ms) pour `histQ`, `loyQ` et `dunQ`.
2. Rendu partiel : chaque page qui a une recherche expose `renderList()`. Elle ne remplace que le conteneur de la liste (`id="list-<page>"`), sans toucher au champ de saisie. `ACTIONS.dunQ`, `loyQ` et `histQ` appellent `renderList()` au lieu de `render()`.
3. Listes longues : affiche 100 lignes puis un bouton « Afficher 100 de plus ». Le filtre s'applique avant la pagination.
4. Écho Firebase : dans `attach`, si le delta reçu est identique à l'écriture locale récente (même chemin, même valeur, moins de 2 s), n'appelle pas les listeners.
5. Instrumente `renderNow` avec `performance.mark` et `performance.measure`. Si l'URL contient `?perf`, affiche la durée du dernier rendu dans un coin.

Critères d'acceptation :

- Avec 5 000 clients (`tools/seed-large.js`), taper 10 caractères dans la recherche des impayés ne déclenche pas plus de 2 rendus de liste et aucun rendu complet.
- Chaque rendu de liste dure moins de 30 ms (mesure `?perf`, CPU x4).
- Le curseur reste en place dans le champ pendant la frappe.

## 10. IMPORTANT : AUCUN FONCTIONNEMENT HORS LIGNE, ALORS QUE LA SAISIE SE FAIT AU CLUB

Constat : aucun service worker ni manifeste. Le SDK Firebase est téléchargé à chaque démarrage depuis `gstatic` (l. 932). Sans réseau, `boot` tombe sur « Connexion impossible » (l. 4453). La file d'attente hors ligne de RTDB est en mémoire seulement : une saisie faite hors réseau est perdue si l'onglet se recharge. Risque : au sous-sol d'un club ou avec un Wi-Fi saturé, le vendeur saisit un contrat, ferme l'application, et la saisie disparaît. Les primes deviennent contestables.

### PROMPT CLAUDE :

Contexte : fitpulse.html, sans service worker. SDK Firebase chargé à chaque démarrage. Aucune persistance des écritures hors ligne.

Changements :

1. Produis manifest.webmanifest (nom Fit Pulse, icônes 192 et 512, display standalone, couleurs #0A0A0A et #FFD600). Lie-le dans <head>.

2. Produis sw.js : met en cache l'app shell (index.html, vendor/\*, fonts/\*, assets/\*) à l'installation. Stratégie stale-while-revalidate pour ces fichiers. Ne met JAMAIS en cache les réponses Firebase. Versionne le cache avec une constante CACHE\_VERSION.

3. File d'écritures persistante : crée outbox.js, qui stocke dans IndexedDB les opérations {id, path, value, at} avant envoi. Au retour du réseau (événement online et .info/connected à true), il les rejoue dans l'ordre puis les supprime. Applique-la aux saisies (entries), aux actions de fidélisation, aux résiliations et aux impayés.

4. Indicateur dans l'en-tête : « Hors ligne : 3 saisies en attente », qui disparaît quand la file est vide.

5. Démarrage hors ligne : si le SDK ou le réseau manque, affiche la dernière copie connue de S, gardée dans IndexedDB et limitée au club courant et au mois en cours, en lecture seule, et autorise la saisie via l'outbox.

Critères d'acceptation :

- Dans Chrome, mode « Offline » : l'application démarre, une saisie est acceptée, l'indicateur affiche 1 en attente.
- Après rechargement toujours hors ligne, l'indicateur affiche encore 1.
- Après retour du réseau, la saisie apparaît sur un second poste en moins de 5 s et l'indicateur disparaît.
- Lighthouse, catégorie PWA : installable.

## 11. IMPORTANT : FICHIER UNIQUE DE 565 KO, TROIS COUCHES DE CSS, CONFIGURATION ÉCRASÉE

Constat : HTML, CSS et JS tiennent dans un seul fichier. Le CSS empile trois chartes : l. 6 à 325, puis « identité Fitness Park » l. 326, puis « v3 » l. 464 à 699, qui redéfinissent les mêmes sélecteurs. Une image base64 de 163 000 caractères est ajoutée l. 4446, et `window.PARKPULSE_ASSETS` est réaffecté à cet endroit : la configuration de la l. 739 est ignorée. La configuration (Firebase, comptes) est mêlée au code. Risque : chaque correction demande de chercher trois définitions concurrentes. On ne peut pas changer la configuration d'un club sans éditer le code. Toute modification fait retélécharger 565 Ko.

**PROMPT CLAUDE** :

Contexte : fitpulse.html, 4 475 lignes, contient tout. Objectif : découper sans changer le comportement, avec un build léger.

Changements :

1. Initialise un projet Vite (vanilla, sans framework) :  
src/main.js, src/config.js, src/store/db.js (S, setPath, db, backends), src/store/firebase.js, src/store/local.js,  
src/core/utils.js (esc, dates, formats), src/core/calc.js (idx, scores, paliers, l. 1169 à 1500 environ),  
src/ui/shell.js, src/ui/modal.js, src/pages/<une page par fichier>.js, src/import/resamania.js, src/import/files.js,

src/styles/tokens.css, base.css, components.css, pages.css.

2. CSS : fusionne les trois couches en UNE seule. Garde les variables de la couche v3 (l. 471 à 480) comme référence dans tokens.css. Supprime toute règle redéfinie plus loin. Produis un rapport tools/css-diff.txt qui liste les sélecteurs fusionnés.
3. Images : sors l'image base64 de la l. 4446 vers assets/fitpulse-logo.png. Supprime la réaffectation de window.PARKPULSE\_ASSETS.
4. Configuration : public/config.js, chargé avant l'application, porte PARKPULSE\_FIREBASE, PARKPULSE\_CLUB et PARKPULSE\_ACCOUNTS. Il n'est pas versionné. Fournis public/config.example.js.
5. Build : npm run build produit dist/ (index.html, JS et CSS hachés). Ajoute aussi npm run build:single, avec vite-plugin-singlefile, qui produit un dist-single/fitpulse.html autonome, pour garder le mode « un seul fichier » le temps de la transition.
6. Ajoute ESLint (eslint:recommended, no-undef, no-unused-vars) et Prettier.

Critères d'acceptation :

- npm run build passe sans avertissement. dist/assets/\*.js pèse moins de 150 Ko gzip au total.
- Captures d'écran Playwright des pages accueil, tableau de bord, classement, impayés et imports : avant et après, différence de pixels inférieure à 1 %.
- npx eslint src renvoie 0 erreur.
- Changer PARKPULSE\_CLUB dans public/config.js change le club affiché sans rebuild.

## 12. IMPORTANT : AUCUN TEST, AUCUNE CI, DÉPLOIEMENT MANUEL

Constat : le fichier ne contient aucun test. Les règles de calcul (paliers 25/50/75/100, plafond 150 %, rythme, l. 1172 à 1186) et le moteur d'import Resamania (l. 3124 et suivantes) touchent directement aux primes. Le fichier de règles Firebase n'est pas versionné avec le code. Risque : une régression sur le calcul des points ou sur la reconnaissance d'un export Resamania fausse les classements et les primes sans que personne ne s'en aperçoive. Un déploiement de règles à la main peut ouvrir la base.

### PROMPT CLAUDE :

Contexte : Fit Pulse après le découpage en modules (point 11). Aucun test aujourd'hui.

Changements :

1. Vitest :
  - tests/calc.test.js : paliers (59 % donne 25 %, 60 % reste à 25 %, 75 % donne 75 %), plafond SCORE\_CAP à 150 %, statut selon le rythme (105 % et 95 %), égalités au classement, objectif club = somme des seuls membres actifs, saisie d'un import annulé ignorée.
  - tests/import.test.js : un fixture anonymisé par type d'export Resamania (CSV UTF-8 avec « ; », ZIP ISO-8859-15) dans tests/fixtures/. Vérifie la détection par colonnes, la clé stable (deux imports donnent 0 doublon) et les montants.
2. Tests de règles (point 1) avec l'émulateur.
3. Playwright : tests/e2e/smoke.spec.js. Connexion, saisie d'un contrat, apparition dans le classement, import d'un fixture, déconnexion.
4. GitHub Actions .github/workflows/ci.yml : sur chaque PR, lint, vitest, tests de règles (firebase emulators:exec), Playwright. Sur main : firebase deploy --only hosting,database. Les PR sont déployées sur un canal de prévisualisation (hosting:channel:deploy pr-<n>, expiration 7 jours).
5. Deux projets Firebase : fitpulse-staging et fitpulse-prod, déclarés dans .firebase. La prod ne se déploie que depuis un tag v\*.

Critères d'acceptation :

- npm test passe en local et en CI.
- Une PR qui passe SCORE\_CAP à 2 fait échouer la CI.
- Une PR qui met ".write": true sous /pulse fait échouer les tests de règles.
- Couverture de src/core/calc.js et src/import/resamania.js supérieure à 80 %.

## 13. CONFORT : ERREURS SILENCIEUSES, AUCUN JOURNAL D'AUDIT, AUCUNE SUPERVISION

Constat : `safeLS.set` renvoie false sans trace (l. 905). `replaceAll` en mode local ignore l'échec (l. 921). `replaceAll` en mode Firebase n'a pas de `catch` (l. 957). Le fichier ne contient aucun `console`, ni `window.onerror`, ni `unhandledrejection`. Les changements de rôle, régénérations de code, exports et restaurations ne laissent aucune trace. Risque : un poste qui perd ses écritures ou une erreur d'import passent inaperçus. En cas de litige sur une prime ou de fuite de données, impossible de savoir qui a fait quoi.

**PROMPT CLAUDE** :

Contexte : Fit Pulse. Aucune remontée d'erreur, aucun journal des actions sensibles.

Changements :

1. Ajoute src/core/log.js :
  - `window.onerror` et `unhandledrejection` capturés ;
  - envoi vers `/logs/<club>/<AAAA-MM-JJ>/<id> = {at, uid, version, page, message, stack tronquée à 2000 caractères}` ;
  - limite de 20 envois par session ;
  - aucune donnée d'adhérent dans le message (remplace les chiffres de plus de 6 caractères par #).Règles : écriture seule pour un membre authentifié, lecture admin, validation de taille.
2. Journal d'audit : `audit/<club>/<id> = {at, by, action, target}`, avec action parmi `role_change`, `code_regen`, `member_archive`, `export_csv`, `export_json`, `restore`, `import`, `import_cancel`, `client_erase`, `purge`. Règle : création seule (`!data.exists()`), `by === auth.uid`, aucune modification ni suppression. Écran « Journal » réservé au créateur, filtrable par action et par date.
3. Les échecs d'écriture (`catch` existants l. 956 et 975) écrivent aussi un log et affichent un toast persistant avec « Réessayer ».
4. Affiche la version de l'application (constante injectée par le build) dans le profil.
5. Firebase : active les sauvegardes quotidiennes de RTDB (offre Blaze). Documente dans docs/exploitation.md la procédure de restauration et une alerte budget à 10 euros par mois.

Critères d'acceptation :

- `throw new Error('test')` dans la console crée une entrée sous `/logs`.
- Changer un rôle crée une entrée audit. Un membre ne peut pas supprimer une entrée audit (test de règles).
- docs/exploitation.md décrit la restauration pas à pas.

## 14. IMPORTANT : ARCHITECTURE CIBLE ET PLAN DE MIGRATION SANS

## CASSER L'EXISTANT

Constat : la structure plate actuelle (collections globales avec un champ `clubId`, l. 878 à 888) empêche des règles de lecture par club. RTDB ne filtre pas une lecture sur la valeur d'un champ. L'identité repose sur l'e-mail (l. 1933) et non sur l'`uid` Firebase. Deux comptes partagent la même adresse (l. 731 et 732). Risque : tant que la structure ne change pas, un membre du club A lit les adhérents et les impayés du club B. Une migration brutale ferait perdre des saisies en cours de mois.

Cible pragmatique : Firebase Auth (e-mail vérifié), RTDB en région europe-west1 avec un arbre

`/v2/clubs/{clubId}/...`, des rôles par club et par `uid`, Firebase Hosting, Storage pour les images, une Cloud Function de purge nocturne. Rester sur RTDB évite de réécrire la couche de données. Firestore n'apporte pas assez à cette taille pour justifier la réécriture.

### PROMPT CLAUDE :

Contexte : Fit Pulse, après les points 1 (règles d'étape 0), 11 (modules) et 12 (tests). Données actuelles sous `/pulse`, structure plate avec un champ `clubId`. Objectif : passer à `/v2` avec isolation par club, sans interruption.

1. Modèle cible, à produire dans docs/architecture.md avec un schéma :

```
/v2/admins/{uid}: true
/v2/profiles/{uid}: {first, last, avatar, email}
/v2/invites/{emailKey}/{clubId}: 'manager' | 'membre'
/v2/settings/kpis/{kpiId}
/v2/clubs/{clubId}/info
/v2/clubs/{clubId}/members/{uid}: 'manager' | 'membre'
/v2/clubs/{clubId}/{entries, targets, imports, monthly, base, clients, loyalty, resiliations, recov,
challenges, chat, reactions, paliers, rsm, tasks, audit}
/v2/prefs/{uid}
/v2/chatAll/{id} (canal inter-clubs)
```

2. database.rules.json v2 complet, avec ces expressions :

```
isAdmin = root.child('v2/admins/'+auth.uid).val() === true
isMember = root.child('v2/clubs/'+$clubId+'/members/'+auth.uid).exists()
isManager = root.child('v2/clubs/'+$clubId+'/members/'+auth.uid).val() === 'manager'
- lecture de /v2/clubs/$clubId : auth.token.email_verified === true && (isAdmin || isMember) ;
- members/$uid : écriture par isAdmin. isManager peut écrire si newData.val() === 'membre' et que
l'ancienne valeur est vide ou 'membre'. L'utilisateur lui-même peut écrire si !data.exists() &&
root.child('v2/invites/'+auth.token.email.toLowerCase().replace('.', ',')+'/'+$clubId).val() ===
newData.val() (réclamation d'invitation) ;
- mêmes règles d'écriture par collection qu'à l'étape 0, transposées avec auth.uid ;
- /pulse passe en lecture seule pour les admins, sans écriture.
```

3. tools/migrate-v1-v2.js (Admin SDK, Node) :

```
- lit /pulse ;
- crée les comptes Auth manquants, ou associe l'uid existant à chaque e-mail ;
- pour une adresse portant deux comptes (createur et manager), garde un seul uid avec le rôle le plus
élevé ;
- réécrit les userId des saisies vers les uid ;
- répartit chaque collection sous /v2/clubs/{clubId} ;
- écrit par lots de 500 ;
- mode --dry-run qui affiche les comptes par collection et par club ;
- mode --verify qui recalcule les scores du mois avec src/core/calc.js sur v1 et v2 et compare.
```

4. Client : un adaptateur `src/store/v2.js` expose la même interface que `db` (set, batch, onChange). L'application bascule par un drapeau `DATA_VERSION` dans `config.js`. Les écoutes se font par club courant (point 8).

5. Plan de bascule, à écrire dans docs/migration.md :

```
J0 règles étape 0 et vérification des e-mails ;
J+7 modules et tests en staging ;
J+14 migration à blanc sur une copie de prod (--dry-run puis --verify) ;
J+21, un dimanche soir : gel des écritures v1 (règles), migration, --verify, DATA_VERSION = 2,
```

déploiement ;  
retour arrière possible en 5 minutes (DATA\_VERSION = 1 et règles v1 réactivées) pendant 14 jours ;  
J+35 suppression de /pulse après un export chiffré conservé 30 jours.

Critères d'acceptation :

- Tests de règles : un membre du club A ne peut lire ni /v2/clubs/B/clients ni /v2/clubs/B/entries.
- --verify affiche 0 écart de score et 0 écart de points pour tous les utilisateurs et tous les mois.
- Le nombre de documents par collection est identique entre v1 et v2 (sortie --dry-run).
- Le smoke test Playwright passe avec DATA\_VERSION = 1 et avec DATA\_VERSION = 2.

# FEUILLE DE ROUTE

Six sprints d'environ une semaine. Lancez les prompts dans l'ordre indiqué (numéro de section, puis numéro de point), un prompt par session Claude Code, et validez les critères d'acceptation avant de passer au suivant.

1. **SPRINT 1 : SÉCURITÉ ET CONFIANCE.** 8.1, 8.2, 8.3, 8.4 (base verrouillée, XSS, écritures qui écrasent), puis 5.1, 5.2, 5.3, 5.5. Critère de sortie : un membre ne peut plus écrire hors de ses saisies, et les fichiers de test des bugs confirmés donnent les bons totaux.
2. **SPRINT 2 : CHIFFRES JUSTES PARTOUT.** 5.4, 5.6 à 5.10, 6.2 (rythme en jours ouvrés), 7.0 (socle valeur), puis 5.15 (page Contrôle qualité et tests). Critère : le même KPI affiche la même valeur sur l'accueil, le tableau de bord, le classement et le récap.
3. **SPRINT 3 : NOUVELLE DA.** 1.1 (charte d'écriture), 1.2, 1.3, 1.4, 1.5, 1.11, 1.12. En parallèle, générer les visuels avec les prompts ChatGPT de 1.6 à 1.9, puis les intégrer. Critère : zéro tiret cadratin et zéro emoji dans l'interface, un seul jaune, une seule feuille de styles.
4. **SPRINT 4 : LES COMMERCIAUX.** 2.1, 2.2, 2.3, 2.4, 2.5, puis 3.1 à 3.7 (modèle de relance unique, file d'appels, appel en un tap, résultat et prochaine action, cadences, scripts). Critère : une vente saisie en 2 taps, une relance appelée et notée en moins de 30 secondes sur téléphone.
5. **SPRINT 5 : ALERTES ET MANAGER.** 4.2 à 4.7 (PWA, push, moteur de règles, alertes planifiées), puis 6.1, 6.3, 6.4, 6.5, 6.7 (vue équipe, barème, fiche coaching, activité, rituels). Critère : le push "Appels du jour" arrive à 10 h, le manager lit l'état de son équipe en un écran.
6. **SPRINT 6 : REVENUS.** 7.1, 7.2, 7.3 (entonnoir, sauvetage en euros, page Opportunités), puis 7.4 à 7.11, 3.8 à 3.14, 4.8 à 4.12, 6.6 et 6.8 à 6.13, 2.6 à 2.14, 8.5 à 8.14. Critère : chaque matin, l'accueil donne les 5 actions qui rapportent le plus, en euros.

## INDICATEURS DE RÉUSSITE À SUIVRE

| Indicateur | Cible à 3 mois |

| --- | --- |

| Écart Fit Pulse contre Resamania sur les contrats du mois | 0 |

| Commerciaux actifs dans l'appli chaque jour ouvré | 90 % |

| Relances notées par commercial et par semaine | 40 |

| Délai médian de régularisation d'un impayé | moins de 15 jours |

| Taux de sauvetage des résiliations | +10 points |

| Taux de transformation prospect vers contrat (cohorte) | +5 points |

Avant de lancer le sprint 1, une vérification : le fichier de règles Firebase cité dans le code

(`club/database.rules.pulse.json`) n'était pas disponible pour l'audit. Le point 8.1 suppose qu'il donne tous les droits à toute adresse de l'équipe.